

Informe Final del Sistema ANE-HX

Documentación técnica, decisiones de diseño, pruebas y transferencia de conocimiento

Equipo de desarrollo

Martín Ramírez Espinosa

Mateo Almeida Gómez

David Ramírez Betancourth

Oscar Andres Gutierrez Estepa

Aldemar Giraldo Pava

2 de junio de 2026

Índice general

1. Introducción general	12
1.1. Propósito y contexto del proyecto	12
1.2. Problema o necesidad principal	12
1.3. Alcance del documento	13
1.4. Resumen de la arquitectura y los módulos principales	13
1.4.1. Capítulo 2: Análisis de requerimientos, restricciones y uso	14
1.4.2. Capítulo 4: Hardware	14
1.4.3. Capítulo 5: Software del sensor	14
1.4.4. Capítulo 6: Plataforma web en la nube	14
1.4.5. Capítulo 7: Análisis espectral y localización en la nube	15
1.4.6. Anexo A: Técnica de desarrollo con agentes IA	15
1.5. Convenciones del documento	15
2. Análisis de requerimientos, restricciones y uso	16
2.1. Propósito del capítulo	16
2.2. Contexto dentro del sistema general	16
2.3. Desarrollo técnico: requerimientos por categoría	17
2.3.1. Requerimientos de arquitectura	17
2.3.2. Requerimientos funcionales	17
2.3.3. Requerimientos de red	20
2.3.4. Requerimientos de hardware	20
2.3.5. Requerimientos de exactitud y confiabilidad	21

2.3.6. Requerimientos de interoperabilidad	21
2.4. Entradas, salidas e interfaces	21
2.5. Decisiones de diseño o implementación	22
2.6. Problemas presentados y ajustes realizados	23
2.7. Validación, pruebas o evidencias	24
2.8. Aprendizajes y transferencia de conocimiento	24
2.9. Limitaciones	25
2.10. Recomendaciones específicas	25
3. Diseño general del sistema	27
3.1. Propósito del capítulo	27
3.2. Contexto dentro del sistema general	27
3.3. Desarrollo técnico	27
3.4. Entradas, salidas e interfaces	27
3.5. Decisiones de diseño o implementación	27
3.6. Problemas presentados y ajustes realizados	28
3.7. Validación, pruebas o evidencias	28
3.8. Aprendizajes y transferencia de conocimiento	28
3.9. Limitaciones	28
3.10. Recomendaciones específicas	28
4. Hardware	29
4.1. Propósito del capítulo	29
4.2. Contexto dentro del sistema general	29
4.3. Desarrollo técnico	32
4.3.1. Arquitectura física del nodo	32
4.3.2. Ruta RF y mux de antenas	33
4.3.3. Árbol de potencia	34
4.4. Entradas, salidas e interfaces	36
4.5. Decisiones de diseño o implementación	36

4.6. Problemas presentados y ajustes realizados	37
4.7. Validación, pruebas o evidencias	38
4.8. Aprendizajes y transferencia de conocimiento	38
4.9. Limitaciones	39
4.10. Recomendaciones específicas	39
5. Software del sensor	40
5.1. Propósito del capítulo	40
5.2. Contexto dentro del sistema general	41
5.3. Desarrollo técnico	42
5.3.1. Adquisición	42
5.3.2. Uso de buffers	43
5.3.3. Preprocesamiento	43
5.3.4. Procesamiento	44
5.3.5. Postprocesamiento	46
5.3.6. Calibración en frecuencia	47
5.3.7. Orquestador	48
5.4. Entradas, salidas e interfaces	49
5.4.1. Entradas	49
5.4.2. Salidas	49
5.4.3. Interfaces y protocolos	50
5.5. Decisiones de diseño o implementación	51
5.6. Problemas presentados y ajustes realizados	52
5.7. Validación, pruebas o evidencias	52
5.8. Aprendizajes y transferencia de conocimiento	53
5.9. Limitaciones	54
5.10. Recomendaciones específicas	55
6. Plataforma web en la nube	57
6.1. Propósito del capítulo	57

6.2.	Contexto dentro del sistema general	57
6.3.	Desarrollo técnico	58
6.3.1.	Frontend	59
6.3.2.	Backend	61
6.3.3.	Base de datos	63
6.3.4.	Despliegue	64
6.4.	Entradas, salidas e interfaces	65
6.4.1.	Entradas	65
6.4.2.	Salidas	65
6.4.3.	Interfaces	66
6.5.	Decisiones de diseño o implementación	66
6.5.1.	Separación frontend/backend con API REST	66
6.5.2.	WebSocket sobre polling para tiempo real	67
6.5.3.	PostgreSQL y TimescaleDB para datos de series temporales	67
6.5.4.	Microservicio de postprocesamiento en Python	68
6.5.5.	Separación de canales WebSocket para datos y audio	68
6.6.	Problemas presentados y ajustes realizados	69
6.7.	Validación, pruebas o evidencias	71
6.8.	Aprendizajes y transferencia de conocimiento	72
6.9.	Limitaciones	73
6.10.	Recomendaciones específicas	74
6.11.	Repositorios	75
7.	Análisis espectral y localización en la nube	77
7.1.	1. Propósito de la sección	77
7.2.	2. Contexto del módulo dentro del sistema ANE-HX	77
7.3.	3. Arquitectura del módulo de postprocesamiento	78
7.4.	4. Flujo general de procesamiento	79
7.5.	5. Entradas del sistema	80

7.6.	6. Salidas del sistema	81
7.7.	7. Modos de operación	83
	7.7.1. Detección automática	84
	7.7.2. Análisis por picos	84
	7.7.3. Cumplimiento contra licencias	84
7.8.	8. Análisis espectral realizado	85
	7.8.1. Frecuencia central	85
	7.8.2. Ancho de banda	86
	7.8.3. Potencia	87
	7.8.4. MER/BER si aplica	88
7.9.	9. Comparación contra licencias y uso de DANE/municipio	88
	7.9.1. 1. Filtrado geográfico y asociación de licencia	89
	7.9.2. 2. Criterios de cumplimiento técnico	89
	7.9.3. 3. Manejo de múltiples jurisdicciones (Múltiples DANE)	89
7.10.	10. Localización administrativa	90
	7.10.1. 1. Origen del parámetro territorial	90
	7.10.2. 2. Reglas de interpretación y jerarquía de prioridad	91
	7.10.3. 3. Normalización de texto y emparejamiento (Matching)	92
7.11.	11. Interfaces del módulo	92
7.12.	12. Decisiones de diseño	92
	7.12.1. Estimación de Frecuencia Central (F_c)	93
	7.12.2. Estimación de Ancho de Banda (BW)	93
	7.12.3. Estimación de Potencia	93
	7.12.4. Estimación de MER y BER	94
7.13.	13. Problemas encontrados y ajustes realizados	94
	7.13.1. Evolución de la detección del piso de ruido	94
	7.13.2. Ajuste final por restricciones de Hardware (Raspberry Pi 5)	95
7.14.	14. Pruebas o validaciones sugeridas	95
	7.14.1. 1. Estrategia de pruebas automatizadas	96

7.14.2. 2. Métricas de evaluación recomendadas	96
7.15. 15. Limitaciones actuales	97
7.15.1. 1. Restricciones de capacidad de procesamiento	98
7.15.2. 2. Saturación del canal de transmisión (Ancho de Banda)	98
7.15.3. 3. Estimación del umbral en entornos con baja SNR	98
7.16. 16. Recomendaciones de mejora	98
7.16.1. Mejora de la infraestructura de comunicaciones	98
7.16.2. Exploración de algoritmos basados en Inteligencia Artificial (IA) . .	99
7.16.3. Actualización del hardware de procesamiento (Hardware Upgrade) .	99
7.17. 17. Cierre técnico de la sección	99
8. Protocolos de prueba edge-cloud	101
8.1. Objetivo del capítulo	101
8.2. Alcance de las pruebas	101
8.3. Arquitectura bajo prueba	102
8.4. Criterios generales de aceptación	103
8.5. Estrategia general de pruebas	104
8.5.1. Pruebas funcionales	104
8.5.2. Pruebas de integración edge	104
8.5.3. Pruebas de integración edge-cloud	104
8.5.4. Pruebas metrológicas y espectrales	104
8.5.5. Pruebas pedagógicas	105
8.6. Matriz general de pruebas	105
8.7. Protocolos detallados de prueba	108
8.7.1. PR-EC-001: registro e identificación del sensor	108
8.7.2. PR-EC-002: heartbeat y salud del sensor	109
8.7.3. PR-EC-003: configuración de adquisición SDR	110
8.7.4. PR-EC-004: validación de RBW	111
8.7.5. PR-EC-005: adquisición IQ y generación de PSD	111

8.7.6. PR-EC-006: detección de emisiones por umbral	112
8.7.7. PR-EC-007: medición de ancho de banda ocupado	112
8.7.8. PR-EC-008: programación y ejecución de campañas	113
8.7.9. PR-EC-009: recuperación ante pérdida de conectividad	113
8.7.10. PR-EC-010: exportación de datos	114
8.8. Pruebas metrológicas recomendadas	115
8.8.1. DANL	115
8.8.2. Exactitud de frecuencia	115
8.8.3. Precisión de amplitud	116
8.8.4. Rango dinámico	116
8.9. Pruebas de instalación y operación de campo	116
8.10. Problemas, decisiones y ajustes esperados	117
8.11. Evidencias mínimas por prueba	118
8.12. Transferencia de conocimiento	119
8.13. Recomendaciones para futuras pruebas	120
8.14. Conclusiones del capítulo	121
9. Integración general del sistema	122
9.1. Propósito del capítulo	122
9.2. Contexto dentro del sistema general	122
9.3. Desarrollo técnico	122
9.4. Entradas, salidas e interfaces	122
9.5. Decisiones de diseño o implementación	122
9.6. Problemas presentados y ajustes realizados	123
9.7. Validación, pruebas o evidencias	123
9.8. Aprendizajes y transferencia de conocimiento	123
9.9. Limitaciones	123
9.10. Recomendaciones específicas	123
10. Conclusiones generales	124

10.1. Síntesis de resultados técnicos	124
10.2. Logros del diseño e implementación	124
10.3. Resultados de la integración	124
10.4. Lecciones aprendidas	124
10.5. Cumplimiento de objetivos	124
11.Recomendaciones	125
11.1. Mejoras de arquitectura	125
11.2. Mejoras de hardware	125
11.3. Mejoras de software	125
11.4. Mejoras de documentación	125
11.5. Reducción de riesgos identificados	125
12.Referencias	126
A. Material de apoyo: uso de agentes de IA para documentación, desarrollo y revisión técnica del proyecto	127
A.1. Nota de alcance y propósito	127
A.2. Contexto y diferenciación funcional	127
A.3. Metodología de trabajo: Ecosistema de Agentes	128
A.3.1. Habilidad del Entorno (Ejemplos CLI)	128
A.3.2. Arquitectura Multi-Agente Asistida	128
A.4. Gobernanza y Uso Responsable de IA	130
A.5. Trazabilidad Metodológica	130
A.6. Flujo de Trabajo Colaborativo (Paso a Paso)	130
A.7. Trazabilidad de Prompts y Relación con GitHub	131
A.8. Problemas, Limitaciones y Ajustes	131
A.9. Métricas de Utilidad y Evidencias	132
A.10.Conclusión del capítulo	132
B. Diagramas adicionales	134

B.1. Diagramas de arquitectura extendida	134
B.2. Diagramas de despliegue y conectividad	134
B.3. Secuencias operativas	134
B.4. Esquemas detallados de componentes	134
C. Manuales o guías de uso	135
C.1. Requisitos previos de instalación	135
C.2. Instrucciones de operación cotidiana	135
C.3. Solución de problemas frecuentes	135
C.4. Recomendaciones para mantenimiento	135
D. Resultados de pruebas	136
D.1. Tablas de métricas y mediciones	136
D.2. Evidencias visuales de ejecución	136
D.3. Comparación entre escenarios de prueba	136
D.4. Observaciones y anomalías detectadas	136

Índice de figuras

4.1. Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia. . . .	31
4.2. Módulos principales del hardware ANE2. Fuente: elaboración propia. . . .	33
4.3. Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en Mon-RaF2. Fuente: elaboración propia.	34
6.1. Diagrama general de la arquitectura de la plataforma web. Fuente: Elaboración propia.	59
A.1. Diseño conceptual del sistema multi-agente: Relación metodológica entre roles de asistencia. Fuente: Elaboración propia.	129
A.2. Flujo de trabajo y validación humana en la integración de resultados. Fuente: Elaboración propia.	132

Índice de tablas

2.1. Entradas y salidas principales del sistema ANE-HX.	22
2.2. Problemas encontrados en la implementación de requerimientos.	23
4.1. Inventario funcional de componentes de hardware del nodo ANE2.	32
4.2. Criterios mínimos para una versión robusta del árbol de potencia.	35
4.3. Interfaces principales del hardware ANE2.	36
4.4. Decisiones de diseño de hardware y justificación.	36
4.5. Problemas presentados, decisiones y ajustes realizados en hardware.	37
4.6. Resumen de telemetría eléctrica y térmica inspeccionada.	38
6.1. Problemas presentados, decisiones y ajustes realizados en la plataforma web.	69
6.2. Repositorio asociado al frontend de la plataforma web.	76
6.3. Repositorio asociado al backend de la plataforma web.	76
8.1. Matriz general de pruebas edge-cloud.	105
8.2. Problemas frecuentes durante las pruebas edge-cloud.	118
8.3. Aprendizajes transferibles asociados a las pruebas.	120
A.1. Trazabilidad de actividades de apoyo con IA.	130
A.2. Gestión de problemas y limitaciones de la IA.	131

Capítulo 1

Introducción general

1.1. Propósito y contexto del proyecto

Este documento constituye el informe final del sistema de monitoreo espectral **ANE-HX**, desarrollado como proyecto académico de ingeniería para la Agencia Nacional del Espectro (ANE) de Colombia. El sistema integra componentes de hardware, software de adquisición por radio definida por software (SDR), procesamiento en borde, plataforma web en la nube y algoritmos de análisis espectral para construir una solución funcional de monitoreo del espectro radioeléctrico.

El informe se organiza como memoria técnica del sistema, registro de decisiones de diseño, evidencia de pruebas realizadas e instrumento de transferencia de conocimiento hacia futuros equipos que deban mantener, reproducir o extender el desarrollo.

1.2. Problema o necesidad principal

El monitoreo del espectro radioeléctrico requiere capturar señales en campo, transportarlas a un entorno de análisis, extraer parámetros técnicos de las emisiones detectadas y contrastarlos contra información regulatoria. Un sistema que resuelva este flujo de extremo a extremo debe coordinar hardware de adquisición, conectividad celular, almacenamiento en la nube, algoritmos de procesamiento espectral y una interfaz para operadores.

El proyecto ANE-HX aborda esta cadena completa mediante una arquitectura modular que separa el sensado en borde, la transmisión de datos, la persistencia y visualización en la nube, y el postprocesamiento analítico. Cada módulo se documenta en un capítulo independiente.

1.3. Alcance del documento

El presente informe cubre el diseño, la implementación, las pruebas y los aprendizajes obtenidos durante el desarrollo del sistema. No se incluye el código fuente completo de los módulos de software, el cual reside en repositorios independientes referenciados en cada capítulo.

Los capítulos que contienen contenido técnico desarrollado son:

- Capítulo 2: Análisis de requerimientos, restricciones y uso.
- Capítulo 4: Hardware del nodo de sensado espectral.
- Capítulo 5: Software del sensor (adquisición SDR y preprocesamiento en borde).
- Capítulo 6: Plataforma web en la nube (frontend, backend, base de datos y despliegue).
- Capítulo 7: Análisis espectral y localización en la nube (postprocesamiento de capturas).
- Anexo A: Técnica de desarrollo colaborativo usando agentes de inteligencia artificial.

Los capítulos restantes (3, 8, 9, 10, 11, 12 y anexos B, C, D) contienen la estructura prevista como placeholders para completar la documentación de diseño general, pruebas, integración, conclusiones, recomendaciones, referencias y material complementario.

1.4. Resumen de la arquitectura y los módulos principales

El sistema ANE-HX se organiza en tres dominios:

1. **Borde:** un nodo físico compuesto por una Raspberry Pi 5, un SDR HackRF One, una tarjeta auxiliar MonRaF2 (concentrador RF, LTE, GNSS y multiplexor de antenas) y antenas RF. El software embebido controla la adquisición, ejecuta preprocesamiento y transmite productos espectrales hacia la nube mediante conectividad celular.
2. **Nube:** una plataforma web con frontend SPA (React), backend de orquestación (Node.js), base de datos PostgreSQL con extensión TimescaleDB para series temporales, y un microservicio de postprocesamiento espectral en Python.
3. **Usuarios:** personas operadoras y administradoras que acceden a la plataforma web para visualizar métricas en tiempo real, gestionar campañas de medición, consultar reportes de cumplimiento normativo y administrar sensores.

1.4.1. Capítulo 2: Análisis de requerimientos, restricciones y uso

Consolida los requerimientos funcionales y no funcionales pactados en el pliego técnico del proyecto. Organiza las especificaciones por categoría —parametrización, medición, cumplimiento normativo, gestión, análisis forense, red, hardware, exactitud e interoperabilidad— y las contrasta con el estado de implementación documentado en los capítulos técnicos. Incluye una matriz de problemas encontrados durante la implementación de los requerimientos y sus ajustes correspondientes.

1.4.2. Capítulo 4: Hardware

Documenta el diseño físico del nodo ANE2. Describe la arquitectura de potencia, la ruta RF desde las antenas hasta el SDR, el árbol de alimentación, la conectividad GPIO entre la Raspberry Pi 5 y la MonRaF2, y las mediciones de voltaje y temperatura bajo carga. Incluye decisiones de diseño como la separación de la fuente de potencia del cómputo frente a los periféricos SDR y LTE, y una matriz de problemas encontrados con sus respectivos ajustes.

1.4.3. Capítulo 5: Software del sensor

Describe la arquitectura del código que se ejecuta en el nodo de borde. Cubre el control de la HackRF One, la selección de antena, la adquisición de muestras IQ, el preprocesamiento (balance IQ, corrección DC, estimación de PSD, calibración en frecuencia), la demodulación de FM y AM, y la comunicación con el backend. El software opera mediante un orquestador que gestiona modos de operación como campañas programadas, PSD en tiempo real y calibración.

1.4.4. Capítulo 6: Plataforma web en la nube

Documenta la capa central de coordinación e interfaz de usuario. El frontend, construido con React y Vite, ofrece autenticación, tableros de monitoreo con gráficas en tiempo real vía WebSocket, gestión de sensores y generación de reportes de cumplimiento. El backend, basado en Node.js y Express, expone una API REST, maneja WebSockets para telemetría y audio, y orquesta las campañas de medición. La base de datos TimescaleDB almacena series temporales de métricas espectrales y coordenadas GPS.

1.4.5. Capítulo 7: Análisis espectral y localización en la nube

Describe el módulo de postprocesamiento encargado de convertir capturas espectrales crudas en parámetros técnicos interpretables: frecuencia central, ancho de banda, potencia, MER y BER para señales TDT. El módulo implementa tres modos de operación (detección automática, análisis por picos y cumplimiento contra licencias), utiliza un algoritmo de linealización con umbral fijo optimizado para la Raspberry Pi 5, y soporta filtrado geográfico mediante códigos DANE y nombre de municipio para la validación normativa contra bases de licencias oficiales.

1.4.6. Anexo A: Técnica de desarrollo con agentes IA

Documenta la metodología de trabajo colaborativo del equipo utilizando agentes de inteligencia artificial (Gemini CLI y GitHub Copilot CLI). Describe la instalación, la arquitectura multi-agente basada en roles, la metodología de prompting estructurada (ROLE → OBJECTIVE → CONTEXT → REASONING), los problemas encontrados durante la colaboración con IA y las lecciones aprendidas.

1.5. Convenciones del documento

El informe está redactado en español, compilado con L^AT_EX y utiliza codificación UTF-8. Cada capítulo sigue una estructura homogénea: propósito, contexto en el sistema, desarrollo técnico, entradas y salidas, decisiones de diseño, problemas y ajustes, validación, aprendizajes, limitaciones y recomendaciones. Las referencias a repositorios de software incluyen el enlace real o la indicación explícita de que el repositorio está pendiente de publicación. No se incluye código fuente extenso dentro del documento.

Capítulo 2

Análisis de requerimientos, restricciones y uso

2.1. Propósito del capítulo

Este capítulo consolida los requerimientos funcionales y no funcionales pactados para el sistema ANE-HX, desarrollado para la Agencia Nacional del Espectro (ANE) de Colombia. El objetivo es establecer un marco de referencia verificable que permita evaluar el grado de cumplimiento alcanzado en cada módulo del sistema y servir como guía para futuras iteraciones de desarrollo.

Los requerimientos aquí documentados provienen del pliego técnico oficial del proyecto. Se presentan agrupados por categoría funcional y se contrastan con el estado de implementación documentado en los capítulos técnicos de hardware (Capítulo 4), software del sensor (Capítulo 5), plataforma web (Capítulo 6) y análisis espectral (Capítulo 7).

2.2. Contexto dentro del sistema general

El sistema ANE-HX responde a la necesidad institucional de la ANE de monitorear el espectro radioeléctrico colombiano de forma automatizada, remota y trazable. La arquitectura del sistema se organiza en tres dominios que cubren el flujo completo desde la captura de señales en campo hasta la generación de reportes de cumplimiento:

1. **Borde:** nodos de sensado desplegados en campo que adquieren señales de RF, ejecutan preprocesamiento local y transmiten productos espectrales hacia la nube mediante conectividad celular, WiFi o Ethernet.
2. **Nube:** plataforma central que recibe, almacena, procesa y visualiza los datos espec-

trales. Incluye el backend de orquestación, la base de datos de series temporales, el frontend de usuario y el módulo de postprocesamiento espectral.

3. **Usuarios:** personal técnico de la ANE que opera el sistema mediante la plataforma web, configura campañas de medición, consulta resultados y genera reportes de cumplimiento normativo.

Los requerimientos se distribuyen a lo largo de esta cadena: la parametrización de adquisición y las capacidades de medición recaen en el software del sensor (borde); la visualización, la gestión de campañas y la generación de reportes residen en la plataforma web (nube); la detección de emisiones, el cumplimiento normativo y el análisis forense se implementan en el módulo de postprocesamiento (nube).

2.3. Desarrollo técnico: requerimientos por categoría

A continuación se presentan los requerimientos finales del sistema, organizados según las categorías definidas en el pliego técnico.

2.3.1. Requerimientos de arquitectura

Categoría	Requerimiento
Seguridad de la información	Toda la información en tránsito entre el sensor y la interfaz de monitoreo debe usar SSL/TLS.

2.3.2. Requerimientos funcionales

Parametrización de adquisición

Parámetro	Especificación	Restricción
Frecuencia central	Desde 1 MHz hasta 6 GHz	Límite del hardware SDR
Sample Rate	Hasta 20 MS/s, mínimo 200 kS/s	Seleccionable por el usuario
Ancho de banda instantáneo	Máximo 20 MHz, mínimo 2 MHz	Determinado por el sample rate
Redundancia GPS	Ubicación manual adicional	Conserva la reportada por el GPS

Parametrización de visualización

Parámetro	Especificación
RBW	Ajustable mediante selección de puntos FFT. $RBW = SampleRate/FFTSize$
VBW	Suavizado del espectro en niveles: bajo, medio, alto
Filtros por software	Pasabajo, pasabanda, pasaalto con frecuencia central, ancho de banda y activación/desactivación
Resolución	Pasos de 500 Hz, resolución de hasta 10 Hz
Unidades de medida	Frecuencia: Hz, kHz, MHz, GHz. Potencia: dBm (prioritario), dBmV, dBuV, V, W. Campo eléctrico: dBuV/m (prioritario), dBmV/m, mV/m, W/m ² , dBW/m ² /MHz
Umbral de detección	Configurable a partir de 3 dB, 5 dB u otro valor definido por el usuario

Medición, visualización y audio

Función	Especificación
Indicadores espectrales	Traza máxima, traza mínima, promedio, RMS, máximo y mínimo
Marcadores	Al menos 4 marcadores individuales y tipo delta, con cálculo de delta de potencia y delta de frecuencia
Ancho de banda	Método $-X$ dB (X seleccionable: 3, 10 o 26 dB) y OBW (99 % de potencia)
Potencia de canal	Para emisiones de hasta 20 MHz de ancho de banda
Demodulación FM/AM	Demodular y escuchar emisiones FM y AM. Medir profundidad de modulación AM y excursión de frecuencia FM

Cumplimiento normativo

Verificación	Especificación
Frecuencias sin uso	Detectar ausencia de emisiones autorizadas en bandas con licencia asignada
Emisiones no autorizadas	Detectar emisiones no registradas en la base de datos de operadores autorizados del MinTIC
Incumplimiento de ancho de banda	Detectar señales que excedan los límites de ancho de banda autorizados por servicio
Radiodifusión sonora	Recolección de datos para identificación de intermodulaciones en la banda 88–108 MHz
Servicio móvil aeronáutico	Detección de emisiones desviadas en la banda 108–137 MHz

Herramientas de gestión

Función	Especificación
Monitoreo de sensores	Alertas sobre violación de umbrales de variables del sensor: disco, RAM, CPU
Monitoreo de conectividad	Calidad de conexión IP mediante RTT y estado del canal (alive/not alive)
Agenda de programaciones	Publicar agenda de escaneos, mantener sesiones privadas y permitir cancelación por rol supervisor
Generación de reportes	Al menos 1 reporte con variables definidas por la entidad, entregado en CSV

Análisis forense y almacenamiento

Función	Especificación
Referenciación de información	Timestamp + geotags + identificador del sensor para cada medición
Almacenamiento en sensor	Archivos de muestreo (.cs8) en el almacenamiento local disponible
Almacenamiento centralizado	Resultados de monitoreo programado en servidores institucionales de la ANE

Funciones avanzadas y soporte

Función	Especificación
Datasets para análisis	Recolección y organización de datos para procesos de análisis posteriores y entrenamiento de modelos
Acceso remoto	Acceso vía IP a sensores para actualización de software y aplicación de parches

2.3.3. Requerimientos de red

Requerimiento	Especificación
WiFi	Habilitar conexión inalámbrica
Ethernet	Puerto de red cableado conservando características ambientales del nodo
Enrutamiento automático	Seleccionar automáticamente el canal de comunicación disponible (Ethernet, WiFi o celular)

2.3.4. Requerimientos de hardware

Requerimiento	Especificación
GPS	Conexión móvil mediante SIM card institucional. No depender de tecnologías propietarias. Precisión horizontal ≤ 10 m, vertical ≤ 15 m (estándar L1 GNSS civil)
Multiplexación de antenas	Board que permita usar 4 antenas distintas en procesos de adquisición espectral
Kit de antenas	Tres antenas por sensor: (1) TDT, (2) telefonía móvil Down link ($f > 2$ GHz), (3) VHF/UHF y frecuencias intermedias
EMC	Optimizar susceptibilidad electromagnética para evitar interferencias que falseen mediciones

2.3.5. Requerimientos de exactitud y confiabilidad

Requerimiento	Especificación
Sincronización NTP	Sincronización a reloj de red NTP en cada sensor para etiquetado temporal de muestreos
Calibración	Procedimientos de calibración de frecuencia, ganancia de recepción, balance IQ y corrección de offset, verificados con equipos certificados
DANL	Determinar el nivel más bajo de señal detectable sin señal presente
Rango dinámico	Determinar la diferencia entre la señal máxima sin distorsión y la mínima detectable sobre ruido
Exactitud de frecuencia	Determinar la diferencia máxima aceptable entre frecuencia real y medida
Precisión de amplitud	Determinar el margen de error en la medición de potencia, expresado en dB

2.3.6. Requerimientos de interoperabilidad

Requerimiento	Especificación
Exportación de datos	Escaneos descargables en formato CSV, JSON y XML para procesamiento offline

2.4. Entradas, salidas e interfaces

El sistema ANE-HX opera como una plataforma integral que recibe señales de RF en los nodos de borde y entrega información procesada a los operadores mediante la interfaz web. La Tabla 2.1 resume las entradas y salidas principales.

Tabla 2.1: Entradas y salidas principales del sistema ANE-HX.

Nivel	Entradas	Salidas
Borde	Señales RF (1 MHz – 6 GHz), parámetros de campaña, comandos de control remoto	Muestras IQ, PSD, métricas espectrales, audio demodulado, telemetría del sensor, coordenadas GPS
Nube	Datos espectrales desde sensores, parámetros de postprocesamiento, bases de licencias del MinTIC	Visualizaciones en tiempo real, reportes de cumplimiento (CSV), datos exportables (CSV/JSON/XML), alertas
Usuario	Credenciales de acceso, parámetros de configuración, solicitudes de reporte	Interfaz web con tableros, gráficas, reportes descargables, agenda de campañas

2.5. Decisiones de diseño o implementación

La definición de los requerimientos condicionó las siguientes decisiones de diseño del sistema:

- **Arquitectura borde-nube:** la exigencia de monitoreo remoto con acceso vía IP y transmisión celular determinó la separación entre adquisición local (Raspberry Pi 5 + HackRF One) y procesamiento centralizado en servidores institucionales de la ANE.
- **Modularidad del software:** los requerimientos de parametrización, medición, cumplimiento normativo y gestión se implementaron como módulos independientes (sensor SDR, plataforma web, postprocesamiento espectral) que se comunican mediante API REST y WebSocket.
- **Selección del SDR:** el HackRF One se eligió por su rango de operación (1 MHz – 6 GHz) y ancho de banda instantáneo (hasta 20 MHz), que cubren exactamente las bandas requeridas. Su naturaleza de código abierto evita dependencias de tecnologías propietarias.
- **Base de datos temporal:** la necesidad de almacenar y consultar series temporales de métricas espectrales con timestamp y geolocalización motivó la adopción de PostgreSQL con extensión TimescaleDB.
- **Postprocesamiento en Python:** los algoritmos de detección de emisiones, estimación de parámetros y comparación contra licencias se implementaron en Python por su ecosistema de librerías científicas (NumPy, SciPy) y su facilidad de integración como microservicio.

- **Umbral fijo con linealización:** las restricciones de CPU de la Raspberry Pi 5 llevaron a descartar algoritmos adaptativos costosos en favor de un umbral fijo precedido por una corrección de linealidad del sistema, como se detalla en el Capítulo 7.

2.6. Problemas presentados y ajustes realizados

Tabla 2.2: Problemas encontrados en la implementación de requerimientos.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión del nodo bajo carga SDR/LTE	Caídas de voltaje en EXT5V con HackRF y módem LTE activos simultáneamente	La Raspberry Pi 5 no puede suministrar corriente suficiente a todos los periféricos por sus puertos	Diseñar árbol de potencia externo sobredimensionado con fuente dedicada	Operación estable del nodo con todos los periféricos activos
Falsos positivos en detección de emisiones	Umbral fijo producía detecciones espurias por curvatura del ruido del receptor	No linealidad del piso de ruido del hardware SDR	Implementar corrección de linealidad antes de aplicar umbral fijo	Reducción drástica de falsas alarmas con bajo costo computacional
Timeouts en generación de reportes	Errores 504 en frontend con grandes volúmenes de datos	Procesamiento y armado de reportes excedía el timeout del servidor	Extender timeouts a 3600 s en Node.js y Nginx	Reportes generados exitosamente incluso para campañas extensas
Saturación del canal de subida	Resolución y tasa de refresco limitadas en transmisiones desde borde	Ancho de banda del canal celular insuficiente para tramas de alta resolución	Ajustar resolución espectral según disponibilidad de canal	Compromiso aceptable entre resolución y capacidad de transmisión

2.7. Validación, pruebas o evidencias

La validación de los requerimientos se realizó de forma incremental a medida que cada módulo alcanzaba un estado funcional:

- **Parametrización de adquisición:** verificada mediante campañas de medición configuradas desde la plataforma web con distintas frecuencias centrales, sample rates y ganancias. Los resultados se validaron por inspección espectral en la interfaz de monitoreo en tiempo real.
- **Medición de ancho de banda y potencia:** contrastada contra señales de referencia conocidas (portadoras de FM comercial en la banda 88–108 MHz) cuyos parámetros nominales están documentados en las bases de licencias del MinTIC.
- **Cumplimiento normativo:** validado mediante el procesamiento de capturas reales contra un archivo CSV de licencias de prueba, verificando la correcta asociación por frecuencia, el filtrado por municipio y código DANE, y la emisión de resultados de cumplimiento.
- **Exportación de datos:** verificada la generación de archivos CSV con la estructura de columnas solicitada por la entidad.
- **Conectividad y monitoreo:** el sistema se desplegó en un servidor institucional de la ANE, validando la comunicación extremo a extremo entre el sensor en campo y la plataforma web, incluyendo el enrutamiento automático entre WiFi y Ethernet.

Las evidencias detalladas de cada prueba se documentan en el Capítulo 7 (pruebas del módulo de postprocesamiento) y en las secciones de validación de los capítulos 4, 5 y 6.

2.8. Aprendizajes y transferencia de conocimiento

- **Anticipar restricciones de potencia desde el inicio:** el cuello de botella más recurrente en el hardware fue la alimentación. La lección aprendida es que el árbol de potencia debe diseñarse antes de integrar los periféricos, no como una corrección posterior.
- **El cumplimiento normativo requiere datos administrativos limpios:** la variabilidad en nombres de municipios, códigos DANE y formatos de licencias obligó a implementar normalización de texto y reglas de compatibilidad. Esto debe considerarse desde la fase de diseño de datos.

- **El ancho de banda del canal celular condiciona la resolución espectral:** la capacidad de transmisión desde el borde es el factor limitante para la resolución de las capturas. Futuros despliegues deben dimensionar el canal de subida según la densidad espectral requerida.
- **Modularidad facilita la validación incremental:** la separación en módulos independientes (sensor, plataforma, postprocesamiento) permitió probar y validar cada requerimiento en el contexto del módulo que lo implementa, sin depender del sistema completo.

2.9. Limitaciones

- **MER/BER solo para TDT:** la estimación de calidad de señal (MER/BER) está implementada únicamente para señales de Televisión Digital Terrestre con modulación 64-QAM. No cubre otros servicios de radiodifusión o comunicaciones.
- **Detección de intermodulaciones:** la recolección de datos para identificación de intermodulaciones en la banda de radiodifusión sonora está iniciada pero no se ha completado el análisis automático.
- **Caracterización metrológica:** la determinación formal de DANL, rango dinámico y exactitud de amplitud requiere equipos de calibración certificados no disponibles durante el desarrollo. Las mediciones actuales son operativas, no metrológicas.
- **Almacenamiento de muestras crudas:** el almacenamiento de archivos .cs8 en el sensor está limitado por la capacidad del almacenamiento local de la Raspberry Pi 5 (tarjeta SD).
- **Acceso remoto:** el acceso remoto a sensores vía IP está implementado a nivel de sistema operativo (SSH) pero no integrado en la plataforma web como funcionalidad de usuario.

2.10. Recomendaciones específicas

- **Completar la caracterización metrológica:** realizar una campaña de calibración con equipos certificados para determinar formalmente DANL, rango dinámico, exactitud de frecuencia y precisión de amplitud del sistema.
- **Extender MER/BER a otros servicios:** ampliar la estimación de calidad de señal a modulaciones de radiodifusión sonora (FM) y servicios móviles, utilizando métricas como SINAD o profundidad de modulación.

- **Automatizar detección de intermodulaciones:** desarrollar el módulo de análisis automático de intermodulaciones a partir de los datasets ya recolectados en las bandas 88–108 MHz y 108–137 MHz.
- **Integrar acceso remoto en la plataforma:** incorporar en la interfaz web una funcionalidad de acceso remoto a sensores que permita actualizaciones de software y diagnóstico sin requerir acceso por SSH.
- **Ampliar almacenamiento local:** evaluar la migración a almacenamiento externo (USB o NAS) para aumentar la capacidad de retención de muestras crudas (.cs8) en el nodo de borde.
- **Implementar reportes adicionales:** desarrollar los formatos de reporte complementarios (JSON, XML) según la especificación de interoperabilidad, más allá del CSV actualmente implementado.

Capítulo 3

Diseño general del sistema

3.1. Propósito del capítulo

[[Placeholder]]

3.2. Contexto dentro del sistema general

[[Placeholder]]

3.3. Desarrollo técnico

[[Placeholder]]

3.4. Entradas, salidas e interfaces

[[Placeholder]]

3.5. Decisiones de diseño o implementación

[[Placeholder]]

3.6. Problemas presentados y ajustes realizados

[[Placeholder]]

3.7. Validación, pruebas o evidencias

[[Placeholder]]

3.8. Aprendizajes y transferencia de conocimiento

[[Placeholder]]

3.9. Limitaciones

[[Placeholder]]

3.10. Recomendaciones específicas

[[Placeholder]]

Capítulo 4

Hardware

4.1. Propósito del capítulo

Esta sección documenta el hardware del nodo de sensado espectral ANE2 con un criterio de transferencia de conocimiento. La intención no es solamente listar piezas, sino explicar cómo se organiza el montaje físico, qué función cumple cada módulo, cuáles interfaces deben respetarse y qué decisiones deben mantenerse en una nueva versión del producto.

La idea técnica central es la alimentación. La experiencia de integración mostró que concentrar la entrega de energía de los periféricos en la Raspberry Pi 5 reduce el margen operativo del nodo. Un diseño derivado o una versión posterior debe incluir un **árbol de potencia externo, sobredimensionado y medible**, capaz de alimentar cómputo, SDR, LTE/GNSS, mux RF y futuras extensiones sin depender del margen de los puertos de la Raspberry Pi.

4.2. Contexto dentro del sistema general

ANE2 es un nodo de borde para monitoreo espectral. En campo, el hardware debe recibir señales RF, seleccionar o enrutar la entrada de antena, digitalizar la señal mediante SDR, ejecutar procesamiento local, obtener ubicación y mantener conectividad con la plataforma. Por tanto, el hardware cumple cuatro responsabilidades:

- **Entrada y selección RF:** las antenas llegan a la tarjeta MonRaF2, donde se concentran rutas RF, filtros y una etapa tipo mux/switch controlada por GPIO.
- **Adquisición SDR:** el HackRF One opera como receptor y entrega muestras IQ al computador de borde.

- **Cómputo de borde:** la Raspberry Pi 5 ejecuta control, telemetría, adquisición, procesamiento y comunicación con servicios superiores.
- **Conectividad y ubicación:** el módulo SIM7600G-H aporta LTE y GNSS/GPS para comunicación y georreferenciación.

La Figura 4.1 presenta el conjunto físico del nodo. La lectura correcta del montaje es: antenas hacia MonRaF2, selección o acondicionamiento RF en MonRaF2, adquisición con HackRF One, procesamiento en Raspberry Pi 5 y comunicación por LTE/GNSS.

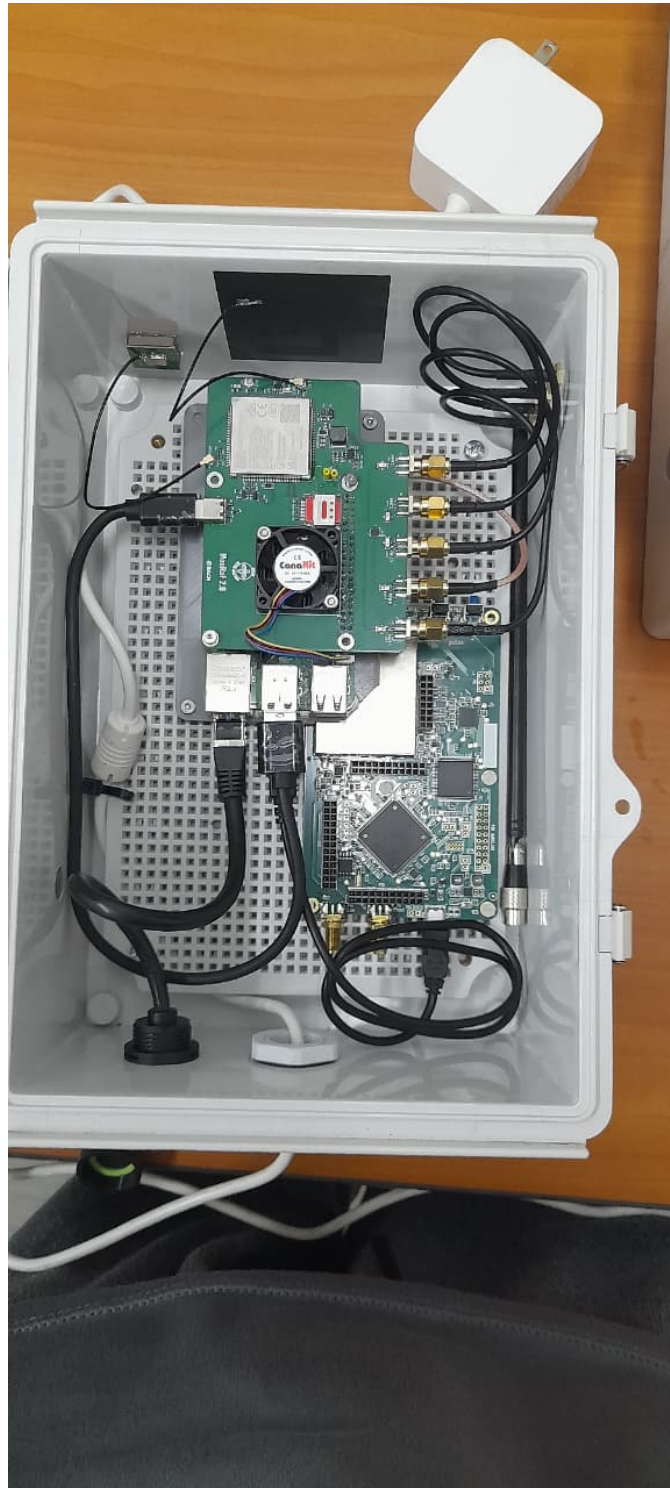


Figura 4.1: Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia.

4.3. Desarrollo técnico

4.3.1. Arquitectura física del nodo

El nodo se compone de una Raspberry Pi 5, una tarjeta HackRF One y una PCB MonRaF2. La Raspberry Pi 5 coordina la ejecución de software, el HackRF One realiza la recepción SDR y MonRaF2 concentra conectividad, geolocalización, señales de control, protecciones y rutas RF.

Tabla 4.1: Inventario funcional de componentes de hardware del nodo ANE2.

Componente	Función principal	Criterio de integración
Raspberry Pi 5	Computador de borde	Ejecuta control, telemetría, adquisición, procesamiento y comunicación. No debe tratarse como fuente principal para todos los periféricos en una versión robusta.
HackRF One	Receptor SDR	Digitaliza señales RF como muestras IQ. Debe recibir la ruta RF ya seleccionada/acondicionada y requiere una alimentación estable para evitar fallas bajo carga.
MonRaF2	Tarjeta de integración	Integra LTE/GNSS, UART/USB/GPIO, protecciones, filtros y una ruta RF con función tipo mux/switch. Es el punto de terminación de antenas y control de rutas RF.
SIM7600G-H	LTE/GNSS	Proporciona datos celulares y ubicación. Su consumo y transitorios deben considerarse dentro del árbol de potencia.
Antenas RF, LTE y GNSS	Interfaz electromagnética	Se conectan al subsistema de MonRaF2. La conexión no debe documentarse como antena directa al HackRF, porque el diseño incluye una etapa de selección/acondicionamiento RF.
Árbol de potencia externo	Distribución de energía	Debe proveer rieles y margen para Raspberry Pi 5, HackRF, MonRaF2, LTE/GNSS y extensiones futuras. Es un requisito de diseño, no una mejora opcional.

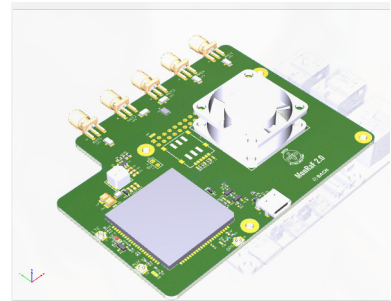
La Figura 4.2 muestra los tres módulos principales del nodo. Su separación ayuda a diagnosticar fallas: no es lo mismo un problema de ruta RF, de alimentación, de procesamiento, de enlace celular o de ubicación.



(a) Raspberry Pi 5.



(b) HackRF One.



(c) MonRaF2.

Figura 4.2: Módulos principales del hardware ANE2. Fuente: elaboración propia.

4.3.2. Ruta RF y mux de antenas

La ruta RF no debe describirse como una antena conectada directamente al HackRF. El esquemático de MonRaF2 muestra redes de antena como MAIN_ANT, GNSS_ANT y AUX_ANT, además de conectores RF, filtros LPF y una etapa de conmutación/mux RF controlada por señales GPIO. En términos operativos, MonRaF2 actúa como la tarjeta que recibe y ordena las rutas de antena antes de que la señal sea usada por el subsistema de adquisición o por el módulo celular/GNSS.

El flujo físico recomendado queda entonces:

1. Las antenas externas se conectan a MonRaF2.
2. MonRaF2 selecciona, protege o acondiciona rutas RF mediante su red de filtros, conectores y mux/switch RF.
3. La ruta RF seleccionada alimenta el subsistema correspondiente: adquisición SDR, LTE o GNSS, según el modo de operación.
4. La Raspberry Pi 5 controla la operación y recibe los datos del HackRF por USB, pero no debe asumirse como concentrador físico de antenas.

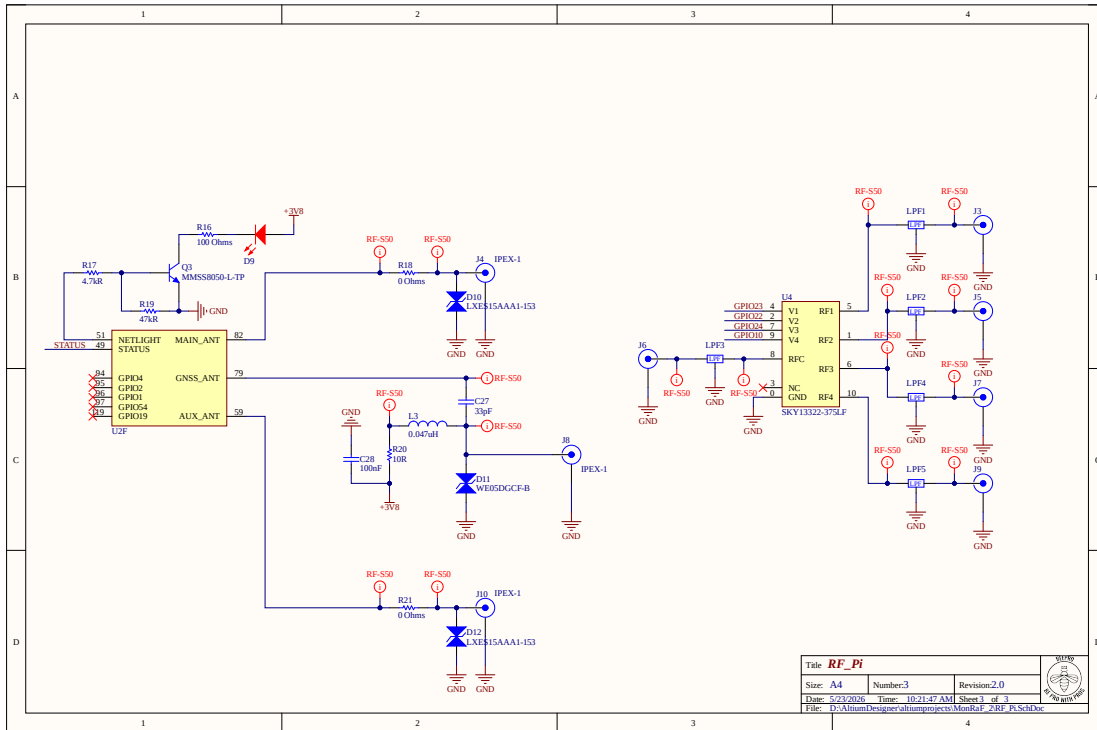


Figura 4.3: Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en MonRaF2. Fuente: elaboración propia.

4.3.3. Árbol de potencia

El principal aprendizaje del hardware es que el diseño de potencia debe preceder al diseño mecánico y de software. En las primeras integraciones, gran parte de la energía de periféricos se entregaba desde la Raspberry Pi 5. Ese enfoque funciona para pruebas de bajo alcance, pero se vuelve frágil cuando se combinan SDR, LTE, GNSS, procesamiento sostenido y tráfico de red.

La evidencia de telemetría muestra tres hechos prácticos:

- Las pruebas con HackRF y peticiones desde servidor elevaron la temperatura del procesador hasta el orden de 75 °C.
- El riel EXT5V presentó caídas bajo cargas pesadas, aun cuando no todas las corridas terminaron marcando una bandera persistente de subtensión.

- Las pruebas con alimentación externa para periféricos tuvieron mejor margen para sostener carga combinada.

Por ello, el criterio para un nuevo producto o una siguiente versión debe ser:

El nodo debe tener un árbol de potencia externo y generoso, con capacidad para la carga actual y para extensiones futuras. La Raspberry Pi 5 debe ser tratada como carga y controlador, no como fuente primaria para el resto del sistema.

Tabla 4.2: Criterios mínimos para una versión robusta del árbol de potencia.

Criterio	Implicación de diseño
Fuente principal sobredimensionada	Debe alimentar simultáneamente Raspberry Pi 5, HackRF, Mon-RaF2, LTE/GNSS y periféricos futuros sin operar en el límite.
Rieles distribuidos	Las cargas de alto consumo o con transitorios, como SDR y LTE, deben alimentarse desde rieles dedicados o reguladores con margen, no desde el puerto de la Raspberry Pi.
Medición de rieles	El sistema debe conservar telemetría de voltaje, corriente, temperatura y eventos de subtensión para diagnóstico.
Protección y desacople	Se deben incluir protecciones, filtrado y capacitancia local cerca de cargas ruidosas o pulsantes.
Reserva para extensiones	El diseño debe contemplar nuevos servicios, antenas, sensores, almacenamiento o comunicaciones sin rediseñar la alimentación desde cero.

4.4. Entradas, salidas e interfaces

Tabla 4.3: Interfaces principales del hardware ANE2.

Interfaz	Tipo	Contrato operacional
Antenas MonRaF2	a RF coaxial	Las antenas se conectan a MonRaF2. La etapa RF de la tarjeta selecciona o acondiciona rutas antes de la adquisición o uso LTE/GNSS.
MonRaF2 HackRF	a RF seleccionado	El HackRF debe recibir una ruta RF controlada, no una conexión de antena asumida como directa.
HackRF Raspberry Pi 5	a USB	Transporta muestras IQ y comandos de configuración. El <i>sample rate</i> aumenta simultáneamente el ancho instantáneo y la presión sobre USB/CPU.
Raspberry Pi 5 a MonRaF2	GPIO, UART, USB	Permite control auxiliar, comunicación con LTE/GNSS y señales de estado.
MonRaF2 red celular	a LTE	Provee conectividad de datos para estado, mediciones, campañas y reintentos.
MonRaF2 GNSS	a GNSS/GPS	Entrega ubicación del nodo para reportes y trazabilidad de mediciones.
Fuente externa a cargas	DC regulado	Debe distribuir potencia a Raspberry Pi 5, HackRF, MonRaF2 y expansiones con margen suficiente.

4.5. Decisiones de diseño o implementación

Tabla 4.4: Decisiones de diseño de hardware y justificación.

Decisión	Justificación	Implicación práctica
Usar MonRaF2 como concentrador RF y de conectividad	La tarjeta contiene rutas de antena, filtros, mux/switch RF, LTE/GNSS y señales de control.	Las antenas se documentan y cablean hacia MonRaF2; el HackRF queda como receptor SDR dentro de la cadena, no como punto directo de antena.
Usar Raspberry Pi 5 como computador de borde	Tiene capacidad suficiente para orquestación, telemetría y control del SDR.	Debe protegerse de sobrecarga eléctrica; no debe alimentar todos los periféricos en una versión robusta.

Decisión	Justificación	Implicación práctica
Usar HackRF One como receptor SDR	Permite recepción flexible por software y adquisición IQ.	Requiere ganancia configurada con criterio y alimentación estable.
Separar potencia de control	Los problemas de sub-tensión aparecen cuando la distribución de energía queda demasiado acoplada a la Raspberry Pi.	El árbol de potencia externo pasa a ser requisito de producto.
Mantener telemetría de salud	Temperatura, voltajes, corrientes y banderas de <i>throttling</i> permiten diagnosticar fallas reales.	Cada prueba de RF o conectividad debe acompañarse de telemetría eléctrica.

4.6. Problemas presentados y ajustes realizados

Tabla 4.5: Problemas presentados, decisiones y ajustes realizados en hardware.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión y bajo margen de potencia	Caídas de EXT5V bajo cargas SDR/LTE y escenarios con eventos de alimentación insuficiente.	Demasiadas cargas alimentadas o condicionadas desde la Raspberry Pi 5.	Diseñar un árbol de potencia externo, generoso y medible.	Requisito obligatorio para una versión robusta.
Aumento de temperatura bajo carga	Pruebas con HackRF y peticiones desde servidor alcanzaron temperaturas cercanas a 75 °C.	Procesamiento sostenido, actividad USB y carga RF/LTE simultánea.	Mantener disipación, ventilación y telemetría de temperatura.	La temperatura queda como restricción operativa.
Ambigüedad en ruta de antenas	El diseño incluye redes MAIN/GNSS/AUX, filtros y mux/switch RF en MonRaF2.	Interpretar la antena como conexión directa al HackRF simplifica de forma incorrecta la cadena RF.	Documentar MonRaF2 como entrada y selección RF.	La conexión física queda alineada con el esquemático.
Dependencia del enlace LTE	Las pruebas de campo usan conectividad celular para operación y envío de datos.	La red celular puede variar por cobertura, antena, SIM y ubicación.	Separar diagnóstico de RF, cómputo, potencia y LTE.	Las fallas se pueden atribuir con mejor evidencia.
Riesgo de saturación RF	HackRF no usa AGC de hardware en esta cadena de operación.	Ganancia excesiva o emisores fuertes pueden saturar el ADC de 8 bits.	Ajustar ganancia por escenario y conservar configuraciones durante comparaciones.	Menor riesgo de confundir artefactos con emisiones reales.

4.7. Validación, pruebas o evidencias

La validación usada para esta sección combina inspección de esquemático, fotografías del montaje y telemetría de nodos bajo reposo, carga SDR, carga LTE y carga combinada. Los indicadores más útiles fueron temperatura ARM, EXT5V, corrientes de rieles, banderas de subtensión, frecuencia de CPU y estados de *throttling*.

Tabla 4.6: Resumen de telemetría eléctrica y térmica inspeccionada.

Escenario	Temperatura ARM	EXT5V	UV	Lectura para operación
Nodo en reposo	54.3–57.6 °C	5.006–5.041 V	No	Referencia base con sistema operativo y telemetría.
HackRF en carga alta	54.9–74.7 °C	4.748–4.965 V	No	La carga SDR aumenta demanda y temperatura.
Periféricos conectados en reposo	45.0–52.1 °C	4.903–4.963 V	No	El margen mejora cuando no hay procesamiento pesado.
Petición media con periféricos	61.5–75.7 °C	4.721–5.083 V	No	El sistema se acerca a límites térmicos y de alimentación.
Carga alta con periféricos externos	49.4–75.2 °C	4.904–5.122 V	No	La alimentación externa mejora el margen frente a carga combinada.

4.8. Aprendizajes y transferencia de conocimiento

- El árbol de potencia define la robustez del nodo. Si se alimentan periféricos exigentes desde la Raspberry Pi, el diseño queda sin margen.
- Las antenas pertenecen a la cadena de MonRaF2 y su etapa RF. El HackRF debe verse como receptor dentro de una ruta seleccionada, no como destino directo de todas las antenas.
- La telemetría eléctrica debe acompañar cualquier prueba RF. Sin voltajes, corrientes y temperatura es fácil confundir fallas de potencia con fallas de software.
- El diseño debe reservar margen para servicios futuros. Cada extensión de software o hardware aumenta demanda de energía, temperatura y tráfico.

4.9. Limitaciones

- Las cifras de telemetría resumen pruebas disponibles; no reemplazan una caracterización eléctrica completa con instrumentos de laboratorio.
- No se presentan curvas completas de consumo por banda, ganancia, temperatura ambiente, antena o ubicación.
- El diseño RF requiere validar pérdidas, aislamiento, adaptación de impedancias y compatibilidad electromagnética antes de una versión final de campo.
- La estabilidad LTE/GNSS depende de cobertura, antenas, SIM, ubicación y configuración del operador.

4.10. Recomendaciones específicas

1. Diseñar una fuente principal externa con margen amplio para carga simultánea y extensiones futuras.
2. Alimentar HackRF, LTE/GNSS y cargas auxiliares desde rieles dedicados o reguladores adecuados, no desde el margen de la Raspberry Pi 5.
3. Mantener medición de voltaje, corriente, temperatura y banderas de subtensión como parte del producto.
4. Conectar y documentar antenas desde MonRaF2 y su ruta tipo mux/switch RF.
5. Revisar disipación térmica para escenarios con SDR y LTE activos al mismo tiempo.
6. Separar pruebas de reposo, SDR, LTE/GNSS y carga combinada para poder atribuir fallas con evidencia.

““latex

Capítulo 5

Software del sensor

5.1. Propósito del capítulo

El propósito de este capítulo es describir, de forma pedagógica y conceptual, la arquitectura del código que corre directamente en el sensor de monitoreo espectral remoto. El sensor está compuesto por una Raspberry Pi 5, una HackRF One y una tarjeta auxiliar que integra comunicación LTE, receptor GPS y un multiplexor para selección de antena.

La sección del proyecto descrita en este capítulo no corresponde a toda la plataforma. El sistema completo también incluye componentes externos, como backend, frontend, bases de datos y servicios de visualización. Este capítulo se concentra en el código embebido del sensor: la parte que recibe instrucciones de la plataforma, controla el hardware de radiofrecuencia, adquiere muestras, procesa señales y devuelve resultados.

El propósito general del código es convertir al sensor físico en un nodo autónomo de medición espectral. Para ello, el software debe recibir órdenes del servidor, configurar la HackRF One, seleccionar la antena adecuada, capturar señales IQ, procesarlas localmente y enviar a la plataforma central productos útiles como PSD, métricas de modulación, audio demodulado, estado del dispositivo y coordenadas GPS.

De manera específica, el software del sensor busca cumplir los siguientes objetivos:

- Controlar la adquisición de radio mediante la HackRF One, ajustando frecuencia central, tasa de muestreo, ganancias, amplificador y puerto de antena.
- Ejecutar mediciones espectrales en modo *realtime* y en campañas programadas desde el servidor.
- Procesar muestras IQ crudas para estimar densidad espectral de potencia mediante métodos como Welch y banco de filtros polifase.
- Demodular señales AM o FM cuando el servidor solicita audio en tiempo real.

- Aplicar etapas de limpieza y corrección para mejorar la calidad de las mediciones, incluyendo balance IQ y remoción del pico DC.
- Mantener una calibración de frecuencia que compense desviaciones del oscilador de la HackRF.
- Coordinar la comunicación con el backend, el envío de resultados, los reintentos ante fallos de red y el reporte de estado del sensor.
- Gestionar recursos físicos del sensor, como LTE, GPS, GPIO y multiplexor de antena.

5.2. Contexto dentro del sistema general

El software del sensor hace parte del sistema de monitoreo espectral multisensor remoto. Dentro de la arquitectura general, el sensor actúa como un nodo de adquisición y procesamiento local, mientras que la plataforma central se encarga de coordinar tareas, almacenar resultados, visualizar información y permitir la interacción con el usuario.

La arquitectura del sensor se puede entender como una cadena de trabajo: el servidor decide qué se debe medir, el orquestador traduce esa instrucción a una configuración de radio, el motor RF adquiere y procesa las muestras, y finalmente los resultados regresan al servidor.

La división principal del sistema es la siguiente:

- **Orquestación en Python:** gestiona la relación con el backend, consulta instrucciones, programa campañas, controla estados del sistema, envía comandos al motor RF y prepara los resultados para publicarlos.
- **Motor RF en C:** controla la HackRF One, recibe muestras IQ, administra buffers de alta velocidad, ejecuta procesamiento DSP, calcula PSD, demodula audio y devuelve resultados al orquestador.
- **Servicios auxiliares:** atienden funciones de soporte como GPS, LTE, estado del sistema, reintentos de datos no enviados, WebRTC para audio y control del multiplexor de antena.

La decisión de separar Python y C responde a una necesidad práctica. Python es adecuado para coordinación, comunicación HTTP, manejo de estados y programación de tareas. C es adecuado para la parte sensible a latencia: captura de muestras, buffers, FFT, filtrado, PSD y demodulación.

El sensor opera con varios servicios o modos complementarios:

- **PSD en tiempo real:** el backend solicita una configuración y el sensor responde con una estimación espectral de la banda capturada.
- **PSD y audio en tiempo real:** además de la PSD, el sensor demodula FM o AM y genera tramas de audio reproducibles por la plataforma.
- **Campañas:** el backend programa mediciones periódicas; el sensor guarda la configuración, agenda ejecuciones y sube resultados en cada intervalo.
- **Calibración en frecuencia:** el sensor estima el error de sintonía y aplica una corrección para que las mediciones queden mejor alineadas en frecuencia.
- **GPS y LTE:** el sensor mantiene conectividad celular y reporta ubicación geográfica al servidor.
- **Estado del dispositivo:** se reportan variables de salud como conectividad, almacenamiento, temperatura, memoria y actividad del software.

5.3. Desarrollo técnico

El código puede leerse como un conjunto de bloques funcionales. Cada bloque cumple una responsabilidad dentro del camino completo de la señal: desde la instrucción del backend hasta la publicación del resultado.

5.3.1. Adquisición

El bloque de adquisición es el encargado de hablar con la HackRF One y convertir una instrucción de medición en muestras IQ reales.

Conceptualmente, este bloque realiza cinco acciones:

1. Recibe una configuración de medición enviada por el orquestador. Esa configuración contiene parámetros como frecuencia central, ancho de muestreo, ganancias, método de PSD, puerto de antena, filtro solicitado y modo de demodulación.
2. Aplica la configuración sobre la HackRF One. Esto incluye sintonía de frecuencia, tasa de muestreo, ganancias de recepción y corrección de frecuencia cuando existe una calibración disponible.
3. Inicia la recepción de muestras IQ crudas desde el dispositivo SDR.
4. Deposita esas muestras en buffers circulares para separar la velocidad de llegada del hardware de la velocidad de procesamiento del software.

5. Entrega bloques de muestras suficientemente grandes a los algoritmos de procesamiento, evitando que cada operación pesada interrumpa directamente la captura.

Las muestras IQ representan la señal recibida en banda base compleja. La componente I corresponde a la parte en fase y la componente Q a la parte en cuadratura. En conjunto permiten conservar información de amplitud y fase, que es necesaria para estimar el espectro, filtrar, demodular FM o demodular AM.

5.3.2. Uso de buffers

La lógica de buffers es central en el sensor. La HackRF entrega datos de forma continua y rápida, mientras que la PSD, el filtrado o la demodulación requieren procesamiento por bloques. Por eso el motor RF usa buffers circulares:

- Un buffer principal recibe muestras IQ para estimación espectral.
- Un buffer de audio conserva muestras necesarias cuando se activa demodulación AM o FM.
- El software drena estos buffers cuando ya hay suficientes muestras para construir una respuesta.

Esta estructura permite que el sensor atienda procesos en tiempo real, como PSD y audio, sin tratar cada muestra de manera aislada. La idea pedagógica es similar a una banda transportadora: el hardware deposita muestras, el procesamiento toma paquetes completos y el orquestador recibe resultados ya elaborados.

5.3.3. Preprocesamiento

El preprocesamiento agrupa las operaciones que preparan la señal antes de extraer productos finales. Su objetivo es reducir artefactos propios del hardware y mejorar la calidad de la medición.

Balance IQ

En un receptor SDR real, las ramas I y Q no siempre quedan perfectamente balanceadas. Puede haber diferencias de ganancia, pequeños desplazamientos DC o correlación no deseada entre ambas componentes. Si estos efectos no se corrigen, pueden aparecer artefactos en el espectro o degradación en la demodulación.

El balance IQ busca compensar esos errores antes del cálculo de PSD o del filtrado. De forma conceptual, el algoritmo:

- remueve desplazamientos promedio en las ramas I y Q;
- compara la energía relativa de ambas ramas;
- corrige diferencias de amplitud;
- reduce correlaciones espurias entre I y Q.

No se trata de cambiar la información de la señal recibida, sino de disminuir imperfecciones introducidas por la cadena de recepción.

Corrección de componente DC

La componente DC aparece cerca del centro de la banda base y suele estar asociada a efectos internos del receptor. En el flujo del sensor se atiende en dos niveles:

- En el preproceso IQ se reduce el offset de las muestras antes de calcular PSD o demodular.
- En el postproceso de PSD se repara el pico central visible en el espectro cuando aparece como un artefacto de medición.

Esta separación es importante: una cosa es limpiar la señal cruda y otra es pulir el producto espectral que se enviará al servidor.

5.3.4. Procesamiento

El bloque de proceso transforma muestras IQ en productos útiles para la plataforma. En este repositorio se implementan tres familias principales de procesamiento: estimación de PSD, demodulación de audio y filtrado de señales.

Estimación de PSD

La PSD, o densidad espectral de potencia, permite observar cómo se distribuye la energía de la señal dentro de un rango de frecuencias. Es el producto principal para monitoreo espectral, porque permite identificar ocupación de banda, niveles relativos de potencia, portadoras y posibles emisiones.

El sensor contempla métodos como:

- **Welch:** divide la señal en segmentos, aplica ventanas, calcula FFT y promedia resultados para obtener una estimación estable del espectro.

- **Banco de filtros polifase:** mejora la separación entre canales y ayuda a controlar fugas espectrales cuando se requiere una estimación más selectiva.

La PSD es una de las tareas más costosas del sistema, porque requiere operar sobre grandes bloques de muestras y ejecutar transformadas de Fourier. Por eso el motor RF está escrito en C y aprovecha una lógica de trabajo multihilo y bibliotecas de procesamiento eficientes.

Demodulación FM y AM

Cuando el backend solicita audio en tiempo real, el sensor no solo calcula PSD. También toma las muestras IQ, extrae la información modulada y la convierte en audio reproducible.

En FM, la información está principalmente en la variación de fase o frecuencia instantánea. El proceso conceptual consiste en observar cómo cambia la fase entre muestras consecutivas, convertir esa variación en una señal de audio y acondicionar el resultado para reproducción.

En AM, la información está principalmente en la envolvente de la señal. El proceso conceptual consiste en calcular la magnitud de la señal compleja, separar las variaciones útiles de la portadora y normalizar el audio resultante.

Después de demodular, el audio se convierte a un formato de transmisión. El sensor genera tramas de audio, las codifica con Opus y las entrega al servicio local que publica el flujo mediante WebRTC. Esto permite que la plataforma escuche la señal con baja latencia sin que el backend tenga que procesar IQ crudo.

Filtrado

El filtrado permite concentrarse en una parte de la banda capturada y rechazar contenido fuera del rango de interés. El servidor puede solicitar un rango de frecuencia y el motor RF aplica un filtro pasabanda antes de calcular la PSD o de preparar la señal.

Desde una perspectiva conceptual, el filtro funciona como una ventana selectiva: conserva la región que interesa y atenúa lo que queda por fuera. En el código se apoya en procedimientos de dominio frecuencial y validaciones para que el rango solicitado sea coherente con la banda realmente capturada por la HackRF.

Trabajo multihilo

El sistema separa tareas para mantener capacidad de respuesta:

- Un flujo atiende la llegada de muestras desde la HackRF.

- Otro flujo procesa bloques para PSD.
- Cuando hay audio, un hilo adicional drena muestras, demodula y envía tramas codificadas.
- El orquestador permanece disponible para comunicarse con el backend y manejar cambios de modo.

Esta organización evita que una tarea pesada, como una PSD de alta resolución, bloquee completamente la captura o la transmisión de audio.

La lógica de multitarea y multihilo usada para paralelizar procesos dentro del motor RF se apoya en la librería OpenMP para C. Esta herramienta permite distribuir regiones de cómputo entre varios hilos de ejecución de forma relativamente sencilla, sin tener que implementar manualmente toda la administración de hilos mediante interfaces de menor nivel.

El uso de OpenMP aporta varias ventajas al desarrollo del sensor. En primer lugar, facilita la paralelización de operaciones repetitivas o costosas, como cálculos sobre bloques de muestras, procesamiento espectral o tareas que pueden dividirse entre núcleos de CPU. En segundo lugar, reduce la complejidad del código frente a una implementación completamente manual con hilos, bloqueos y sincronización explícita. Finalmente, es una solución ampliamente soportada en entornos de compilación en C, lo que permite mantener una implementación portable y fácil de integrar dentro del motor RF.

5.3.5. Postprocesamiento

El postprocesamiento corresponde a la pulida final aplicada a los resultados antes de enviarlos al servidor. En este sistema, la etapa más importante es la corrección del pico DC en la PSD.

Remoción del pico DC en PSD

Aunque el preproceso reduce offset en las muestras IQ, en la PSD puede quedar un pico artificial cerca del centro de la banda. Ese pico no siempre representa una emisión real; con frecuencia es un artefacto propio del receptor SDR.

El algoritmo de postproceso analiza el comportamiento estadístico de la PSD como una señal con variabilidad local. En lugar de borrar siempre un número fijo de puntos, estima la región afectada y decide cómo reconstruirla.

Conceptualmente, el procedimiento realiza:

- detección de la región central afectada usando simetría y cambios locales en la pendiente;

- análisis de ocupación espectral alrededor del centro;
- validaciones con descriptores estadísticos como media, mediana, dispersión y comportamiento de la cola alta;
- ampliación de la ventana de reparación cuando la zona tiene baja ocupación espectral;
- reconstrucción local mediante interpolación lineal o ajuste polinomial.

El resultado enviado al backend conserva una PSD más limpia. Cuando aplica, el software también puede conservar información de diagnóstico sobre la corrección, como la ventana reparada y el modo de reconstrucción usado.

5.3.6. Calibración en frecuencia

La calibración en frecuencia busca compensar la desviación entre la frecuencia nominal solicitada y la frecuencia real a la que queda sintonizado el receptor. En SDR de bajo costo, esta desviación puede aparecer por tolerancias del oscilador, temperatura o condiciones de operación.

El sensor usa una referencia conocida para estimar el error de frecuencia. En el caso de emisiones FM comerciales, una referencia útil es el tono piloto estéreo de 19 kHz que aparece después de demodular FM. Si una estación FM fuerte contiene ese tono, el sensor puede usarlo como punto de comparación.

El flujo conceptual es:

1. Capturar una porción amplia de la banda donde se esperan emisoras FM.
2. Identificar candidatos con potencia suficiente.
3. Bajar cada candidato a banda base y demodular FM.
4. Buscar el tono piloto alrededor de 19 kHz en la señal demodulada.
5. Comparar la posición observada con la posición esperada.
6. Estimar un error de frecuencia y expresarlo como factor de corrección.

Cuando hay varias emisiones candidatas, el sistema puede usar más de una referencia para evitar depender de una sola estación. Las emisiones más confiables reciben mayor peso, por ejemplo si tienen mejor potencia o una detección más clara del tono piloto. Con esa información se obtiene una estimación más robusta del error.

El resultado de la calibración se guarda como un error de frecuencia, normalmente expresado en partes por millón. Luego, cuando el backend solicita una frecuencia central, el orquestador incluye ese factor en la configuración enviada al motor RF.

El motor usa la corrección para ajustar la sintonía de la HackRF. De esta manera, la frecuencia física de adquisición queda más alineada con la frecuencia nominal que la plataforma espera visualizar.

5.3.7. Orquestador

El orquestador es la lógica de coordinación del sensor. Su función no es procesar IQ de forma intensiva, sino dirigir el trabajo de todos los bloques.

Sus responsabilidades principales son:

- Consultar al backend para saber si hay solicitudes en tiempo real.
- Consultar y sincronizar campañas programadas.
- Validar configuraciones antes de enviarlas al motor RF.
- Enviar comandos al motor RF usando mensajería local ZeroMQ.
- Recibir resultados de PSD y métricas desde el motor RF.
- Aplicar postproceso sobre los resultados cuando corresponde.
- Formatear y publicar datos hacia el backend mediante HTTP REST.
- Controlar el inicio y parada del servicio de audio WebRTC.
- Mantener estados globales como `IDLE`, `REALTIME`, `CAMPAIGN`, `CALIBRATING` y `ERROR`.
- Gestionar memoria compartida local para parámetros persistentes, como `ppm_error`, coordenadas GPS o configuraciones de campaña.

Las campañas son mediciones programadas por el servidor. El orquestador consulta las campañas activas, selecciona la que corresponde ejecutar, guarda sus parámetros y agenda ejecuciones periódicas. Cada ejecución captura PSD y envía el resultado al backend. Si la red falla, el resultado puede quedar en una cola local para ser reenviado posteriormente.

En modo *realtime*, el servidor entrega una configuración inmediata. El orquestador la envía al motor RF, espera la respuesta, aplica postproceso y sube el resultado. Mientras el backend siga enviando una solicitud válida, el sensor mantiene este modo activo.

Cuando la solicitud incluye demodulación, el orquestador activa también el camino de audio. En ese caso, el motor RF demodula AM o FM y genera tramas Opus; el servicio WebRTC se encarga de hacer disponible ese audio para la plataforma.

Antes o durante ciertos ciclos de operación, el orquestador puede solicitar al motor RF una calibración de frecuencia. El resultado se guarda localmente y se usa en adquisiciones posteriores.

5.4. Entradas, salidas e interfaces

El software del sensor se comunica con diferentes elementos internos y externos. Por esta razón, se puede describir en términos de entradas, salidas e interfaces.

5.4.1. Entradas

Las principales entradas del sistema son:

- Configuraciones de medición enviadas desde el backend.
- Parámetros de campañas programadas.
- Solicitudes de PSD en tiempo real.
- Solicitudes de PSD con audio en tiempo real.
- Parámetros de frecuencia central, span, ganancias, filtro, puerto de antena y modo de demodulación.
- Muestras IQ crudas entregadas por la HackRF One.
- Datos GPS recibidos desde el módulo de posicionamiento.
- Estado de conectividad entregado por el servicio LTE.
- Parámetros persistentes almacenados localmente, como el error de frecuencia en partes por millón.

5.4.2. Salidas

Las principales salidas del sistema son:

- Estimaciones de PSD enviadas al backend.

- Métricas de modulación y resultados derivados del procesamiento.
- Audio demodulado en tiempo real cuando se solicita AM o FM.
- Tramas de audio codificadas con Opus.
- Coordenadas GPS del sensor.
- Estado general del dispositivo.
- Información de conectividad, almacenamiento, temperatura, memoria y actividad del software.
- Resultados de calibración en frecuencia.
- Datos almacenados localmente para reintento cuando hay fallos de red.

5.4.3. Interfaces y protocolos

Los protocolos y procedimientos principales que conectan los bloques del sensor son los siguientes:

Elemento	Uso conceptual en el sensor
HTTP REST	Comunicación entre sensor y backend para consultar instrucciones y publicar datos.
ZeroMQ	Canal local de comandos entre el orquestador Python y el motor RF en C.
libhackrf/USB	Interfaz de control y adquisición de muestras desde la HackRF One.
WebRTC	Entrega de audio demodulado hacia cliente/plataforma con baja latencia.
Opus	Codificación de audio antes de enviarlo al servicio WebRTC.
GPIO	Control de líneas físicas para multiplexor de antena y soporte de hardware.
PPP/LTE	Conectividad celular del sensor hacia la red.
GPS/NMEA	Obtención y conversión de coordenadas geográficas.
Welch	Método de estimación de PSD por segmentos y promediado.
Banco polifase	Método de estimación espectral con mejor control de separación entre canales.

Elemento	Uso conceptual en el sensor
OpenMP	Paralelización de tareas de cómputo dentro del motor RF desarrollado en C.

5.5. Decisiones de diseño o implementación

Una de las decisiones principales de diseño fue dividir el software del sensor en dos capas: una capa de orquestación en Python y un motor RF en C. Esta separación permite aprovechar las ventajas de cada lenguaje. Python facilita la comunicación con el backend, el manejo de estados, la programación de campañas y la integración con servicios auxiliares. C permite ejecutar tareas de adquisición y procesamiento digital de señales con mayor control sobre memoria, buffers y rendimiento.

Otra decisión importante fue usar buffers circulares para desacoplar la velocidad de llegada de las muestras desde la HackRF de la velocidad de procesamiento del software. Esta estrategia permite atender flujos de datos continuos y evitar que operaciones costosas, como la estimación de PSD o la demodulación, interrumpen la captura de muestras.

También se decidió usar ZeroMQ como canal local de comunicación entre el orquestador Python y el motor RF en C. Esto permite separar responsabilidades, mantener independencia entre procesos y enviar comandos o recibir resultados sin integrar todo el sistema en un único ejecutable.

Para la entrega de audio en tiempo real, se optó por demodular localmente la señal en el sensor, codificar el audio con Opus y publicarlo mediante WebRTC. Esta decisión evita enviar IQ crudo al backend para tareas de escucha y reduce la carga de procesamiento de la plataforma central.

En cuanto a procesamiento paralelo, se decidió usar OpenMP para implementar la lógica de multitarea y multihilo dentro del motor RF. Esta elección se tomó porque OpenMP facilita la paralelización de regiones de código en C, tiene soporte amplio en compiladores comunes y permite aprovechar mejor los núcleos disponibles de la Raspberry Pi 5 sin aumentar excesivamente la complejidad del código.

Finalmente, se incluyeron etapas de preprocesamiento y postprocesamiento para mejorar la calidad de los productos enviados al backend. Entre estas etapas se encuentran el balance IQ, la reducción de componente DC en las muestras y la remoción del pico DC en la PSD.

5.6. Problemas presentados y ajustes realizados

Durante el diseño del software fue necesario considerar varios problemas propios de un sistema SDR remoto.

Uno de los primeros problemas es la presencia de imperfecciones en la cadena de recepción. En un receptor SDR real, las ramas I y Q pueden presentar diferencias de ganancia, desplazamientos DC o correlación no deseada. Para mitigar estos efectos se incorporó una etapa de balance IQ y reducción de offset antes del procesamiento espectral o la demodulación.

Otro problema identificado es la aparición de un pico artificial cerca del centro de la PSD. Este pico no necesariamente corresponde a una emisión real, sino que puede ser un artefacto de medición asociado al receptor. Para atenderlo se incluyó una etapa de postprocesamiento que analiza la región central de la PSD, estima la zona afectada y reconstruye localmente el resultado mediante interpolación lineal o ajuste polinomial.

También se contemplaron fallos de red asociados a la conectividad LTE. Por esta razón, el orquestador debe manejar reintentos, estados internos y almacenamiento temporal de resultados cuando el envío al backend no puede realizarse de forma inmediata.

La desviación de frecuencia de la HackRF también fue considerada como un problema relevante. Debido a tolerancias del oscilador, temperatura o condiciones de operación, la frecuencia real de adquisición puede diferir de la frecuencia nominal solicitada. Para corregir este efecto se implementó una lógica de calibración en frecuencia basada en referencias presentes en emisiones FM comerciales.

Además, la operación en tiempo real exige que el sistema no bloquee la adquisición mientras procesa datos. Para esto se ajustó la arquitectura mediante buffers circulares, separación de tareas y paralelización con OpenMP dentro del motor RF.

5.7. Validación, pruebas o evidencias

La validación del software del sensor puede entenderse desde el flujo completo de funcionamiento. De forma resumida, el ciclo operativo del sistema es el siguiente:

1. El sensor se identifica ante la plataforma mediante su dirección MAC y su estado local.
2. El orquestador consulta si hay una tarea *realtime* o una campaña activa.
3. Si hay tarea, se construye una configuración de radio y se envía al motor RF.
4. El motor RF configura la HackRF, selecciona antena si aplica y captura IQ.

5. Las muestras pasan por preprocesamiento para mejorar su condición de medición.
6. El bloque de proceso calcula PSD, aplica filtros y, si se solicita, demodula audio.
7. El postproceso corrige artefactos finales, especialmente el pico DC en la PSD.
8. El orquestador empaqueta el resultado y lo envía al backend.
9. Si el envío falla, el dato puede quedar almacenado localmente para reintento.
10. Los servicios auxiliares mantienen GPS, LTE, estado del sensor y audio según el modo activo.

Este flujo permite validar que las principales etapas del sensor están conectadas: recepción de instrucciones, adquisición IQ, procesamiento, postprocesamiento, publicación de resultados y soporte de servicios auxiliares.

Como evidencias funcionales del sistema se consideran:

- La capacidad de recibir configuraciones desde el backend.
- La configuración efectiva de la HackRF One.
- La adquisición de muestras IQ.
- La generación de PSD mediante métodos como Welch o banco de filtros polifase.
- La activación de demodulación AM o FM cuando se solicita audio.
- La entrega de audio codificado mediante Opus y publicado por WebRTC.
- El reporte de estado del sensor, GPS, conectividad y variables de salud.
- La aplicación de corrección de frecuencia cuando existe una calibración disponible.
- El manejo de reintentos ante fallos de comunicación con el backend.

5.8. Aprendizajes y transferencia de conocimiento

El desarrollo del software del sensor deja varios aprendizajes importantes para la continuidad del proyecto.

En primer lugar, se evidencia la conveniencia de separar las tareas de orquestación y las tareas de procesamiento intensivo. Python resulta adecuado para la integración con servicios, consultas HTTP, manejo de estados y coordinación general. C resulta más adecuado para las tareas críticas de adquisición, procesamiento DSP, manejo de buffers y demodulación.

En segundo lugar, el uso de buffers circulares es fundamental en sistemas de adquisición continua. En un sensor SDR, el hardware puede entregar muestras de forma sostenida y a alta velocidad, por lo que el software debe estar preparado para recibir datos sin bloquearse mientras ejecuta procesamiento por bloques.

En tercer lugar, se confirma la importancia del preprocesamiento y el postprocesamiento. La señal entregada por un SDR real puede contener artefactos como desbalance IQ, offset DC o picos centrales en la PSD. Por tanto, el software no solo debe adquirir y calcular espectros, sino también aplicar correcciones que mejoren la calidad de los productos entregados al backend.

Otro aprendizaje importante es que las funcionalidades de tiempo real dependen tanto del procesamiento local como de la comunicación con la plataforma. Aunque el motor RF procese los datos rápidamente, la experiencia de visualización también depende de la latencia de red, la publicación al backend y la actualización de la interfaz.

Finalmente, el uso de OpenMP muestra que es posible mejorar el aprovechamiento de los recursos de cómputo disponibles sin aumentar excesivamente la complejidad del código. Esta herramienta permite introducir paralelización en C de forma más sencilla que una implementación manual completa basada en hilos.

5.9. Limitaciones

El sistema presenta varias limitaciones que deben tenerse en cuenta para su operación y evolución futura.

Una primera limitación está asociada a la latencia de comunicación con el servidor. Aunque el sensor pueda adquirir y procesar señales en tiempo real, la tasa de actualización observada en la plataforma siempre estará limitada por los tiempos de envío, recepción, procesamiento en backend y visualización en frontend. Por tanto, el tiempo real del sistema no depende únicamente de la velocidad del motor RF, sino también de la infraestructura de comunicación y de la plataforma central.

Otra limitación corresponde al span configurable. El sistema trabaja con un límite de span de hasta 20 MHz, asociado al ancho de banda práctico de adquisición de la HackRF One y a la configuración implementada en el software. Actualmente no se incluye una funcionalidad de *sweep* que permita recorrer secuencialmente varias porciones del espectro para observar un ancho de banda mayor al que permite una sola captura directa del hardware.

También existe una limitación en la calibración en frecuencia. La estrategia implementada depende de detectar emisiones FM comerciales y, en particular, referencias como el tono piloto estéreo de 19 kHz. Por esta razón, en zonas aisladas o en lugares donde no existan emisoras FM suficientemente fuertes o claras, el sensor puede no contar con referencias adecuadas para ejecutar la calibración.

Otra limitación importante corresponde a la calibración en potencia. Las HackRF One usadas en el sistema no son instrumentos calibrados en potencia absoluta. Por esta razón, al calcular el logaritmo de la potencia de la PSD, el resultado se interpreta en términos relativos, como dBFS, y no directamente como dBm o dB absolutos referenciados. Para obtener mediciones robustas de potencia absoluta, sería necesario realizar una calibración externa con señales de referencia, instrumentación adecuada y una curva de corrección que relacione los niveles medidos por el sensor con niveles reales de potencia.

Finalmente, al tratarse de un sistema remoto basado en conectividad LTE, el envío de resultados puede verse afectado por cobertura, disponibilidad de red, pérdida de paquetes o variaciones de latencia. Aunque el software contempla reintentos y almacenamiento local, la operación continua depende de la calidad de la conexión en el sitio de despliegue.

5.10. Recomendaciones específicas

A partir del análisis del software del sensor, se proponen las siguientes recomendaciones específicas:

- Mantener la separación entre orquestación en Python y procesamiento RF en C, ya que esta división permite conservar claridad arquitectónica y rendimiento en las tareas críticas.
- Documentar con mayor detalle las interfaces entre el orquestador y el motor RF, especialmente los mensajes ZeroMQ, los parámetros de configuración y los formatos de respuesta.
- Conservar el uso de buffers circulares y revisar periódicamente sus tamaños, tiempos de drenaje y condiciones de saturación para evitar pérdidas de muestras.
- Continuar usando OpenMP para paralelizar tareas de procesamiento en C, evaluando cuidadosamente qué secciones del código realmente se benefician de la paralelización.
- Implementar en el futuro una funcionalidad de *sweep* si se requiere observar un span mayor a 20 MHz. Esta funcionalidad debería coordinar capturas sucesivas en diferentes frecuencias centrales y reconstruir una vista espectral más amplia.
- Desarrollar una metodología de calibración en potencia si el sistema se va a usar para mediciones absolutas. Esta calibración debería permitir convertir los niveles relativos de PSD, expresados en dBFS, a unidades referenciadas como dBm.
- Complementar la calibración en frecuencia con otros métodos de referencia para escenarios donde no existan emisoras FM comerciales disponibles o donde el tono piloto de 19 kHz no pueda detectarse de manera confiable.

- Medir y caracterizar la latencia extremo a extremo del sistema, desde la adquisición en el sensor hasta la visualización en la plataforma, para definir con precisión el límite real de actualización en tiempo real.
- Mantener registros de diagnóstico sobre corrección de pico DC, calibración de frecuencia, fallos de red y estado del dispositivo, con el fin de facilitar soporte técnico y depuración.
- Evaluar periódicamente el desempeño térmico y de recursos de la Raspberry Pi 5, especialmente durante tareas de PSD de alta resolución, demodulación de audio y operación multihilo.

““

Capítulo 6

Plataforma web en la nube

6.1. Propósito del capítulo

Este capítulo documenta la plataforma web que constituye la capa de interfaz humana y de coordinación central del sistema de monitoreo espectral ANE-HX. La plataforma está compuesta por un frontend de aplicación de página única (SPA) y un backend de servicios que actúa como centro de orquestación entre las personas operadoras, los sensores RF en campo, la base de datos y los servicios de postprocesamiento.

El capítulo describe la arquitectura general, las funcionalidades ofrecidas a los usuarios, las interfaces de comunicación entre componentes, las decisiones de diseño que guiaron la implementación, los problemas encontrados durante el desarrollo, los aprendizajes obtenidos y las limitaciones actuales. El objetivo es proporcionar una comprensión completa de cómo la plataforma web hace posible el ciclo operativo completo: desde la autenticación de una persona usuaria hasta la generación de un reporte de cumplimiento normativo basado en mediciones de espectro capturadas en campo.

6.2. Contexto dentro del sistema general

La plataforma web ocupa la posición central en la arquitectura del sistema ANE-HX. Como se describe en el capítulo de diseño general (Capítulo 3), el sistema se organiza en tres dominios principales: borde (sensores RF y adquisición), nube (plataforma web, base de datos y postprocesamiento) y usuarios (personas operadoras y administradoras). La plataforma web constituye la totalidad del dominio de nube desde la perspectiva de coordinación y la totalidad del dominio de usuarios desde la perspectiva de interfaz.

La relación con los demás módulos del sistema es la siguiente:

- **Hardware y sensores RF (Capítulo 4):** la plataforma recibe los datos de espectro, estado, GPS y audio demodulado que los sensores capturan en campo. También entrega a los sensores las configuraciones de adquisición y las campañas programadas que deben ejecutar.
- **SDR y adquisición en borde (Capítulo 5):** el flujo de adquisición definido en el borde depende de los parámetros que la plataforma configura y transmite. La plataforma es la consumidora de los datos generados por la cadena de procesamiento SDR.
- **Análisis espectral y localización en la nube (Capítulo 7):** la plataforma invoca al servicio de postprocesamiento Python para el análisis de emisiones, detección de señales, cruce con licencias y evaluación de cumplimiento normativo. Los resultados del análisis se integran en los reportes que la plataforma entrega a las personas usuarias.
- **Protocolos de prueba (Capítulo 8):** la plataforma es el objeto de pruebas de integración extremo a extremo y pruebas de usabilidad. Los protocolos de prueba definidos en ese capítulo se aplican sobre la plataforma para verificar su correcto funcionamiento en conjunto con los demás módulos.
- **Integración general (Capítulo 9):** la plataforma es el componente integrador por excelencia. El capítulo de integración detalla cómo todos los módulos se conectan a través de la plataforma para formar el sistema completo.

En el flujo extremo a extremo del sistema, la plataforma web es el punto de entrada para las personas y el punto de convergencia para los datos. Sin la plataforma, los sensores podrían capturar datos pero no habría forma de configurarlos, visualizar sus mediciones, planificar campañas, detectar alertas ni generar reportes.

6.3. Desarrollo técnico

La plataforma web se implementa como dos aplicaciones independientes que se despliegan como contenedores Docker coordinados por un proxy inverso Nginx: un frontend de aplicación de página única (SPA) y un backend de servicios API.

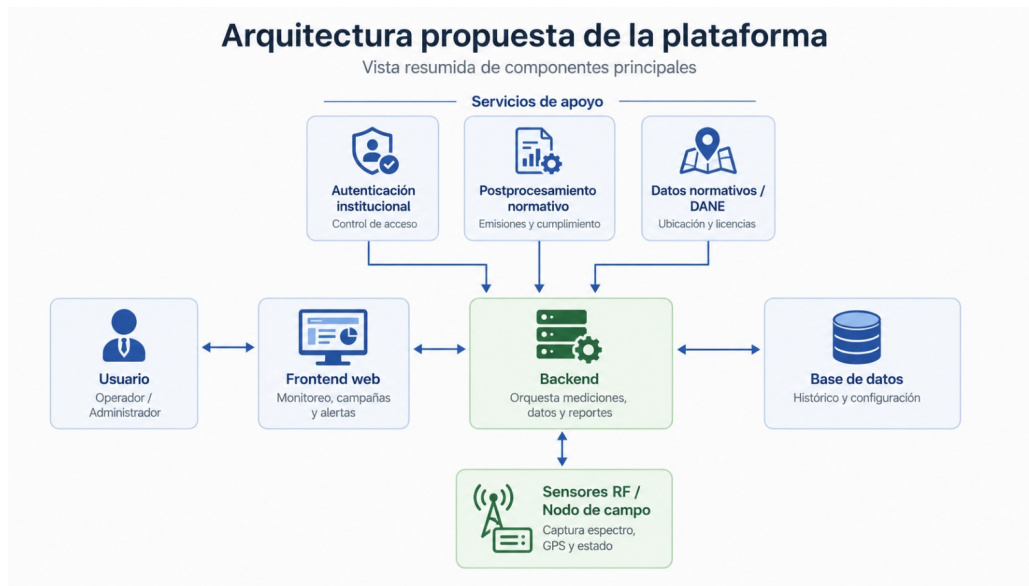


Figura 6.1: Diagrama general de la arquitectura de la plataforma web. Fuente: Elaboración propia.

6.3.1. Frontend

El frontend es una aplicación web de página única desarrollada con React 19 y TypeScript, construida con el empaquetador Vite. La interfaz sigue un diseño de panel de control con menú lateral de navegación que da acceso a siete pantallas principales.

Tecnologías

La pila tecnológica del frontend está conformada por:

- **React 19** como librería principal de componentes de interfaz, seleccionado por su modelo eficiente de actualización del DOM virtual y su capacidad para manejar datos en tiempo real sin re-renderizaciones innecesarias.
- **TypeScript** como lenguaje de desarrollo, proporcionando tipado estático que mejora la detección temprana de errores y documenta implícitamente las interfaces de datos entre componentes.
- **Vite 5** como empaquetador y servidor de desarrollo, seleccionado por su velocidad de compilación basada en ESM nativo y su configuración simplificada.
- **Tailwind CSS 3** como framework de estilos mediante utilidades atómicas, facilitando la iteración rápida sobre el diseño y la consistencia visual entre componentes.

- **Plotly.js** para la visualización interactiva de espectro de potencia y gráficas waterfall (espectrograma), seleccionada por su soporte nativo para gráficos científicos con miles de puntos de datos y actualización en tiempo real.
- **Leaflet** y **React-Leaflet** para la visualización de mapas geográficos con marcadores de sensores, permitiendo la ubicación espacial de la red de monitoreo.
- **Recharts** para gráficas estadísticas complementarias en el panel de inicio.
- **React Router 7** para el enrutamiento del lado del cliente, permitiendo navegación entre pantallas sin recarga completa de la página.
- **Microsoft Authentication Library (MSAL)** para la integración con Azure AD como proveedor de identidad institucional.
- **Axios** como cliente HTTP para la comunicación con el backend.

Pantallas principales

La interfaz se organiza en las siguientes pantallas, accesibles desde un menú lateral cuya visibilidad se adapta al rol de la persona usuaria:

Inicio (Dashboard): pantalla de bienvenida que presenta un mapa geográfico con la ubicación de todos los sensores registrados, indicadores de estado por colores, y un resumen de estadísticas del sistema (sensores activos, campañas en curso, alertas recientes). Es el punto de partida para la navegación.

Dispositivos: vista de gestión de la red de sensores. Muestra el listado completo de sensores con su estado actual, ubicación y antenas asociadas. Incluye un mapa interactivo con marcadores de estado y acceso al detalle de cada sensor. Desde esta pantalla se puede iniciar un monitoreo en vivo sobre un sensor seleccionado.

Monitoreo en vivo: flujo central de la plataforma para realizar mediciones inmediatas. La persona selecciona un sensor y una antena, define los parámetros de adquisición (rango de frecuencia, ancho de banda, resolución, ganancias) e inicia la captura. Durante la medición se visualiza en tiempo real el espectro de potencia (PSD) mediante Plotly.js, la evolución temporal en waterfall, y el audio demodulado AM/FM cuando está habilitado. La persona puede detener la medición o guardar la configuración como una nueva campaña.

Campañas: gestión de mediciones programadas. Permite crear campañas definiendo nombre, fechas de inicio y fin, horas de operación, intervalo entre mediciones, parámetros espectrales y sensores asignados. Incluye listado con filtros por estado, visualización de detalle, inicio y detención manual, y consulta de datos capturados. Los datos de campaña son la fuente para la generación de reportes de cumplimiento.

Alertas: revisión de incidentes y eventos operativos detectados por el sistema. Presenta un listado cronológico con filtros por sensor, tipo de alerta y rango de fechas. Cada alerta se vincula con los datos del sensor para facilitar el diagnóstico.

Configuración: panel administrativo para la gestión de recursos. Incluye CRUD del catálogo de antenas (tipo, rango de frecuencia, ganancia, código de inventario), administración de cuentas de usuario (roles y permisos), y configuración de parámetros globales del sistema.

Audio: reproductor de audio demodulado AM/FM, accesible desde el monitoreo en vivo o desde la vista de un sensor. Muestra métricas de modulación en tiempo real: profundidad para AM, excursión para FM.

Mecanismo de actualización en tiempo real

El frontend mantiene una conexión WebSocket con el backend para recibir actualizaciones sin necesidad de recarga manual. Los eventos recibidos incluyen cambios de estado de sensores, nuevas ubicaciones GPS, datos de espectro en tiempo real y confirmaciones de comandos. Estos eventos actualizan automáticamente los mapas, gráficas e indicadores en todas las pantallas relevantes, implementando un modelo de datos reactivo donde la interfaz refleja el estado del sistema en todo momento.

6.3.2. Backend

El backend es el centro de coordinación de la plataforma, implementado como un servidor Node.js con TypeScript sobre el framework Express. Su función principal es conectar a las personas usuarias (a través del frontend), los sensores RF en campo, la base de datos PostgreSQL y los servicios de postprocesamiento.

Tecnologías

- **Node.js 18+** como entorno de ejecución, seleccionado por su modelo de E/S no bloqueante que permite manejar múltiples conexiones concurrentes (frontend, sensores, WebSocket) en un solo proceso.
- **TypeScript** como lenguaje, compilado a JavaScript para producción, proporcionando verificación de tipos en tiempo de compilación.
- **Express** como framework HTTP, ofreciendo un modelo maduro de middleware, enrutamiento y manejo de solicitudes.
- **WebSocket nativo** (librería ws) para comunicación en tiempo real con el frontend, seleccionado sobre alternativas como Socket.IO por su menor sobrecarga de

protocolo.

- **PostgreSQL 13+** con cliente pg y pool de conexiones, como sistema de base de datos relacional.
- **TimescaleDB** como extensión opcional de PostgreSQL para optimización de datos de series temporales.
- **JSON Web Tokens (JWT)** y librería jose para la gestión de sesiones y autenticación.
- **Swagger/OpenAPI** para la documentación interactiva de la API.

Capas de la arquitectura

El backend está organizado en seis capas lógicas con responsabilidades bien definidas:

Capa de entrada HTTP: expone el servidor en el puerto configurado (3000) y aplica middleware común: CORS para permitir solicitudes del frontend, registro (logging) de cada solicitud con método y ruta, y parsing de JSON con un límite ampliado de 50 MB para permitir la recepción de tramas espectrales de alta resolución enviadas por los sensores.

Capa de autenticación y autorización: valida la identidad de la persona usuaria mediante dos mecanismos complementarios. Para el acceso institucional desde el frontend, se integra con Azure AD utilizando el protocolo OpenID Connect y la librería jose para validación de tokens. Para el acceso administrativo local, se utilizan credenciales almacenadas en la base de datos con hash bcrypt y tokens JWT firmados por el servidor. Los middleware de autenticación se aplican de forma selectiva según la ruta: los endpoints de ingesta de datos desde sensores operan con validación por dirección MAC; los endpoints de administración requieren sesión validada con rol apropiado.

Capa de rutas y controladores: organiza la lógica de negocio en módulos independientes: autenticación, gestión de sensores y antenas (CRUD, asignación), recepción de datos de campo (estado, GPS, espectro, audio), control remoto de sensores, campañas de medición, reportes de cumplimiento y configuración del sistema.

Capa de modelos y acceso a datos: abstrae las consultas SQL mediante funciones tipadas que utilizan un pool de conexiones pg.Pool con hasta 20 conexiones simultáneas. Las operaciones que modifican múltiples tablas se ejecutan dentro de transacciones PostgreSQL para garantizar consistencia.

Capa de comunicación en tiempo real: mantiene un servidor WebSocket que transmite cinco tipos de eventos al frontend: **sensor_status** (estados de sensores), **sensor_gps** (ubicaciones), **sensor_data** (espectro en tiempo real), **sensor_configure** y **sensor_stop** (comandos). Un segundo servidor WebSocket independiente gestiona el streaming de audio demodulado en formato Opus.

Capa de mantenimiento automático: ejecuta tareas programadas en intervalos regulares de 30 segundos. La tarea principal valida la antigüedad del último reporte de cada sensor activo y marca como offline aquellos que exceden el umbral de inactividad. Los cambios de estado se notifican al frontend mediante broadcast WebSocket.

API REST

La interfaz de programación del backend expone aproximadamente treinta endpoints organizados en ocho categorías funcionales, todas documentadas mediante especificación OpenAPI y accesibles a través de Swagger UI:

1. **Sensor Data:** endpoints POST para que los sensores envíen estado operativo, ubicación GPS, datos de espectro radioeléctrico y audio demodulado. Son los endpoints de mayor tráfico durante la operación normal.
2. **Sensor Query:** endpoints GET para consulta de datos históricos: último estado, última ubicación, últimos datos de espectro, datos por rango temporal y configuración activa, identificando cada sensor por su dirección MAC.
3. **Sensor Control:** endpoints POST para enviar comandos a los sensores: configuración de parámetros de escaneo, comando de detención y almacenamiento de configuración activa.
4. **Sensors Management:** CRUD completo de sensores más consultas por ID o MAC, y gestión de la relación sensor-antena (asignar, desasignar, consultar).
5. **Antennas Management:** CRUD completo del catálogo de antenas.
6. **Campaigns:** gestión de campañas con operaciones de creación, consulta, actualización, eliminación, inicio, detención y consulta de datos capturados.
7. **Reports:** generación de reportes de cumplimiento normativo que integran datos de campaña, ubicación y análisis del servicio de postprocesamiento.
8. **System:** endpoint raíz con versión de la API y directorio de endpoints.

6.3.3. Base de datos

La plataforma utiliza PostgreSQL como sistema de gestión de base de datos relacional. El esquema está organizado en cuatro dominios funcionales que agrupan quince tablas:

Dominio de identidad y acceso: almacena credenciales y roles de las personas usuarias (tabla `users`, con hash bcrypt para contraseñas) y un registro de auditoría de acciones sensibles (tabla `audit_logs`).

Dominio de sensores y configuración: mantiene el registro maestro de sensores RF identificados por MAC única (tabla `sensors`), el catálogo de antenas con sus parámetros técnicos (`antennas`), la relación muchos-a-muchos entre sensores y antenas con puerto de conexión (`sensor_antennas`), y los parámetros de adquisición por sensor: frecuencias, sample rate, ganancias, ventana, solapamiento y configuración de demodulación (`sensor_configurations`).

Dominio de datos de campo: almacena las series temporales generadas por los sensores: estado operativo con métricas de CPU, RAM, temperatura y latencia (`sensor_status`), ubicaciones GPS (`sensor_gps`), datos de espectro con arreglos de potencia, métricas de demodulación AM/FM y asociación opcional a campaña (`sensor_data`), y alertas operativas históricas (`sensor_history_alert`).

Dominio de campañas y reportes: define las campañas de medición con parámetros de programación y configuración espectral (`campaigns`), la asignación de sensores a campañas (`campaign_sensors`), el caché de reportes de cumplimiento en formato JSON (`compliance_reports_cache`), y parámetros globales del sistema en formato clave-valor (`system_configurations`).

La tabla `sensor_status`, que recibe actualizaciones periódicas de cada sensor y es la de mayor crecimiento, está configurada como hypertable de TimescaleDB, particionada por la columna `timestamp_ms`. Esta configuración permite compresión y retención automática de datos, y optimiza consultas temporales mediante índices compuestos por MAC y timestamp.

6.3.4. Despliegue

La plataforma se despliega en un servidor institucional privado de la ANE, accesible a través de la red interna de la entidad. La arquitectura de despliegue utiliza contenedores Docker coordinados por un proxy inverso Nginx:

- **Frontend:** imagen Docker construida en dos etapas: compilación con Node.js Alpine y servidor web con Nginx Alpine. Expone el puerto 80 con configuración para aplicaciones de página única (SPA) y compresión gzip.
- **Backend:** imagen Docker construida en dos etapas: compilación de TypeScript y ejecución con Node.js Alpine. Expone el puerto 3000.
- **Nginx:** actúa como punto único de entrada, enrutando el tráfico HTTP al frontend (archivos estáticos) y al backend (API REST en `/api/`, WebSocket en `/ws/`). La configuración incluye timeouts extendidos de una hora para la generación de reportes de larga duración.
- **PostgreSQL:** base de datos con extensión TimescaleDB, ejecutándose en el mismo servidor. En producción se recomienda la imagen `timescale/timescaledb:latest-pg14`.

La configuración de entorno se gestiona mediante variables de entorno (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, PORT) que permiten desplegar la misma imagen en entornos de desarrollo, pruebas y producción sin recompilación.

6.4. Entradas, salidas e interfaces

6.4.1. Entradas

La plataforma recibe datos de tres fuentes principales:

Desde el frontend: solicitudes HTTP de las personas usuarias para autenticarse, consultar información, configurar mediciones, gestionar campañas, revisar alertas y generar reportes. Estas solicitudes incluyen tokens de autenticación (JWT o Azure AD) y parámetros específicos según la operación.

Desde los sensores RF: datos de campo en formato JSON enviados mediante solicitudes POST:

- Estado del sensor: métricas de CPU por núcleo, RAM, disco, temperatura, latencia de red y logs del sistema operativo del dispositivo.
- Ubicación GPS: latitud, longitud y altitud.
- Datos de espectro: arreglo de potencias PSD (P_{xx}), frecuencia inicial y final en Hz, marca de tiempo en epoch milliseconds, coordenadas de captura, y métricas de demodulación AM (profundidad) o FM (excursión).
- Audio demodulado: tramas de audio PCM codificadas en base64 o en formato Opus.

Desde el servicio de postprocesamiento: resultados de análisis de emisiones en formato JSON, con frecuencias centrales detectadas, anchos de banda, potencias, cruce con licencias y evaluación de cumplimiento por municipio (código DANE).

6.4.2. Salidas

La plataforma entrega:

Al frontend: respuestas JSON a todas las consultas y operaciones, eventos WebSocket para actualización en tiempo real de estado de sensores, GPS, datos de espectro y audio.

A los sensores RF: configuraciones de adquisición (frecuencias, sample rate, ganancias, ventana, solapamiento, demodulación) y listas de campañas asignadas con todos los parámetros necesarios para ejecutar mediciones programadas.

A las personas usuarias: reportes de cumplimiento normativo estructurados con emisiones detectadas, comparación contra licencias, evaluación por parámetro (frecuencia central, ancho de banda, potencia) y estimación de intensidad de campo.

6.4.3. Interfaces

Las interfaces de comunicación entre componentes son:

- **Frontend – Backend:** API REST sobre HTTP con formato JSON. Autenticación mediante tokens JWT en cabecera **Authorization**. WebSocket para eventos en tiempo real y streaming de audio.
- **Sensores – Backend:** API REST sobre HTTP con formato JSON. Identificación del sensor por dirección MAC. Los sensores actúan como clientes HTTP que envían datos (POST) y consultan configuración (GET).
- **Backend – PostgreSQL:** conexión TCP nativa mediante el cliente pg sobre el puerto 5432, utilizando un pool de conexiones configurable.
- **Backend – Postprocesamiento:** invocación HTTP síncrona al servicio Python Flask expuesto en el puerto 8000, con formato JSON para solicitudes y respuestas.
- **Nginx – Frontend:** entrega de archivos estáticos (HTML, JS, CSS, assets) por HTTP en puerto 80.
- **Nginx – Backend:** proxy inverso HTTP y WebSocket, con timeouts extendidos para operaciones de larga duración.

6.5. Decisiones de diseño o implementación

6.5.1. Separación frontend/backend con API REST

Se decidió separar el frontend y el backend como aplicaciones independientes comunicadas por API REST, en lugar de una aplicación monolítica con renderizado en el servidor. Esta decisión se fundamenta en tres consideraciones:

1. **Despliegue independiente:** el frontend (archivos estáticos) y el backend (servidor Node.js) pueden escalarse, actualizarse y desplegarse de forma independiente. El frontend se sirve desde Nginx como archivos estáticos con caché agresiva; el backend maneja la lógica de negocio y la concurrencia.

2. **Separación de responsabilidades:** el equipo de frontend puede iterar sobre la interfaz sin afectar la lógica del backend, y viceversa. La API actúa como contrato estable entre ambas capas.
3. **Cliente único vs. múltiples clientes:** aunque actualmente el frontend web es el único cliente, la arquitectura REST permite que en el futuro otros clientes (aplicaciones móviles, scripts de automatización, dashboards externos) consuman la misma API sin modificar el backend.

6.5.2. WebSocket sobre polling para tiempo real

La elección de WebSocket como mecanismo de comunicación en tiempo real, en lugar de polling HTTP periódico, se justifica por el perfil de tráfico del sistema:

- Durante un monitoreo activo, un sensor puede enviar datos de espectro cada pocos segundos. Con polling, el frontend tendría que consultar al backend repetidamente para detectar nuevos datos, generando tráfico innecesario y latencia.
- WebSocket permite que el backend notifique proactivamente al frontend solo cuando hay datos nuevos, en un modelo push eficiente.
- La librería ws (WebSocket nativo) fue seleccionada sobre Socket.IO por su menor sobrecarga de protocolo: no requiere el handshake adicional de Socket.IO ni sus mecanismos de fallback a long-polling, que son innecesarios en un entorno de red controlada.

6.5.3. PostgreSQL y TimescaleDB para datos de series temporales

La decisión de adoptar PostgreSQL como base de datos principal —y específicamente TimescaleDB como extensión para tablas de series temporales— fue la de mayor impacto en la arquitectura del backend. El sistema comenzó utilizando SQLite, una base de datos sin servidor que almacena toda la información en un solo archivo.

SQLite ofrecía ventajas iniciales de simplicidad: sin necesidad de instalar ni configurar un servidor de base de datos separado, sin autenticación, con despliegue trivial. Sin embargo, a medida que el volumen de datos creció —particularmente en las tablas de estado de sensores y datos de espectro— se manifestaron limitaciones críticas:

- El límite práctico de aproximadamente 2 GB por archivo de base de datos comenzó a causar degradación y bloqueos.

- SQLite no soporta concurrencia real de escritura: múltiples sensores enviando datos simultáneamente generaban contención de bloqueo.
- Las consultas analíticas sobre grandes volúmenes de datos históricos (necesarias para reportes) tenían rendimiento deficiente.

La migración a PostgreSQL resolvió estas limitaciones y habilitó capacidades nuevas: tipos de datos nativos para valores geoespaciales y series temporales, transacciones ACID completas en operaciones multi-tabla, índices compuestos para consultas por sensor y rango temporal, y compresión y retención automática de datos antiguos mediante TimescaleDB. La migración se realizó de forma progresiva, actualizando primero la capa de conexión, luego los modelos de datos y finalmente las rutas con consultas directas.

6.5.4. Microservicio de postprocesamiento en Python

El análisis espectral avanzado —detección de emisiones, estimación de piso de ruido por escalones, separación de regiones por valles, cruce con base de datos de licencias— se implementó como un servicio independiente en Python (Flask), separado del backend Node.js.

Esta decisión responde a la naturaleza del procesamiento requerido: el análisis involucra cálculos numéricos intensivos (FFT, integración de potencia, suavizado Savitzky-Golay, detección de picos, evaluación de cumplimiento) para los cuales el ecosistema Python ofrece librerías maduras y optimizadas (NumPy, SciPy, Pandas). Implementar estas mismas rutinas en Node.js habría requerido reescribir algoritmos complejos o depender de puentes a librerías nativas con mantenimiento incierto.

La separación como microservicio también permite escalar el postprocesamiento de forma independiente, asignar más recursos computacionales al servicio Python sin afectar la capacidad de respuesta del backend, y mantener ciclos de desarrollo separados para la lógica de análisis espectral.

6.5.5. Separación de canales WebSocket para datos y audio

El audio demodulado se transmite por un canal WebSocket independiente del canal de datos de sensores. Esta separación permite tratar cada flujo según sus requisitos específicos:

- El canal de datos transmite eventos discretos (estado, GPS, espectro) con frecuencia de segundos y payloads de tamaño moderado.
- El canal de audio transmite tramas continuas en formato Opus con frecuencia de milisegundos, requiriendo menor latencia y mayor throughput.

- La separación evita que la carga de streaming de audio afecte la entrega de datos de espectro durante un monitoreo, y viceversa.

6.6. Problemas presentados y ajustes realizados

A continuación se documentan los problemas identificados durante el desarrollo y operación de la plataforma, las evidencias recolectadas, el análisis de causas, las decisiones de ajuste tomadas y los resultados obtenidos.

Tabla 6.1: Problemas presentados, decisiones y ajustes realizados en la plataforma web.

Problema	Evidencia	Causa probable	Decisión o Resultado
Límite de capacidad de SQLite causaba bloqueos en producción al alcanzar 2 GB de datos acumulados	Logs de error del servidor, mediciones de tamaño de archivo de base de datos	SQLite no está diseñado para volúmenes de series temporales con múltiples escrituras concurrentes desde sensores	Migración completa a PostgreSQL con extensión TimescaleDB. Se reescribió la capa de conexión, modelos de datos y rutas con consultas directas manteniendo compatibilidad de formatos
Endpoint de campañas para sensores con nombre incorrecto y resultados duplicados en asignaciones múltiples	Logs de consulta de sensores físicos, pruebas de integración	Falta de deduplicación en consulta SQL de asignación sensor-campaña y nomenclatura inconsistente del endpoint	Corrección del nombre del endpoint, incorporación de lógica de deduplicación, adición de filtro opcional por estado y validación de campos requeridos
			Sensores reciben lista única de campañas asignadas con parámetros completos para ejecutar mediciones

Problema	Evidencia	Causa probable	Decisión o Resultado
Timeouts en generación de reportes de cumplimiento para campañas con grandes volúmenes de datos	Errores HTTP 504 en frontend, logs de timeout en Nginx	Timeouts por defecto (30–60 s) insuficientes para el pipeline completo de consulta, post-procesamiento y armado de respuesta	Configuración de timeouts extendidos a 3600 s en servidor Node.js y proxy Nginx (proxy_read_timeout, proxy_connect_timeout, proxy_send_timeout)
Sensores que dejaban de reportar permanecían con estado activo en la plataforma	Reportes de personas usuarias, inspección de estados inconsistentes en base de datos	Ausencia de mecanismo automático de verificación de frescura de datos de sensores	Implementación de tarea programada cada 30 s que marca como offline sensores sin reporte reciente y notifica cambios al frontend por WebSocket
Carga inicial lenta del frontend por inclusión de Plotly.js en el bundle principal	Métricas de Lighthouse, reportes de personas usuarias sobre tiempo de carga	Plotly.js (3 MB) se incluía en el bundle principal, bloqueando renderización	Separación de Plotly.js en chunk independiente mediante code splitting en Vite; pantallas sin gráficos no esperan su carga
Errores de Mixed Content en producción con HTTPS: conexiones WebSocket y API bloqueadas por navegador	Errores en consola del navegador, fallos en conexión WebSocket, datos en tiempo real sin actualizar	URLs del backend configuradas como absolutas (HTTP) desde página servida por HTTPS	Migración a URLs relativas (/api, /ws) en variables de entorno de compilación, delegando resolución al proxy Nginx

Problema	Evidencia	Causa probable	pro-ajuste	Decisión o Resultado
Inconsistencia de sesión entre múltiples pestañas del navegador	Reportes de personas usuarias, pruebas con múltiples pestañas abiertas	Estado de autenticación y WebSocket gestionado por pestaña sin coordinación	Revisión del flujo de MSAL para propagar eventos de login/logout entre pestañas y manejo de renovación de tokens	Comportamiento consistente de sesión independiente del número de pestañas abiertas

6.7. Validación, pruebas o evidencias

La plataforma web ha sido validada mediante múltiples estrategias de verificación a lo largo de su desarrollo y operación:

Pruebas funcionales manuales: cada pantalla del frontend y cada endpoint de la API han sido verificados manualmente durante el desarrollo, cubriendo los flujos principales de autenticación, monitoreo en vivo, creación de campañas, revisión de alertas y generación de reportes.

Pruebas de integración con sensores físicos: se ha verificado la comunicación extremo a extremo con sensores HackRF reales, incluyendo el envío de configuraciones, la recepción de datos de espectro y estado, y la transmisión de audio demodulado AM/FM. El simulador de sensores AM/FM documentado en el backend permite reproducir escenarios de prueba sin hardware físico.

Documentación interactiva de API: la especificación OpenAPI publicada a través de Swagger UI permite verificar la estructura de cada endpoint, los esquemas de solicitud y respuesta, y ejecutar pruebas directamente desde el navegador mediante la función “Try it out”.

Validación en producción: la plataforma se encuentra en operación en el servidor institucional de la ANE, accesible a través de la red interna. El uso continuo por parte del personal técnico constituye una validación operativa de las funcionalidades implementadas.

Logs de servidor: el backend registra cada solicitud HTTP con método, ruta, marca de tiempo y código de respuesta, permitiendo auditoría de la actividad y diagnóstico de problemas. Los logs de Nginx complementan esta trazabilidad con información de tiempos de respuesta y estado de las conexiones.

Mantenimiento automático verificado: la tarea programada de validación de estado de sensores se monitorea a través de los eventos WebSocket que notifican cambios de estado al frontend. Los sensores que dejan de reportar son marcados como offline de forma automática, y este comportamiento ha sido confirmado en operación.

6.8. Aprendizajes y transferencia de conocimiento

Los siguientes aprendizajes se derivan de la experiencia de diseño, implementación y operación de la plataforma web:

1. **Dimensionar la persistencia desde el inicio según la carga esperada:** SQLite fue adecuado para desarrollo y prototipado, pero mantener una base de datos sin servidor para una carga de producción con múltiples escritores concurrentes y grandes volúmenes de series temporales generó un cuello de botella que requirió una migración significativa. La enseñanza es que la elección de la base de datos debe basarse en el perfil de carga previsto —volumen, concurrencia, tipo de consultas— y no solo en la conveniencia de desarrollo.
2. **La comunicación en tiempo real debe ser push, no pull:** el uso de WebSocket para notificar al frontend sobre nuevos datos y cambios de estado evitó la necesidad de implementar lógica de polling, reduciendo la carga en el servidor y la latencia de visualización. Para sistemas donde los datos fluyen de forma continua desde múltiples fuentes, el patrón push es la estrategia correcta.
3. **Los timeouts deben calibrarse según la operación más costosa:** los valores por defecto de los servidores HTTP (30 segundos) no contemplan operaciones de análisis que involucran consultas complejas, procesamiento numérico y cruce con bases de datos externas. Identificar la operación de mayor duración —los reportes de cumplimiento— y configurar todos los timeouts de la cadena (servidor, proxy) de forma consistente evita fallos intermitentes difíciles de diagnosticar.
4. **La separación de canales por tipo de dato simplifica el manejo de capacidades de servicio:** mantener datos de espectro y audio en canales WebSocket independientes permitió tratar cada flujo con sus propios requisitos de latencia y frecuencia, sin que la carga de streaming de audio afectara la entrega de datos de espectro.
5. **Las URLs relativas eliminan problemas de configuración por entorno:** migrar de URLs absolutas a relativas para API y WebSocket resolvió permanentemente los errores de Mixed Content y permitió que una misma compilación funcione en desarrollo, VPN y producción sin reconfiguración.

6. **La división de código es necesaria en aplicaciones con dependencias de visualización pesadas:** las librerías de gráficos científicos pueden dominar el tamaño del bundle. Planificar code splitting desde el inicio evita resolver problemas de rendimiento cuando la aplicación ya está en producción.
7. **El mantenimiento automático del estado elimina falsos positivos:** sin verificación periódica de la frescura de datos de sensores, los equipos desconectados aparecían como operativos. La automatización de esta verificación, con notificación por WebSocket, eliminó este problema y redujo la carga de monitoreo manual del personal.
8. **La documentación interactiva de API acelera la integración:** contar con Swagger UI permitió que el equipo de frontend y los desarrolladores de los sensores pudieran explorar y probar la API sin depender de documentación estática, reduciendo el tiempo de integración.

6.9. Limitaciones

La plataforma web en su estado actual presenta las siguientes limitaciones:

1. **Base de datos sin alta disponibilidad:** PostgreSQL opera en instancia única, sin replicación ni mecanismo de failover automático. Una falla del servidor de base de datos interrumpiría la totalidad de la plataforma.
2. **Escalamiento vertical del backend:** el backend está diseñado como un solo proceso Node.js. Si bien el pool de 20 conexiones soporta la concurrencia actual, la arquitectura no contempla escalamiento horizontal a múltiples instancias, lo que requeriría un backend compartido para WebSocket (como Redis) y balanceo de carga.
3. **Sin balanceo de carga:** el punto único de entrada a través de Nginx no está configurado para distribuir tráfico entre múltiples réplicas del backend.
4. **Dependencia de conectividad de sensores:** el modelo de operación asume que los sensores consultan periódicamente su configuración. Si un sensor pierde conectividad en el momento de consulta, puede no recibir una campaña programada hasta el siguiente ciclo.
5. **Postprocesamiento síncrono sin cola:** la generación de reportes invoca al servicio Python de forma síncrona. Si el servicio no está disponible, el reporte falla sin mecanismo de reintento automático o encolamiento.
6. **Tablas de series temporales sin optimización completa:** solo `sensor_status` está configurada como hypertable de TimescaleDB. Las tablas `sensor_data` y `sensor_gps`,

que también almacenan series temporales, pueden presentar degradación al crecer en volumen.

7. **Tamaño de dependencias de visualización:** Plotly.js (3 MB) sigue siendo una dependencia significativa que afecta la primera visita a las pantallas de monitoreo, incluso estando en un chunk separado.
8. **Sin pruebas automatizadas:** el desarrollo no incluye pruebas unitarias ni de integración automatizadas, dependiendo de validación manual para verificar cambios.
9. **Sin modo offline:** la aplicación requiere conectividad constante con el backend. No hay almacenamiento local de datos para consulta sin conexión.
10. **Sin diseño responsivo para dispositivos móviles:** la interfaz está optimizada para estaciones de escritorio, que es el contexto de uso principal. No se ha priorizado la adaptación a pantallas pequeñas.

6.10. Recomendaciones específicas

Con base en las limitaciones identificadas y la experiencia acumulada, se proponen las siguientes recomendaciones para la evolución de la plataforma:

1. **Alta disponibilidad de base de datos:** configurar replicación streaming en PostgreSQL con al menos una réplica en caliente, permitiendo continuidad operativa ante falla del servidor principal.
2. **Escalamiento horizontal del backend:** evaluar la introducción de Redis como backend compartido para WebSocket y como caché de datos frecuentes, habilitando múltiples instancias del servidor Node.js coordinadas detrás de un balanceador de carga.
3. **Optimización de todas las tablas de series temporales:** migrar `sensor_data` y `sensor_gps` a hypertables de TimescaleDB, y configurar políticas de compresión y retención automática para todas las tablas de series temporales.
4. **Resiliencia del postprocesamiento:** implementar una cola de trabajos (por ejemplo, Bull con Redis) para las solicitudes de reportes, con reintentos automáticos con backoff exponencial cuando el servicio Python no esté disponible.
5. **Pruebas automatizadas:** introducir pruebas unitarias con Vitest/Jest para el frontend y backend, y pruebas de integración para los flujos críticos (autenticación, monitoreo en vivo, creación de campañas, generación de reportes).

6. **Evaluación de alternativas de visualización:** analizar la viabilidad de reemplazar Plotly.js por ECharts o una implementación basada en Canvas/WebGL para reducir el tamaño del bundle sin perder capacidades de visualización.
7. **Monitoreo operacional:** instrumentar el backend con métricas de Node.js (consumo de memoria, event loop lag, latencia de respuesta, conexiones activas) y exportarlas a un sistema de monitoreo institucional.
8. **Internacionalización preparatoria:** si bien el contexto actual es exclusivamente colombiano, estructurar los textos de la interfaz de forma que una futura traducción no requiera reescribir componentes, utilizando una capa de localización (i18n).
9. **Revisión de seguridad:** auditar los endpoints de administración para verificar que todos requieran autenticación y autorización por rol, y documentar explícitamente el modelo de seguridad (endpoints públicos de ingesta, endpoints autenticados de administración).
10. **Documentación operativa:** elaborar guías de operación para el personal técnico de la ANE, cubriendo procedimientos de despliegue, respaldo, recuperación y diagnóstico de problemas comunes.

6.11. Repositorios

El software de la plataforma web está organizado en dos módulos dentro del repositorio del sistema. A continuación se presentan las fichas técnicas de cada componente.

Tabla 6.2: Repositorio asociado al frontend de la plataforma web.

Nombre del software	ANE-HX Frontend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Interfaz web para la operación y administración del sistema de monitoreo espectral
Entradas	Interacciones de la persona usuaria, datos en tiempo real por WebSocket, respuestas de API REST
Salidas	Visualizaciones de espectro y mapas, formularios de configuración, reportes de cumplimiento, streaming de audio
Dependencias	React 19, TypeScript, Vite 5, Tailwind CSS 3, Plotly.js, Leaflet, MSAL, Axios, Nginx
Estado actual	En producción en servidor institucional privado

Tabla 6.3: Repositorio asociado al backend de la plataforma web.

Nombre del software	ANE-HX Backend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Centro de coordinación: API REST, WebSocket, persistencia, gestión de campañas, alertas y reportes
Entradas	Solicitudes HTTP del frontend y sensores; datos de espectro, GPS, estado y audio desde sensores RF
Salidas	Respuestas JSON, eventos WebSocket, reportes de cumplimiento normativo
Dependencias	Node.js 18+, Express, TypeScript, PostgreSQL 13+, TimescaleDB (opcional), ws, Swagger/OpenAPI, Azure AD, servicio Python de postprocesamiento
Estado actual	En producción en servidor institucional privado

Capítulo 7

Análisis espectral y localización en la nube

7.1. 1. Propósito de la sección

Esta sección documenta el módulo de post procesamiento espectral desarrollado para analizar, en un entorno centralizado, las capturas generadas por el sistema de monitoreo. El objetivo de este avance es explicar cómo el software recibe una traza espectral, organiza la información de frecuencia y amplitud, selecciona un modo de operación y obtiene los primeros parámetros técnicos de las emisiones detectadas.

En esta etapa se describe el proceso hasta la estimación de la **frecuencia central**, el **ancho de banda**, la **potencia** y, cuando la captura corresponde a un caso aplicable, **MER/BER**. La verificación detallada de cumplimiento contra licencias, el uso formal de DANE/municipio, la localización administrativa, las pruebas finales y las recomendaciones quedan como continuación de esta misma sección.

El propósito del módulo no es reemplazar la adquisición SDR realizada en el borde, sino convertir las capturas recibidas en resultados interpretables para la plataforma. Por esta razón, el capítulo se enfoca en la función del software dentro del flujo general del sistema, en sus entradas y salidas principales, y en las decisiones iniciales usadas para diferenciar ruido, picos solicitados y emisiones candidatas.

7.2. 2. Contexto del módulo dentro del sistema ANE-HX

El sistema completo puede entenderse como una cadena de sensado, transmisión, procesamiento y visualización. En el borde, el nodo de monitoreo recibe señales de radiofrecuencia

mediante el hardware SDR y genera una captura espectral. Posteriormente, esa captura se envía hacia la plataforma central o se analiza de forma local mediante archivos de entrada. El módulo descrito en esta sección se ubica justamente después de la adquisición y antes de la presentación de resultados al usuario.

La información que llega al módulo corresponde principalmente a una serie de muestras espectrales asociadas a un rango de frecuencias. Con esos datos, el software reconstruye el eje de frecuencia, identifica regiones de interés y entrega resultados estructurados que pueden ser usados por una interfaz web, por un servicio de consulta o por un proceso posterior de validación.

Nota sobre el alcance actual: En este punto del desarrollo, la palabra localización se interpreta de forma limitada. El módulo no calcula una posición geográfica exacta de la emisión ni realiza triangulación con varios nodos. La relación espacial disponible se maneja a través de filtros administrativos, como código DANE, lista de DANEs o municipio, los cuales permiten contextualizar una medición dentro de una zona de análisis. El desarrollo detallado de esta parte queda para la continuación de la sección.

7.3. 3. Arquitectura del módulo de postprocesamiento

El desarrollo se organiza como un módulo de Python separado en archivos con responsabilidades específicas. Esta división facilita el mantenimiento, la prueba por componentes y la integración posterior con una plataforma web o con una API.

La arquitectura permite ejecutar el mismo procesamiento desde dos frentes: por consola, usando `main.py`, o como servicio, usando `server_flask.py`. En ambos casos, la función central es `process_input`, definida en `src/processor.py`, porque allí se decide el modo de análisis y se construye la respuesta final.

A continuación se detallan los componentes y archivos principales del módulo:

Archivo	Función dentro del módulo
main.py	Permite ejecutar el procesamiento desde consola. Lee un archivo JSON o una trama en línea, interpreta argumentos como picos, cumplimiento, DANE, umbral y rutas auxiliares, y entrega una respuesta estructurada.
server_flask.py	Expone el procesamiento mediante una API basada en Flask. Permite recibir solicitudes HTTP con una captura espectral y parámetros de análisis.
src/payload_parser.py	Convierte el contenido del JSON de entrada en un objeto espectral interno. Valida que existan las muestras y los límites de frecuencia necesarios.
src/spectrum_frame.py	Define la estructura SpectrumFrame, que contiene amplitudes, frecuencia inicial, frecuencia final, eje de frecuencias y resolución espectral.
src/processor.py	Orquesta el flujo principal. Normaliza la entrada, selecciona el modo de operación, aplica corrección si existe, llama a los algoritmos de detección y arma la salida final.
src/spectral_analysis.py	Contiene las funciones de análisis espectral, detección de picos, segmentación de emisiones y estimación de parámetros como frecuencia central, ancho de banda, potencia, MER y BER cuando aplica.
src/calibration_io.py	Contiene utilidades para trabajar con archivos auxiliares, como bases de licencias o información de comparación. En este avance solo se menciona su papel dentro de la arquitectura.
src/power_utils.py	Contiene funciones asociadas al cálculo robusto de potencia. Integra la energía de la emisión en dominio lineal y evita reportar valores no finitos cuando la captura no permite una estimación confiable.

7.4. 4. Flujo general de procesamiento

El flujo de trabajo inicia con una captura espectral en formato JSON. El sistema espera una lista de valores espectrales y dos frecuencias límite que definan el intervalo analizado. A partir de esa información, se construye una representación interna de la traza y se ejecuta el modo de operación correspondiente.

De forma resumida, el algoritmo ejecuta los siguientes pasos secuenciales:

- **Recepción:** Obtención del JSON espectral o de una trama equivalente.

- **Lectura:** Extracción de los campos principales de frecuencia y amplitud.
- **Construcción:** Instanciación interna del objeto SpectrumFrame.
- **Corrección:** Aplicación opcional de corrección de ganancia mediante archivos auxiliares, si se suministran.
- **Selección de Modo:** Elección del modo de operación según los picos recibidos y la bandera de cumplimiento.
- **Umbralización:** Cálculo adaptativo del umbral de detección o uso del umbral fijo configurado por el usuario.
- **Búsqueda:** Localización de picos o regiones candidatas de emisión.
- **Estimación de F_c :** Cálculo de la frecuencia central de cada emisión candidata.
- **Estimación de BW:** Estimación del ancho de banda mediante el criterio de $-x$ dB u OBW (cuando está disponible).
- **Estimación de Potencia:** Integración de la potencia de la emisión en la región medida.
- **Estimación de Calidad:** Estimación opcional de MER/BER si la señal corresponde a un caso de Televisión Digital Terrestre (TDT) compatible.
- **Salida:** Generación de una estructura final formateada con el modo empleado y los resultados métricos.

Este flujo conserva una separación clara entre los datos de entrada, la representación interna y los resultados. Esa decisión es importante porque evita que cada componente del sistema tenga que interpretar directamente el JSON original.

7.5. 5. Entradas del sistema

Las entradas principales del módulo son la captura espectral y algunos parámetros de control. No todos los campos son obligatorios en todos los modos; algunos se usan solo cuando se quiere hacer análisis por picos o preparar una validación posterior de cumplimiento.

El parser de entrada también acepta algunos alias de nombres para los campos principales, como pxx, PSD, f_start_hz o f_stop_hz. Esta flexibilidad permite integrar capturas provenientes de fuentes distintas sin cambiar la estructura general del procesamiento.

Entrada	Tipo esperado	Descripción
Pxx	Lista numérica	Vector de amplitudes o potencias espectrales de la captura. Es la señal base que se analiza.
start_freq_hz	Numérico	Frecuencia inicial de la captura, expresada en Hz.
end_freq_hz	Numérico	Frecuencia final de la captura, expresada en Hz.
picos	Lista numérica	Frecuencias específicas solicitadas por el usuario. Si se entregan, el sistema entra automáticamente al modo de análisis por picos.
cumplimiento	Entero (0/1)	Bandera que indica si se desea ejecutar el modo de cumplimiento normativo cuando no se especifican picos.
corr	Ruta de archivo	Archivo opcional de corrección de ganancia. Permite ajustar la traza antes del análisis.
lic	Ruta de archivo	Archivo opcional de licencias (en este avance solo se registra como dependencia para el modo de cumplimiento).
dane, danes, municipio	Texto o lista	Filtros administrativos que permiten contextualizar la captura dentro de una zona geográfica de análisis.
umbral_db	Numérico opcional	Umbral en dB sobre el piso de ruido usado para decidir si una región puede considerarse candidata de emisión.
delta_fc_khz	Numérico opcional	Tolerancia de frecuencia central, en kHz, usada para comparar o emparejar frecuencias cercanas.
delta_bw_khz	Numérico opcional	Tolerancia de ancho de banda, en kHz (se menciona como parámetro disponible, pero no se desarrolla la evaluación normativa completa en este avance).

7.6. 6. Salidas del sistema

La salida del módulo se entrega como una estructura de datos limpia con información general del procesamiento y una lista de resultados empacados. En este avance se docu-

mentan las salidas relacionadas con el modo, picos, estado de detección, frecuencia central, ancho de banda, potencia y MER/BER cuando el caso aplica.

Esta estructura desacoplada permite que otros módulos consuman los resultados sin depender de logs o mensajes impresos en consola, facilitando su integración con una plataforma web, una base de datos o una capa de visualización.

Tabla 6.1: Salidas generales del procesamiento

Salida	Tipo esperado	Descripción
mode	Texto	Modo de operación seleccionado: all_emissions, peaks o compliance.
cumplimiento	Entero (0/1)	Valor de la bandera de cumplimiento recibido en la entrada.
picos_count	Entero	Cantidad de picos específicos entregados por el usuario.
picos	Lista numérica	Copia de los picos solicitados originalmente, si existen.
umbral	Numérico	Umbral absoluto calculado o configurado que sirvió de apoyo para la detección.
num_emissions	Entero	Cantidad total de emisiones o candidatos válidos encontrados.
results	Lista de objetos	Lista con los resultados individuales de cada emisión, pico solicitado o candidato analizado.

Tabla 6.2: Campos principales dentro de cada resultado (results)

Campo	Tipo esperado	Descripción
status	Texto	Indica si se encontró una emisión real o si el punto analizado se consideró ruido de fondo.
reason	Texto	Explicación breve cuando no se logra detectar una emisión válida en la región.
requested_pico	Numérico	Pico exacto solicitado por el usuario (aplica solo para el modo de análisis por picos).
matched_peak_hz	Numérico	Frecuencia del pico real detectado más cercano al punto solicitado.
delta_match_hz	Numérico	Diferencia en Hz entre el pico solicitado y el pico detectado asociado.
fc_hz	Numérico	Frecuencia central estimada de la emisión, expresada en Hz.
fc_mhz	Numérico	Frecuencia central estimada de la emisión, expresada en MHz.
bw_hz	Numérico	Ancho de banda reportado para la emisión, expresado en Hz.
bw_khz	Numérico	Ancho de banda reportado para la emisión, expresado en kHz.
power_dbm	Numérico o nulo	Potencia estimada de la emisión. Si el cálculo matemático no es finito, se conserva como nulo para no forzar valores artificiales.
snr_db	Numérico opcional	Relación señal a ruido (SNR) estimada cuando la función interna la provee.
mer_db	Numérico opcional	MER estimado para señales compatibles, especialmente TDT. Solo aparece si aplica.
ber_est	Numérico opcional	BER estimado para señales compatibles, especialmente TDT. Solo aparece si aplica.

7.7. 7. Modos de operación

El módulo selecciona automáticamente el modo de operación usando dos elementos de entrada: la lista de picos y la bandera de cumplimiento. La regla implementada en el enrutador es simple:

1. Si hay picos en la lista, se prioriza el **Análisis por picos**.
2. Si no hay picos y cumplimiento = 1, se activa el modo de **Cumplimiento contra licencias**.
3. Si no hay picos y cumplimiento = 0, se activa la **Detección automática** de emisiones.

Modo	Condición de entrada	Uso principal
all_emissions	Lista de picos vacía y cumplimiento = 0.	Detectar automáticamente emisiones candidatas dentro de toda la captura espectral y reportar sus primeros parámetros técnicos.
peaks	Se entrega una lista con picos, sin importar el valor de cumplimiento.	Analizar frecuencias específicas solicitadas por el usuario y determinar si cerca de ellas existe una emisión o si el punto se comporta como ruido.
compliance	Lista de picos vacía y cumplimiento = 1.	Preparar el procesamiento para comparar emisiones medidas contra información de referencia. En este avance se llega hasta la medición técnica de la emisión y la estimación MER/BER si aplica, sin desarrollar todavía el análisis completo de licencias.

7.7.1. Detección automática

En el modo all_emissions, el sistema analiza toda la traza espectral. Primero estima un umbral de detección a partir del piso de ruido o del valor indicado por el usuario mediante umbral_db. Después busca picos o regiones candidatas y, para cada una, mide frecuencia central, ancho de banda y potencia. Si la emisión corresponde a un caso TDT compatible, también puede estimar MER y BER. Este modo es útil cuando el usuario no sabe previamente en qué frecuencias existen emisiones de interés. La salida principal de esta etapa es una lista de candidatos con su estado y sus parámetros espectrales iniciales.

7.7.2. Análisis por picos

En el modo peaks, el usuario entrega una o varias frecuencias de interés. El sistema ubica la frecuencia solicitada dentro del eje de la captura y busca un pico cercano que pueda representar una emisión real. Si encuentra una región suficientemente consistente, reporta los parámetros medidos de la emisión. Si no encuentra evidencia suficiente, el resultado se marca como ruido o como pico no asociado a una emisión clara. Este modo es conveniente cuando se desea comprobar una frecuencia puntual, por ejemplo una frecuencia observada visualmente en una gráfica o una frecuencia que se sospecha activa.

7.7.3. Cumplimiento contra licencias

En el modo compliance, el sistema queda preparado para relacionar las emisiones detectadas con una base de referencia, normalmente una base de licencias. Para entrar en este modo no deben entregarse picos y la bandera cumplimiento debe estar activa. En este

avance no se desarrolla la comparación completa de cumplimiento. Solo se documenta que el modo existe dentro del enrutamiento del software y que utiliza la misma base inicial de detección espectral para obtener los parámetros medidos que luego pueden ser comparados con valores nominales.

7.8. 8. Análisis espectral realizado

El análisis espectral del módulo se centra en convertir una región detectada de la traza en parámetros técnicos que puedan ser interpretados por la plataforma. En este avance se documentan cuatro salidas principales: frecuencia central, ancho de banda, potencia y, solo cuando la captura lo permite, MER/BER. Estos valores no se escriben de forma manual, sino que se derivan de la región espectral detectada por el algoritmo.

7.8.1. Frecuencia central

La frecuencia central representa la frecuencia principal o representativa de una emisión detectada dentro de la captura. En el módulo, esta frecuencia se entrega principalmente en dos unidades: Hz mediante `fc_hz` y MHz mediante `fc_mhz`. El proceso inicia con el eje de frecuencias generado por `SpectrumFrame`. Si la captura contiene N muestras, el objeto construye un vector de frecuencias entre `start_freq_hz` y `end_freq_hz`. Con ese eje, cada muestra de amplitud queda asociada a una frecuencia específica.

Cuando el sistema detecta un pico candidato, se toma una frecuencia inicial de referencia desde el eje de frecuencias. Luego se delimita una región de análisis alrededor del candidato y se llama a las funciones de medición espectral. El resultado final de esa medición se almacena como frecuencia central de la emisión.

En términos operativos, la frecuencia central puede provenir de tres situaciones:

- En detección automática, se obtiene a partir del pico o segmento candidato encontrado por el algoritmo.
- En análisis por picos, se calcula a partir de la emisión real encontrada cerca de la frecuencia solicitada.
- En modo de cumplimiento, se conserva como frecuencia central medida para que posteriormente pueda compararse con una frecuencia nominal.

La frecuencia central no debe interpretarse simplemente como el punto más alto de la traza. En algunas capturas, especialmente cuando hay señales anchas o regiones con ruido, el punto de mayor amplitud puede no representar de forma adecuada el centro de la emisión.

Por eso el módulo no solo toma un valor visual, sino que usa la región detectada para estimar una frecuencia más representativa.

Campo	Unidad	Interpretación
fc_hz	Hz	Frecuencia central estimada en la unidad base usada internamente por el procesamiento.
fc_mhz	MHz	Versión escalada de fc_hz, más cómoda para reportes y lectura humana.
matched_peak_hz	Hz	Frecuencia de pico asociada a una solicitud del usuario en modo peaks. Puede servir como referencia inicial, pero no siempre reemplaza la frecuencia central estimada.
delta_match_hz	Hz	Diferencia entre la frecuencia solicitada y el pico asociado para evaluar la cercanía.

7.8.2. Ancho de banda

El ancho de banda describe la extensión espectral ocupada por una emisión. En el software, este valor se obtiene después de delimitar la región asociada al pico o segmento candidato. La función de medición calcula dos referencias: un ancho a $-x$ dB respecto al pico y un ancho de banda ocupado u OBW, que representa el intervalo que contiene un porcentaje definido de la potencia de la emisión.

Para el reporte final, el procesador selecciona el ancho de banda más conveniente mediante una regla conservadora: si existe OBW válido, se prefiere este valor porque suele ser más estable en señales con mesetas, varios picos o modulaciones anchas. Si el OBW no está disponible, el sistema usa el ancho a $-x$ dB. Esta decisión evita depender de un solo bin o de una lectura visual de la gráfica.

El resultado se reporta como bw_hz y bw_khz. El primer campo mantiene la unidad interna del procesamiento y el segundo facilita la lectura del reporte. En modo de cumplimiento, el mismo valor medido se conserva como bw_medido_kHz, pero la comparación normativa completa queda para una etapa posterior de la sección.

Campo	Unidad	Interpretación
bandwidth_xdb_hz	Hz	Ancho calculado a $-x$ dB respecto al pico de la emisión.
obw_hz	Hz	Ancho de banda ocupado, calculado a partir de la potencia acumulada de la región.
bw_hz	Hz	Ancho de banda seleccionado para reportar la emisión.
bw_khz	kHz	Versión del ancho de banda reportado en una unidad más cómoda para lectura humana.
bw_medido_kHz	kHz	Nombre usado en modo de cumplimiento para conservar el ancho de banda medido.

7.8.3. Potencia

La potencia se estima a partir de la región espectral asociada a la emisión. Para evitar errores de interpretación, el cálculo no se realiza directamente en dBm como si fuera una suma lineal. El procedimiento convierte las muestras al dominio lineal, integra o suma la contribución de los bins dentro de la región seleccionada y finalmente devuelve el resultado en dBm. El módulo usa funciones de apoyo para hacer el cálculo más robusto. Por ejemplo, la función de potencia robusta permite integrar usando el ancho de bin del SpectrumFrame o una estimación basada en el eje de frecuencias.

Si la captura no permite calcular una potencia finita, el procesador evita reportar -inf, NaN o valores artificiales, y conserva el resultado como nulo. Esto es importante porque una potencia no finita no debe interpretarse como una medición real.

En los modos all_emissions y peaks, la salida principal se reporta como power_dbm. En modo de cumplimiento, la misma magnitud se puede conservar como p_medida_dBm. En este avance se documenta la medición de potencia, pero no se desarrolla todavía la evaluación frente a potencia nominal de una licencia.

Campo	Unidad	Interpretación
power_dbm	dBm	Potencia estimada de la emisión en los modos de detección general y análisis por picos.
p_medida_dBm	dBm	Nombre usado en modo de cumplimiento para conservar la potencia medida.
channel_power_dbm	dBm	Potencia de canal calculada durante la medición de parámetros de la emisión.
snr_db	dB	Relación señal a ruido estimada cuando la función de análisis la entrega.

7.8.4. MER/BER si aplica

El módulo también incluye una estimación opcional de MER y BER para señales compatibles con el caso TDT. Este cálculo no se aplica a todas las emisiones detectadas. La función primero verifica si la frecuencia central de la región cae dentro del rango TDT definido en el código y si el ancho de la banda es plausible para ese tipo de señal. Si esas condiciones no se cumplen, la función retorna un resultado vacío y el reporte no incluye campos de MER/BER.

Cuando el caso sí aplica, el sistema toma la banda medida como región de señal y usa regiones laterales o información fuera de banda como referencia de ruido. Con esa relación se estima una magnitud de calidad equivalente a MER en dB y luego se calcula un BER estimado para una modulación M-QAM, usando $M = 64$ en el flujo implementado para TDT.

Estos valores deben interpretarse como estimaciones de calidad obtenidas desde la traza espectral disponible, no como una demodulación completa de la señal. Por lo tanto, son útiles para orientar el diagnóstico de una captura TDT, pero deben validarse con mediciones especializadas si se requiere una certificación formal.

Campo	Unidad	Interpretación
mer_db	dB	MER estimado de la señal compatible. Solo aparece si se encuentran condiciones válidas para el cálculo.
ber_est	Adimensional	BER estimado a partir de la relación de calidad calculada para la señal.
f_center_hz	Hz	Frecuencia central de la banda usada internamente para el cálculo de MER/BER.
band_start_hz	Hz	Límite inferior de la banda evaluada.
band_stop_hz	Hz	Límite superior de la banda evaluada.

7.9. 9. Comparación contra licencias y uso de DANE/municipio

El proceso de validación normativa vincula las mediciones espectrales obtenidas por el módulo con la base de datos oficial. Este procedimiento permite determinar si una emisión detectada cuenta con permiso para operar en la zona y si respeta los parámetros técnicos asignados.

El emparejamiento y la evaluación se dividen en los siguientes pasos:

7.9.1. 1. Filtrado geográfico y asociación de licencia

Para optimizar el proceso y evitar falsos positivos con estaciones de otras regiones, la función `_match_licencia` ejecuta primero un filtro sobre el archivo CSV de licencias utilizando los parámetros `dane_filtro` y `municipio_filtro`. Esto reduce drásticamente el espacio de búsqueda al área geográfica de interés.

Una vez filtrada la base de datos, el algoritmo busca la licencia cuya frecuencia nominal ($f_{c_nominal}$) sea la más cercana a la frecuencia central medida (f_{c_medida}). Esta búsqueda está limitada por una ventana de tolerancia máxima definida por la constante `FC_MARGIN_MHZ`. Si ninguna licencia registrada cae dentro de esta ventana, la emisión se marca automáticamente en los resultados como **Licencia: NO**.

7.9.2. 2. Criterios de cumplimiento técnico

Cuando el sistema encuentra un match válido (una licencia asociada a la emisión), procede a evaluar tres criterios de cumplimiento de forma completamente independiente.

Criterio evaluado	Regla de cumplimiento	Comportamiento y consideraciones
Frecuencia Central (Cumple_FC)	$ f_c^{medida} - f_c^{nominal} \leq FC_MARGIN_MHZ$	La frecuencia central medida debe estar dentro de una ventana de tolerancia respecto a la frecuencia nominal de la licencia.
Ancho de Banda (Cumple_BW)	$BW_{medido} \leq BW_{nominal} + BW_MARGIN_KHZ$	Implementa una asimetría intencional. Si el ancho de banda medido es menor al nominal, siempre cumple (emitir con menos ancho de banda no es una infracción). Solo falla si el exceso supera la tolerancia <code>BW_MARGIN_KHZ</code> .
Potencia (Cumple_P)	$P_{medida} \leq P_{nominal}$	Es la comparación más estricta, sin margen de tolerancia; cualquier exceso de potencia se considera un incumplimiento. Si la potencia medida no está disponible (por ejemplo, en un frame sin suficiente SNR), este campo se reporta como <i>None</i> .

7.9.3. 3. Manejo de múltiples jurisdicciones (Múltiples DANE)

El módulo está diseñado para soportar análisis en fronteras municipales o zonas extensas. Cuando se envían múltiples códigos DANE en la solicitud, el procesador no vuelve a calcular el espectro; en su lugar, repite el proceso de matching completo para cada jurisdicción de forma iterativa.

La salida de este proceso genera una estructura llamada `results_by_dane`, que es un diccionario que contiene una tabla de resultados independiente por cada DANE evaluado. Por compatibilidad de lectura, la clave principal `results` del JSON de salida siempre apunta a los resultados del primer DANE de la lista. Esto permite que una única captura espectral se valide simultáneamente contra los registros de licencias de varios municipios con alta eficiencia computacional.

7.10. 10. Localización administrativa

Para realizar una evaluación normativa precisa, el módulo requiere delimitar geográficamente la búsqueda de licencias. Esto evita falsos positivos cruzados con emisiones legítimas de otras regiones. El sistema está diseñado para capturar la ubicación administrativa desde diferentes interfaces, aplicar una estricta jerarquía de prioridades y normalizar los textos antes de efectuar cualquier comparación.

A continuación, se detallan el origen de estos parámetros, sus reglas de interpretación y el proceso de normalización en el backend.

7.10.1. 1. Origen del parámetro territorial

El sistema puede recibir la información geográfica desde tres frentes operativos distintos, adaptándose tanto a ejecuciones en caliente de la API como a pruebas locales por consola o reportes programados:

<i>Interfaz / Origen</i>	<i>Mecanismo de captura</i>	<i>Referencia en Código</i>
<i>Cliente / Petición HTTP</i>	<i>Se envía como el campo municipio dentro del cuerpo (body/meta) de la petición HTTP, o bien como un parámetro de consulta (query param) si únicamente se transmite la traza espectral.</i>	<i>server_flask.py (Líneas 308-336)</i>
<i>Consola (CLI) / Local</i>	<i>Se especifica de manera explícita en la terminal de comandos mediante el argumento formal – municipio.</i>	<i>main.py (Líneas 150-235)</i>
<i>Backend de Reportes</i>	<i>El backend centralizado construye un objeto consolidado de ubicacion (que empaqueta tanto el nombre del municipio como el código DANE) y lo inyecta al módulo de postprocesamiento al invocar el servicio.</i>	<i>reports.ts (Líneas 253-271)</i>

7.10.2. 2. Reglas de interpretación y jerarquía de prioridad

Debido a la diversidad de formatos en las entradas, el servicio implementa un flujo lógico interno para resolver ambigüedades. Las reglas de prioridad se aplican de forma secuencial en el enrutamiento de *main.py* (Líneas 206-225) y *server_flask.py* (Líneas 344-376) bajo el siguiente criterio:

1. **Prioridad Absoluta (danes):** Si la entrada contiene una lista explícita de códigos (danes), el procesador ignora los demás campos territoriales y la toma como la referencia definitiva para el análisis multijurisdiccional.
2. **Segunda Prioridad (dane):** En ausencia de una lista, el sistema evalúa si se suministró un único código identificador bajo el campo dane.
3. **Mecanismo de Compatibilidad:** Si no se detecta el parámetro dane, pero el campo municipio recibido es de tipo puramente numérico, el software lo interpreta automáticamente como un código DANE para mantener compatibilidad con lecturas indexadas.

Mecanismo de protección contra falsos positivos: Si el usuario no especifica nin-

gún filtro territorial en la solicitud (dejando vacíos tanto los campos de dane como de municipio), el procesador aborta preventivamente el matching con el archivo CSV de licencias. Esta restricción técnica —implementada en el wrapper de `processor.py` (Líneas 696-716)— impide realizar comparaciones globales ciegas que generarían reportes de cumplimiento erróneos.

7.10.3. 3. Normalización de texto y emparejamiento (Matching)

Dado que los nombres de los municipios pueden presentar variaciones de digitación (tildes, mayúsculas erróneas o espacios dobles), el módulo realiza un preprocesamiento higiénico del texto antes de consultar el archivo de licencias:

- **Filtro de Texto:** El nombre del municipio se convierte a mayúsculas sostenidas, se eliminan los caracteres con acentuación (tildes) y se colapsan los espacios en blanco sobrantes. Este texto limpio se contrasta contra la columna indexada `_mun_norm` en la base de datos, garantizando una coincidencia exacta sin importar errores de formato en origen. Las funciones encargadas de este saneamiento residen en `calibration_io.py` (Líneas 72-90) y el filtrado por municipio en la subrutina `comparar_parametros` (Líneas 492-508).
- **Resolución de Deltas:** El emparejamiento final lo ejecuta la función medular `comparar_parametros(...)` (Líneas 406-420). Al recibir el parámetro sanitizado `municipio_filtro`, extrae la licencia correspondiente del registro y calcula en el acto las desviaciones o deltas técnicas reales medidos en la traza (desviación de frecuencia central ΔF_c , exceso de ancho de banda ΔBW y diferencia de potencia ΔP).

7.11. 11. Interfaces del módulo

(Nota: La distribución funcional inicial se describe en la sección de Arquitectura. En fases posteriores se añadirá el detalle técnico de operación para cada interfaz de usuario).

- Consola con `main.py`
- API con `server_flask.py`

7.12. 12. Decisiones de diseño

El desarrollo del módulo de postprocesamiento requirió tomar decisiones algorítmicas específicas para garantizar que la caracterización de las emisiones fuera robusta frente a

diferentes tipos de modulaciones y escenarios espectrales. Las principales decisiones de diseño radican en los métodos seleccionados para calcular la frecuencia central, el ancho de banda, la potencia y los indicadores de calidad.

7.12.1. Estimación de Frecuencia Central (F_c)

Se decidió no utilizar el punto medio geométrico del span (ancho) detectado, ya que esto induce a errores en señales que no son perfectamente simétricas. En su lugar, el algoritmo calcula la frecuencia central como el centroide de potencia (baricentro energético) dentro de los límites de la emisión. Esta decisión garantiza que, en señales asimétricas como la FM comercial con piloto estéreo de 19 kHz, la medición refleje con precisión el verdadero centro de energía de la transmisión.

7.12.2. Estimación de Ancho de Banda (BW)

Dado que las formas espectrales varían drásticamente entre tecnologías, se optó por medir dos métricas de ancho de banda de forma paralela: Ancho de Banda Ocupado (OBW) y Ancho de Banda a $-x$ dB.

- **Regla de selección:** El sistema prefiere siempre reportar el OBW (que integra el 99 % de la energía de la emisión) porque es mucho más estable frente a señales con múltiples picos o mesetas planas, como OFDM (TDT) o FM estéreo.
- Solo se utiliza el criterio de $-x$ dB como método de respaldo si el cálculo del OBW no arroja un resultado válido.

7.12.3. Estimación de Potencia

Para evitar los errores matemáticos de sumar valores logarítmicos, el diseño establece que las amplitudes deben convertirse estrictamente de dBm a dominio lineal (W/Hz) antes de procesarse.

- Se implementaron mecanismos de robustez (fallbacks): si la suma directa produce un valor no finito (por ejemplo, debido a artefactos en la captura), el sistema intenta realizar una integración trapezoidal como segundo recurso.
- Adicionalmente, para la banda TDT de Colombia (cuyas capturas tienen un span muy ancho), se decidió utilizar la media del espectro en lugar de la suma directa para calcular la potencia integrada, logrando así mayor estabilidad numérica.

7.12.4. Estimación de MER y BER

Para las señales de televisión digital (TDT), se diseñó un flujo que estima la Tasa de Error de Bit (BER) asumiendo una modulación M-QAM (con $M = 64$), partiendo de la relación señal a ruido (SNR) medida en banda. El diseño maneja este cálculo de forma estrictamente opcional; si la captura no ofrece un nivel de SNR suficiente para realizar una estimación matemática confiable, el campo simplemente se omite sin forzar resultados inválidos.

7.13. 13. Problemas encontrados y ajustes realizados

Durante la fase de desarrollo y despliegue del módulo, el principal reto técnico estuvo relacionado con la etapa de detección de emisiones y el control de falsas alarmas derivadas de la variabilidad del piso de ruido. La evolución del algoritmo pasó por varias iteraciones hasta llegar a la solución definitiva que balancea precisión y consumo de recursos.

7.13.1. Evolución de la detección del piso de ruido

Para minimizar los falsos positivos (ruido interpretado como señal), se exploraron metodologías documentadas en la literatura como detectores CFAR (Tasa de Falsa Alarma Constante), pero la implementación atravesó los siguientes ajustes:

1. Piso de ruido fijo basado en histograma (Iteración inicial): Inicialmente, se planteó utilizar un umbral fijo calculado a partir de la moda del histograma de amplitudes del espectro. **Problema:** Se evidenció una fuerte no linealidad en el piso de ruido capturado por el hardware (curvaturas espectrales naturales del receptor). Un umbral fijo y completamente plano provocaba que regiones con ruido curvo cruzaran el límite, generando un alto índice de falsas alarmas, mientras que otras emisiones reales quedaban enmascaradas.* **2. Umbral adaptativo por ventanas (Segunda iteración):** Para compensar la curvatura, se exploró un enfoque dinámico empleando ventanas deslizantes que estimaban el ruido de forma local. **Problema:** Aunque este enfoque funcionó excepcionalmente bien para aislar portadoras de banda angosta, fracasó al procesar señales de banda ancha (como los canales OFDM de la TDT). Debido al comportamiento extendido y plano de estas señales, la ventana deslizante terminaba asimilando la propia señal ancha como si fuera el nuevo piso de ruido, "tragándose" la emisión y abortando la detección.* **3. Mínimo relativo + Umbral de separación (Tercera iteración):** Para solucionar los problemas mixtos entre banda angosta y ancha, se desarrolló un método intermedio que buscaba un nivel mínimo basal en el espectro y le sumaba un umbral de seguridad, lo cual estabilizó la detección combinada.

7.13.2. Ajuste final por restricciones de Hardware (Raspberry Pi 5)

A pesar de haber logrado una detección algorítmicamente robusta, las pruebas de integración revelaron un cuello de botella crítico: los limitados recursos de procesamiento del nodo en el borde (Raspberry Pi 5). El cálculo constante de pisos de ruido adaptativos mediante ventaneos y búsquedas locales resultaba demasiado costoso en CPU para procesar tramas espectrales en tiempo real de forma sostenida.

Solución definitiva: *Se descartó el análisis por ventanas complejas. En su lugar, se implementó un algoritmo de **corrección de linealidad del sistema** (aplicado globalmente a la traza) seguido de la aplicación de un **umbral fijo de detección**. Al corregir matemáticamente las curvaturas o defectos de ganancia del hardware antes del análisis, el piso de ruido queda lo suficientemente aplanado (linealizado) como para que un umbral fijo funcione de manera confiable. Esta decisión sacrificó un leve porcentaje de sensibilidad teórica a cambio de liberar drásticamente los recursos computacionales de la Raspberry Pi 5, garantizando la estabilidad operativa del módulo continuo.*

7.14. 14. Pruebas o validaciones sugeridas

Durante las fases iniciales de desarrollo y sintonización del algoritmo, la validación del módulo dependió principalmente de inspecciones visuales. Este enfoque consistió en graficar las trazas espectrales en crudo y superponer las regiones detectadas, los umbrales calculados y los centros energéticos (F_c). Aunque la inspección visual es fundamental para confirmar cualitativamente el comportamiento del software frente al ruido y las curvaturas del espectro, un despliegue en producción requiere transicionar hacia un marco de validación cuantitativo y automatizado.

Para garantizar la estabilidad del sistema a largo plazo, se recomienda implementar la siguiente estrategia de pruebas y métricas de evaluación:

7.14.1. 1. Estrategia de pruebas automatizadas

<i>Tipo de Prueba</i>	<i>Descripción y Objetivo</i>
<i>Pruebas con Señales Sintéticas</i>	<i>Inyectar al procesador archivos JSON generados matemáticamente (con portadoras, anchos de banda, potencias y ruido blanco gaussiano conocidos o ground truth). Esto permite evaluar si el módulo mide exactamente lo que se inyectó.</i>
<i>Pruebas Unitarias</i>	<i>Validar de forma aislada las funciones matemáticas críticas (como la conversión de dominio logarítmico a lineal en <code>to_power_w()</code>, o los cálculos del <code>channel_power_dbm_uniform_bins</code>) para asegurar que no existan desbordamientos o valores NaN.</i>
<i>Pruebas de Casos Frontera (Edge Cases)</i>	<i>Procesar capturas con condiciones extremas: espectros completamente vacíos (solo ruido), espectros saturados, JSONs incompletos y escenarios con múltiples códigos DANE para comprobar la respuesta del enrutador y del bloque de cumplimiento.</i>
<i>Pruebas de Estrés (Carga)</i>	<i>Simular la llegada concurrente de cientos de capturas por minuto a la API (<code>server_flask.py</code>) para evaluar cómo maneja el sistema la memoria y el procesamiento sin encolarse.</i>

7.14.2. 2. Métricas de evaluación recomendadas

Para sustituir la evaluación netamente visual, se recomienda incorporar métricas estadísticas que califiquen el desempeño de la detección y la precisión de la medición:

Métricas de Detección (Clasificación)

Para medir qué tan bien el algoritmo diferencia una emisión real del piso de ruido:

- **Tasa de Falsos Positivos (FPR):** Porcentaje de veces que el algoritmo detecta un pico de ruido y lo clasifica erróneamente como una emisión candidata.

- **Tasa de Falsos Negativos (FNR):** Porcentaje de emisiones reales y presentes en el espectro que el algoritmo ignoró o fusionó incorrectamente.
- **F1-Score:** Media armónica entre la precisión y la exhaustividad (recall) de las detecciones. Un F1-Score cercano a 1 indicará un balance óptimo entre sensibilidad y robustez al ruido.

Métricas de Precisión de Medida (Regresión)

Para medir qué tan exactos son los parámetros técnicos extraídos, utilizando el Error Absoluto Medio (MAE) respecto a señales de referencia controladas:

- **Error en Frecuencia Central (ΔF_c MAE):** Promedio de la desviación absoluta en Hz entre la frecuencia detectada y el valor nominal de la portadora de prueba.
- **Error en Ancho de Banda (ΔBW MAE):** Diferencia promedio en kHz entre el OBW estimado por el módulo y el ancho de banda real ocupado por la señal sintética.
- **Error en Potencia (ΔP MAE):** Diferencia absoluta promedio en dBm entre la potencia calculada por integración y la energía real inyectada. Debe mantenerse idealmente por debajo de ± 1 dB.

Métricas de Rendimiento del Sistema

- **Latencia de Procesamiento:** Tiempo promedio (en milisegundos) que transcurre desde que la función `process_input` recibe la traza hasta que retorna el diccionario de results.
- **Consumo Pico de CPU/RAM:** Fundamental para asegurar que el procesamiento cruzado multijurisdicción (múltiples DANE) no provoque caídas por falta de memoria o bloquee la ejecución de otros servicios.

7.15. 15. Limitaciones actuales

A pesar de las optimizaciones implementadas en el algoritmo de detección y medición, el módulo de postprocesamiento enfrenta actualmente limitaciones operativas derivadas de la arquitectura de hardware, la infraestructura de red y la naturaleza del análisis espectral clásico.

A continuación, se detallan las principales barreras identificadas en esta fase del proyecto:

7.15.1. 1. Restricciones de capacidad de procesamiento

El diseño actual delega el postprocesamiento a la nube, lo que impone una restricción directa ligada a la capacidad y los costos de cómputo del servidor centralizado. Aunque una solución lógica sería migrar este procesamiento al borde (Edge Computing) ejecutando el algoritmo directamente en la Raspberry Pi de cada nodo, esta alternativa se encuentra fuertemente limitada por la disponibilidad de recursos (CPU y RAM) en el hardware remoto, lo que podría comprometer la estabilidad del nodo al competir con los procesos de adquisición SDR.

7.15.2. 2. Saturación del canal de transmisión (Ancho de Banda)

Incluso si se lograran liberar los recursos en la Raspberry Pi para procesar o preprocesar la información localmente, el sistema se enfrenta a un cuello de botella en la red. El canal de comunicación por el que se envían los datos espectrales a la nube se encuentra actualmente saturado. Esta limitación restringe la resolución, la tasa de refresco y el volumen de capturas que pueden ser transmitidas y procesadas en tiempo real.

7.15.3. 3. Estimación del umbral en entornos con baja SNR

En entornos altamente ruidosos, la estimación del piso de ruido —y por consiguiente, el umbral de detección— se ve significativamente afectada por la distribución de la energía en el espectro. Aunque el software incluye un algoritmo adaptativo que toma en cuenta la Relación Señal a Ruido (SNR) para ajustar los márgenes de detección, este modelo sigue siendo susceptible a errores de estimación de SNR cuando las emisiones legítimas quedan enmascaradas por el ruido de fondo, provocando fluctuaciones en la tasa de detección.

7.16. 16. Recomendaciones de mejora

Para superar las limitaciones actuales y escalar la capacidad del sistema de monitoreo, se plantean las siguientes directrices de evolución tecnológica y algorítmica:

7.16.1. Mejora de la infraestructura de comunicaciones

La optimización del ancho de banda del canal de subida desde los nodos hacia la nube es prioritaria. Aumentar esta capacidad no solo permitiría transmitir espectrogramas de mayor resolución al módulo de procesamiento central, sino que resolvería un problema estructural que beneficiaría el flujo de datos general del sistema, habilitando diagnósticos más precisos.

7.16.2. Exploración de algoritmos basados en Inteligencia Artificial (IA)

*Para resolver la susceptibilidad algorítmica frente a entornos ruidosos y variaciones de SNR, el estado del arte sugiere abandonar las técnicas de estimación estadística clásica en favor de modelos de **Machine Learning** y **Deep Learning**. El entrenamiento de redes neuronales convolucionales (CNN) o autoencoders para la supresión de ruido y la identificación de patrones espectrales permitiría una detección de emisiones altamente robusta y una estimación de parámetros independiente de las curvaturas del receptor. Hasta el punto actual del proyecto, estas metodologías no se han explorado.*

7.16.3. Actualización del hardware de procesamiento (Hardware Upgrade)

La transición hacia técnicas de Deep Learning, e incluso la mitigación de los cuellos de botella en la nube mediante Edge Computing, introduce una nueva dependencia. Para que estas estrategias sean viables, se requerirá dotar a la arquitectura (ya sea en los servidores o en los nodos de monitoreo) de hardware lo suficientemente potente, como aceleradores tensoriales (TPUs) o unidades de procesamiento neuronal (NPUs), que soporten la carga de inferencia en tiempo real sin comprometer el flujo del sistema.

7.17. 17. Cierre técnico de la sección

El desarrollo e implementación del módulo de postprocesamiento espectral representa un avance fundamental en la arquitectura del sistema de monitoreo. A lo largo de esta sección, se ha documentado cómo el software logra cerrar la brecha entre la adquisición cruda de radiofrecuencia en el borde y la toma de decisiones en la plataforma central.

Al transformar matrices de amplitudes y frecuencias en parámetros estructurados (frecuencia central, ancho de banda, potencia y estimaciones de calidad como MER/BER), el módulo dota de significado analítico a las trazas espectrales. Entre los hitos técnicos más relevantes de esta etapa del proyecto, destacan:

*1. **Flexibilidad operativa:** La implementación de un enrutador inteligente que permite conmutar entre modos de análisis (exploración libre, búsqueda de picos específicos o validación normativa) utilizando un mismo núcleo algorítmico, evitando la duplicidad de código. 2. **Rigor matemático:** La decisión de forzar conversiones al dominio lineal para las integraciones de potencia y la priorización del Ancho de Banda Ocupado (OBW), lo cual garantiza métricas confiables incluso ante señales asimétricas o de banda ancha como la TDT. 3. **Adaptabilidad geográfica:** La integración de un sistema robusto de emparejamiento con bases de licencias oficiales, capaz de resolver ambigüedades administrativas*

y procesar validaciones multijurisdiccionales (múltiples códigos DANE) con alta eficiencia computacional. 4. **Pragmatismo frente al hardware:** La adopción de un algoritmo de linealización y umbral fijo que, si bien es más tradicional, logró resolver el cuello de botella crítico de procesamiento en la Raspberry Pi 5, garantizando la estabilidad operativa continua del sistema actual.

Perspectiva a futuro El diseño modular implementado (separando el parser, la representación interna en el *SpectrumFrame* y el motor de análisis) asegura que el código actual no sea un producto estático, sino una base escalable. Como se expuso en las limitaciones y recomendaciones, el sistema ha llegado al tope de las capacidades de las técnicas estadísticas clásicas dadas las restricciones de red y de hardware actuales.

El siguiente paso evolutivo para este componente será la transición hacia modelos de Inteligencia Artificial (Deep Learning) orientados a la clasificación y supresión de ruido en entornos de baja SNR, así como la posible migración de la carga de inferencia parcial hacia el borde (Edge Computing). Con la estructura actual, estos nuevos motores podrán integrarse o sustituir a los actuales sin alterar el flujo de las interfaces de consumo (API o Consola), asegurando que el sistema de monitoreo mantenga su operatividad mientras se actualiza hacia tecnologías de nueva generación.

Capítulo 8

Protocolos de prueba edge-cloud

8.1. Objetivo del capítulo

Este capítulo presenta el protocolo de pruebas propuesto para validar el funcionamiento integral de la plataforma de monitoreo espectral ANE-HX, considerando tanto el componente de borde *edge* –sensor, receptor SDR, procesamiento local, almacenamiento temporal y telemetría– como el componente *cloud* –plataforma web, gestión de campañas, almacenamiento centralizado, visualización, reportes y administración de usuarios–.

El objetivo no es únicamente verificar que cada módulo funcione de manera aislada, sino comprobar que el flujo completo de monitoreo espectral sea trazable, repetible, técnicamente coherente y útil para procesos de vigilancia, análisis y transferencia de conocimiento.

Desde el enfoque pedagógico del informe final, este capítulo busca que un nuevo integrante del equipo pueda entender qué se debe probar, por qué se debe probar, cómo debe ejecutarse cada prueba y qué evidencia debe conservarse para respaldar los resultados obtenidos.

8.2. Alcance de las pruebas

El protocolo cubre las pruebas necesarias para verificar que el sistema pueda ejecutar campañas de monitoreo espectral, adquirir datos de radiofrecuencia, procesar señales localmente, transmitir resultados hacia la plataforma central, visualizar mediciones, generar históricos, exportar información y soportar operación remota.

El alcance incluye los siguientes aspectos:

- Configuración de sensores y selección de antenas.
- Parametrización de frecuencia central, ancho de banda de monitoreo, sample rate, tamaño FFT, RBW, VBW y umbrales.
- Adquisición de datos IQ y estimación de densidad espectral de potencia.

- Validación de detección de emisiones.
- Cálculo de métricas espectrales: frecuencia central, ancho de banda, potencia integrada, ancho de banda ocupado y desviación respecto a valores nominales.
- Evaluación de conectividad entre sensor y plataforma.
- Supervisión de variables de salud del sensor: CPU, RAM, disco, temperatura, estado del proceso SDR, GPS, NTP y red.
- Programación y ejecución de campañas de monitoreo.
- Almacenamiento local y centralizado de datos.
- Exportación de resultados.
- Validación de roles, usuarios, trazabilidad y reportes.

No todas las pruebas descritas en este capítulo deben entenderse como pruebas de aceptación contractual estricta. Algunas corresponden a pruebas funcionales, otras a pruebas técnicas de ingeniería, otras a pruebas de diagnóstico y otras a pruebas pedagógicas orientadas a la transferencia de conocimiento.

8.3. Arquitectura bajo prueba

La arquitectura bajo prueba se entiende como una cadena completa de adquisición, procesamiento, transmisión, almacenamiento y análisis. El sistema puede representarse conceptualmente mediante los siguientes bloques:

1. **Frente de radiofrecuencia:** antenas, conmutador RF, conectores, acoples, filtros y receptor SDR.
2. **Nodo edge:** computador embebido, receptor SDR, motor de adquisición, procesamiento local, almacenamiento temporal y telemetría.
3. **Canal de comunicación:** Ethernet, WiFi o red móvil, con verificación de disponibilidad, latencia y estabilidad.
4. **Backend institucional o cloud:** recepción de datos, base de datos, administración de campañas, gestión de sensores y procesamiento adicional.
5. **Interfaz de usuario:** visualización espectral, mapa, campañas, historial, alertas, usuarios, reportes y exportaciones.

La validación debe comprobar que estos bloques funcionen de manera coordinada. Un sensor puede adquirir correctamente, pero si no transmite datos con timestamp, geolocalización, identificador único y parámetros de adquisición, la medición pierde trazabilidad. De forma equivalente, una interfaz puede visualizar datos, pero si no conserva los parámetros técnicos de captura, no permite auditoría posterior ni reproducción del análisis.

8.4. Criterios generales de aceptación

Para cada prueba se deben registrar, como mínimo, los siguientes elementos:

- Identificador de prueba.
- Módulo evaluado.
- Requerimiento asociado.
- Condiciones iniciales.
- Procedimiento paso a paso.
- Resultado esperado.
- Resultado observado.
- Evidencia: captura, archivo JSON, archivo CSV, log, gráfica, reporte o fotografía.
- Estado: aprobada, aprobada con observaciones, fallida o no ejecutada.
- Responsable y fecha de ejecución.

Se recomienda clasificar los resultados de la siguiente manera:

- **Aprobada:** el comportamiento observado coincide con el resultado esperado.
- **Aprobada con observaciones:** la funcionalidad opera, pero existen restricciones, desviaciones menores o recomendaciones.
- **Fallida:** el sistema no cumple el criterio definido o presenta comportamiento no reproducible.
- **No ejecutada:** la prueba no pudo realizarse por falta de equipo, datos, acceso o condiciones controladas.

8.5. Estrategia general de pruebas

La estrategia propuesta combina pruebas funcionales, pruebas de integración, pruebas metrológicas, pruebas de operación remota y pruebas pedagógicas. Esta división permite diferenciar entre errores de software, fallas de comunicación, limitaciones del hardware SDR y problemas de operación.

8.5.1. Pruebas funcionales

Las pruebas funcionales verifican que las opciones visibles para el usuario operen de acuerdo con lo esperado. Incluyen inicio de sesión, selección de sensores, configuración de campañas, visualización de mediciones, consulta de históricos, generación de reportes y exportación de datos.

8.5.2. Pruebas de integración edge

Las pruebas de integración edge evalúan la interacción entre el hardware y el software local del sensor. Incluyen comunicación con el SDR, selección de antenas, adquisición de muestras IQ, generación de PSD, almacenamiento local, lectura de GPS, sincronización horaria y monitoreo de recursos del sistema.

8.5.3. Pruebas de integración edge-cloud

Las pruebas de integración edge-cloud verifican la comunicación entre el sensor y la plataforma central. Incluyen registro del sensor, envío de telemetría, envío de resultados espectrales, ejecución remota de campañas, recuperación ante pérdida de conectividad y consistencia de timestamps.

8.5.4. Pruebas metrológicas y espectrales

Las pruebas metrológicas y espectrales permiten conocer los límites reales del sistema de medición. Incluyen sensibilidad, DANL, rango dinámico, exactitud de frecuencia, precisión de amplitud, estabilidad del piso de ruido, medición de ancho de banda y detección de emisiones.

8.5.5. Pruebas pedagógicas

Las pruebas pedagógicas verifican que los procedimientos puedan ser reproducidos por nuevos usuarios técnicos. El objetivo es que el informe final no sea solamente un documento de cierre, sino también una guía de transferencia de conocimiento para operación, mantenimiento y evolución de la plataforma.

8.6. Matriz general de pruebas

La Tabla 8.1 presenta una matriz general de pruebas recomendadas para el sistema edge-cloud. Los estados deben actualizarse de acuerdo con la evidencia real disponible.

Tabla 8.1: Matriz general de pruebas edge-cloud.

ID	Prueba	Módulo	Resultado esperado
PR-EC-001	Registro e identificación del sensor	Plataforma y edge	El sensor aparece registrado con identificador único, ubicación, estado y metadatos técnicos.
PR-EC-002	Heartbeat del sensor	Edge-cloud	El sensor reporta periódicamente estado de CPU, RAM, disco, temperatura, red, GPS y proceso SDR.
PR-EC-003	Verificación de conectividad	Red	La plataforma identifica si el sensor está en línea o fuera de línea y registra latencia cuando aplique.
PR-EC-004	Sincronización temporal	Edge, NTP y GPS	Los timestamps de adquisición son coherentes con la hora del sistema.
PR-EC-005	Selección de sensor activo	Interfaz	El usuario puede seleccionar un sensor disponible y habilitar el panel de adquisición.
PR-EC-006	Selección de antena	Hardware RF	La antena seleccionada corresponde al puerto RF activado.

ID	Prueba	Módulo	Resultado esperado
PR-EC-007	Configuración de frecuencia central	SDR	El sistema permite configurar una frecuencia válida dentro del rango técnico del hardware.
PR-EC-008	Configuración de sample rate	SDR y DSP	El sistema acepta valores permitidos y rechaza configuraciones fuera del rango soportado.
PR-EC-009	Configuración de FFT y RBW	DSP	El RBW mostrado coincide con la relación entre sample rate y tamaño FFT.
PR-EC-010	Adquisición IQ local	Edge SDR	El sensor captura muestras IQ y registra una salida válida sin corrupción.
PR-EC-011	Cálculo de PSD	DSP edge	La PSD generada presenta eje de frecuencia, unidades de potencia y número de bins coherentes.
PR-EC-012	Visualización espectral	Interfaz cloud	La plataforma muestra la traza espectral correspondiente al sensor y configuración seleccionados.
PR-EC-013	Detección por umbral	Algoritmo espectral	El sistema diferencia piso de ruido y ocupación de señal mediante un umbral configurable.
PR-EC-014	Medición de frecuencia central	Algoritmo espectral	La frecuencia medida de una emisión conocida se mantiene dentro de la tolerancia definida.
PR-EC-015	Medición de ancho de banda	Algoritmo espectral	El sistema calcula ancho de banda por método X dB y por OBW cuando aplique.
PR-EC-016	Medición de potencia de canal	Algoritmo espectral	El sistema integra potencia sobre el ancho de canal definido y reporta un valor trazable.
PR-EC-017	Corrección de DC offset	DSP edge	La componente central espuria se reduce respecto a la señal cruda.

ID	Prueba	Módulo	Resultado esperado
PR-EC-018	Corrección de balance IQ	DSP edge	Se reduce la imagen espectral y mejora la calidad de la PSD.
PR-EC-019	Ejecución de campaña programada	Plataforma y edge	La campaña se ejecuta en el horario configurado y genera resultados históricos.
PR-EC-020	Cancelación de campaña	Plataforma	Un usuario autorizado puede cancelar una campaña programada antes de su ejecución.
PR-EC-021	Persistencia ante pérdida de red	Edge	Si se pierde conectividad, el sensor conserva resultados temporalmente cuando sea posible.
PR-EC-022	Almacenamiento centralizado	Backend	Los resultados quedan asociados a sensor, timestamp, ubicación y parámetros de adquisición.
PR-EC-023	Consulta de histórico	Plataforma	El usuario puede consultar campañas y mediciones anteriores con filtros básicos.
PR-EC-024	Exportación CSV y JSON	Plataforma	La plataforma exporta resultados completos y legibles para procesamiento offline.
PR-EC-025	Validación de roles	Seguridad	Los permisos dependen del rol del usuario.
PR-EC-026	Alertas de salud del sensor	Plataforma y telemetría	La plataforma genera alertas cuando una variable supera un umbral definido.
PR-EC-027	Prueba de mapa geográfico	Interfaz y geolocalización	El sensor aparece ubicado correctamente en el mapa.
PR-EC-028	Prueba de DANL	Caracterización RF	Se documenta el nivel de ruido propio del sistema por banda o configuración.
PR-EC-029	Prueba de rango dinámico	Caracterización RF	Se identifica el intervalo entre señal mínima detectable y máxima señal sin distorsión evidente.

ID	Prueba	Módulo	Resultado esperado
PR-EC-030	Prueba de exactitud de frecuencia	Caracterización RF	La diferencia entre frecuencia inyectada y frecuencia medida queda dentro de la tolerancia definida.
PR-EC-031	Prueba de precisión de amplitud	Caracterización RF	La diferencia entre potencia de referencia y potencia medida se registra en dB.
PR-EC-032	Prueba de documentación pedagógica	Transferencia de conocimiento	Un usuario técnico puede reproducir una prueba siguiendo el procedimiento documentado.

8.7. Protocolos detallados de prueba

8.7.1. PR-EC-001: registro e identificación del sensor

Objetivo: verificar que cada sensor pueda ser registrado en la plataforma con un identificador único y metadatos suficientes para trazabilidad.

Condiciones iniciales:

- Plataforma cloud activa.
- Usuario con permisos de administración.
- Sensor energizado y con conectividad disponible.

Procedimiento:

1. Ingresar a la plataforma con usuario administrador.
2. Acceder al módulo de configuración o registro de dispositivos.
3. Crear o editar un sensor.
4. Asociar nombre, identificador, ubicación, antenas disponibles y descripción técnica.
5. Guardar la configuración.
6. Verificar que el sensor aparezca en el listado general.

Resultado esperado: el sensor queda registrado, aparece disponible para campañas y conserva sus metadatos técnicos.

Evidencia sugerida: captura del formulario de registro, captura del listado de sensores y registro en base de datos.

8.7.2. PR-EC-002: heartbeat y salud del sensor

Objetivo: comprobar que el nodo edge reporte periódicamente su estado operativo.

Variables mínimas a registrar:

- Estado en línea o fuera de línea.
- Uso de CPU.
- Uso de RAM.
- Uso de disco.
- Temperatura.
- Estado del proceso SDR.
- Estado de red.
- Estado GPS o coordenadas.
- Hora del sistema.

Procedimiento:

1. Encender el sensor.
2. Verificar conectividad IP.
3. Ingresar al panel de sensores.
4. Observar la actualización periódica del estado.
5. Desconectar temporalmente la red del sensor.
6. Verificar que la plataforma detecte el cambio de estado.
7. Reconectar el sensor.
8. Confirmar recuperación del estado en línea.

Resultado esperado: la plataforma refleja correctamente el estado operativo del sensor y permite detectar fallas de conectividad o recursos.

8.7.3. PR-EC-003: configuración de adquisición SDR

Objetivo: validar que la plataforma pueda enviar parámetros de adquisición al sensor y que el sensor los aplique correctamente.

Parámetros bajo prueba:

- Frecuencia central.
- Ancho de banda de monitoreo.
- Sample rate.
- Ganancia de recepción, si aplica.
- Tamaño FFT.
- Antena seleccionada.
- Duración de adquisición.

Procedimiento:

1. Seleccionar un sensor en línea.
2. Seleccionar una antena compatible con la banda de prueba.
3. Configurar una frecuencia central conocida.
4. Configurar sample rate y tamaño FFT.
5. Iniciar adquisición.
6. Revisar logs del sensor.
7. Verificar que la configuración aplicada coincide con la configuración solicitada.

Resultado esperado: el sensor aplica los parámetros y genera una medición válida sin errores de configuración.

8.7.4. PR-EC-004: validación de RBW

Objetivo: verificar que el valor de RBW mostrado por la plataforma sea coherente con los parámetros DSP.

El RBW debe calcularse como:

$$RBW = \frac{F_s}{N_{FFT}}$$

donde F_s es el sample rate y N_{FFT} es el tamaño de la FFT.

Procedimiento:

1. Configurar un sample rate conocido.
2. Configurar un tamaño FFT conocido.
3. Iniciar adquisición.
4. Consultar el RBW mostrado en la interfaz o archivo de salida.
5. Calcular manualmente el RBW.
6. Comparar ambos valores.

Resultado esperado: el RBW reportado coincide con el cálculo teórico, dentro de una tolerancia asociada al redondeo de visualización.

8.7.5. PR-EC-005: adquisición IQ y generación de PSD

Objetivo: comprobar que el sensor capture datos IQ y genere una PSD correctamente indexada en frecuencia.

Procedimiento:

1. Configurar una banda con actividad espectral conocida.
2. Ejecutar una adquisición.
3. Confirmar la existencia de archivo IQ, buffer o salida procesada.
4. Verificar número de muestras.
5. Generar PSD.

6. Confirmar que el eje de frecuencia corresponde a la frecuencia central y al ancho de banda configurados.
7. Verificar que la potencia se encuentre en unidades consistentes.

Resultado esperado: la PSD debe contener vector de frecuencias, vector de potencia, metadatos de adquisición y timestamp.

8.7.6. PR-EC-006: detección de emisiones por umbral

Objetivo: validar que el algoritmo pueda distinguir entre piso de ruido y emisiones presentes.

Procedimiento:

1. Capturar una banda con al menos una emisión visible.
2. Estimar el piso de ruido.
3. Definir un umbral de detección, por ejemplo $NF + 3$ dB o $NF + 5$ dB.
4. Identificar bins por encima del umbral.
5. Agrupar bins contiguos como emisiones.
6. Registrar frecuencia central, ancho de banda y potencia.

Resultado esperado: el algoritmo detecta emisiones reales y evita clasificar fluctuaciones aleatorias de ruido como señales.

8.7.7. PR-EC-007: medición de ancho de banda ocupado

Objetivo: comprobar que el sistema mida ancho de banda usando criterios X dB y OBW.

Procedimiento:

1. Seleccionar una emisión estable.
2. Calcular el punto de máxima potencia.
3. Aplicar el método X dB para $X = 3$, $X = 10$ o $X = 26$, según configuración.
4. Calcular OBW al 99 % integrando potencia acumulada.

5. Comparar ambos resultados.
6. Registrar diferencias y condiciones de medición.

Resultado esperado: el sistema reporta ancho de banda de forma consistente y diferencia explícitamente el método usado.

8.7.8. PR-EC-008: programación y ejecución de campañas

Objetivo: verificar que una campaña programada desde la plataforma sea ejecutada por el sensor correspondiente.

Procedimiento:

1. Crear una campaña con sensor, antena, frecuencia, ancho de banda, duración y periodicidad.
2. Guardar la campaña.
3. Verificar que aparece como programada.
4. Esperar la hora de ejecución.
5. Confirmar inicio de adquisición en el sensor.
6. Verificar recepción de datos en la plataforma.
7. Revisar histórico de campaña.

Resultado esperado: la campaña se ejecuta automáticamente y genera registros consultables.

8.7.9. PR-EC-009: recuperación ante pérdida de conectividad

Objetivo: evaluar la tolerancia del sistema ante desconexiones temporales entre edge y cloud.

Procedimiento:

1. Iniciar una campaña o adquisición.
2. Interrumpir temporalmente la conectividad del sensor.
3. Verificar que el sensor no pierda la adquisición local, si el diseño lo permite.

4. Restaurar la conectividad.
5. Confirmar si los resultados pendientes son transmitidos.
6. Revisar logs de error y recuperación.

Resultado esperado: el sistema debe registrar la pérdida de conexión, conservar datos críticos y recuperar operación sin intervención manual, cuando sea técnicamente posible.

8.7.10. PR-EC-010: exportación de datos

Objetivo: validar que los resultados puedan descargarse para análisis offline.

Formatos mínimos sugeridos:

- CSV para análisis tabular.
- JSON para interoperabilidad técnica.
- XML si se requiere integración con sistemas institucionales.

Procedimiento:

1. Seleccionar una campaña terminada.
2. Descargar resultados en CSV.
3. Descargar resultados en JSON.
4. Verificar que los archivos incluyan sensor, timestamp, coordenadas, parámetros de adquisición y resultados espectrales.
5. Abrir los archivos en una herramienta externa.

Resultado esperado: los archivos exportados son legibles, completos y consistentes con la medición mostrada en plataforma.

8.8. Pruebas metrológicas recomendadas

8.8.1. DANL

El DANL permite estimar el nivel promedio de ruido mostrado por el sistema en ausencia de señales intencionales. Esta prueba es fundamental para conocer la sensibilidad práctica del sistema.

Procedimiento sugerido:

1. Terminar el puerto RF en $50\ \Omega$.
2. Configurar frecuencia central y sample rate.
3. Realizar varias capturas.
4. Promediar la PSD.
5. Registrar el nivel de ruido por banda.
6. Repetir para distintas frecuencias representativas.

Resultado esperado: se obtiene una curva o tabla de DANL por frecuencia y configuración.

8.8.2. Exactitud de frecuencia

Procedimiento sugerido:

1. Inyectar una señal senoidal de frecuencia conocida.
2. Configurar el sensor alrededor de esa frecuencia.
3. Medir la frecuencia de pico.
4. Calcular el error absoluto y relativo en ppm.

$$Error_{Hz} = f_{medida} - f_{referencia}$$

$$Error_{ppm} = \frac{f_{medida} - f_{referencia}}{f_{referencia}} \times 10^6$$

Resultado esperado: el error de frecuencia queda documentado y puede compararse entre sensores.

8.8.3. Precisión de amplitud

Procedimiento sugerido:

1. Inyectar una señal de potencia conocida.
2. Medir la potencia reportada por el sistema.
3. Repetir para varios niveles de potencia.
4. Calcular error en dB.

$$Error_{dB} = P_{medida} - P_{referencia}$$

Resultado esperado: se obtiene una tabla de error de amplitud por nivel de potencia.

8.8.4. Rango dinámico

Procedimiento sugerido:

1. Estimar el DANL del receptor.
2. Inyectar señales de amplitud creciente.
3. Identificar el nivel máximo antes de compresión, saturación o aparición significativa de espurios.
4. Calcular la diferencia entre el nivel máximo lineal y el nivel mínimo detectable.

Resultado esperado: se documenta el rango dinámico práctico del sistema bajo condiciones de prueba.

8.9. Pruebas de instalación y operación de campo

Las pruebas de campo deben validar que el sensor opere en condiciones reales de instalación. Se recomienda incluir:

- Verificación de alimentación estable.
- Estado de sellado y protección ambiental.

- Correcta instalación de antenas.
- Confirmación de GPS.
- Conectividad IP.
- Prueba de adquisición corta.
- Verificación de reporte en plataforma.
- Registro fotográfico del emplazamiento.

Estas pruebas son importantes porque un sistema puede funcionar correctamente en laboratorio, pero presentar fallas en campo por alimentación inestable, mala conectividad, ubicación deficiente de antenas, ausencia de sincronización temporal o interferencias locales.

8.10. Problemas, decisiones y ajustes esperados

La Tabla [8.2](#) resume problemas típicos que pueden aparecer durante la validación del sistema, junto con causas probables y ajustes recomendados.

Tabla 8.2: Problemas frecuentes durante las pruebas edge-cloud.

Problema	Evidencia	Causa probable	Ajuste recomendado
Pérdida temporal de conectividad del sensor	Sensor aparece fuera de línea o sin heartbeat	Red inestable, pérdida de enlace WiFi, LTE o Ethernet	Implementar persistencia local y reintento de transmisión.
Espurio en frecuencia central	Pico persistente en el centro de la PSD	DC offset del receptor de conversión directa	Aplicar corrección DC o estrategia de sintonía con desplazamiento.
Imágenes espectrales	Componentes reflejadas alrededor de la portadora	Desbalance de amplitud o fase entre ramas I y Q	Aplicar corrección de balance IQ y documentar mejora.
Medición inestable del piso de ruido	Variación fuerte del umbral entre capturas	Ganancia no controlada, entorno RF variable o promediado insuficiente	Fijar ganancia, aumentar promediado y registrar condiciones de prueba.
Inconsistencia entre campaña y datos recibidos	Resultados sin metadatos completos	Falta de asociación entre adquisición, sensor y campaña	Exigir identificador de campaña, sensor, timestamp y configuración en cada paquete.
Errores de usuario en configuración	Frecuencias o anchos de banda fuera de rango	Falta de validación en interfaz	Agregar validaciones y mensajes pedagógicos.

8.11. Evidencias mínimas por prueba

Cada prueba debe acompañarse de evidencia verificable. Se recomienda conservar, como mínimo, los siguientes tipos de evidencia:

- Capturas de pantalla de la interfaz.
- Logs del sensor.
- Logs de backend.
- Archivos de salida en CSV o JSON.
- Gráficas de PSD.

- Fotografías del montaje físico.
- Tabla de resultados observados.
- Comentarios del responsable de la prueba.

Una posible estructura de carpetas para organizar evidencias es la siguiente:

```
section/img/test_protocols/
  PR-EC-001-sensor-registration.png
  PR-EC-002-heartbeat.png
  PR-EC-005-psd-example.png
  PR-EC-008-campaign-execution.png

test-results/
  PR-EC-005-psd-output.json
  PR-EC-010-export.csv
  PR-EC-014-frequency-accuracy.csv
  PR-EC-028-danl-results.csv
```

Para cada evidencia se debe indicar:

- Nombre del archivo.
- Prueba asociada.
- Fecha.
- Responsable.
- Descripción breve.
- Interpretación del resultado.

8.12. Transferencia de conocimiento

El capítulo debe cumplir una función pedagógica: permitir que un nuevo integrante del equipo entienda qué se prueba, por qué se prueba y cómo interpretar los resultados.

Tabla 8.3: Aprendizajes transferibles asociados a las pruebas.

Tema	Aprendizaje obtenido	Prueba asociada
RBW y FFT	El RBW depende directamente del sample rate y del tamaño FFT; no es un parámetro independiente.	PR-EC-004
Piso de ruido y umbral	El umbral debe definirse por encima del piso de ruido para evitar falsos positivos.	PR-EC-006
DC offset	Los SDR de conversión directa pueden presentar espurios en frecuencia central que afectan la detección.	PR-EC-017
Balance IQ	El desbalance IQ genera imágenes espectrales que pueden confundirse con emisiones reales.	PR-EC-018
Trazabilidad	Toda medición debe conservar sensor, timestamp, ubicación y parámetros de adquisición.	PR-EC-022
Flujo edge-cloud	El sistema no se valida únicamente en el sensor ni únicamente en la web; debe validarse el flujo completo.	PR-EC-001 a PR-EC-027
Caracterización RF	DANL, exactitud de frecuencia, precisión de amplitud y rango dinámico permiten conocer los límites reales del sistema.	PR-EC-028 a PR-EC-031

8.13. Recomendaciones para futuras pruebas

Se recomienda que las pruebas futuras incorporen mayor automatización, especialmente en los siguientes aspectos:

- Generación automática de reportes de prueba.
- Ejecución periódica de pruebas de conectividad y heartbeat.
- Validación automática de estructura JSON.
- Comparación automática entre configuración solicitada y configuración aplicada.
- Pruebas con señales sintéticas controladas.
- Pruebas de regresión después de cada actualización del software.
- Banco de pruebas con generador RF, atenuadores calibrados y carga de 50 Ω .
- Registro sistemático de temperatura, ganancia, frecuencia, antena y condiciones ambientales.

También se recomienda separar claramente tres niveles de evidencia:

1. **Evidencia de funcionamiento:** capturas de interfaz, logs y archivos exportados.
2. **Evidencia técnica:** mediciones, gráficas de PSD, cálculos de error y resultados de caracterización.
3. **Evidencia pedagógica:** explicación del procedimiento, interpretación de resultados y recomendaciones de uso.

8.14. Conclusiones del capítulo

El protocolo de pruebas edge-cloud permite validar la plataforma ANE-HX como un sistema integral de monitoreo espectral, no como una suma aislada de componentes. La correcta operación depende de la coordinación entre el receptor SDR, el procesamiento local, la conectividad, la plataforma central, la visualización, el almacenamiento histórico y la capacidad de exportar resultados.

Las pruebas propuestas cubren el flujo completo desde la adquisición RF hasta la consulta de resultados en la plataforma. Además, incorporan pruebas metrológicas necesarias para entender las limitaciones reales del sistema, tales como DANL, rango dinámico, exactitud de frecuencia y precisión de amplitud.

Desde el enfoque pedagógico del informe final, este capítulo debe servir como guía para que futuros usuarios, desarrolladores o funcionarios puedan reproducir pruebas, interpretar evidencias y comprender por qué cada validación es necesaria dentro de un sistema institucional de monitoreo del espectro.

Capítulo 9

Integración general del sistema

9.1. Propósito del capítulo

[[Placeholder]]

9.2. Contexto dentro del sistema general

[[Placeholder]]

9.3. Desarrollo técnico

[[Placeholder]]

9.4. Entradas, salidas e interfaces

[[Placeholder]]

9.5. Decisiones de diseño o implementación

[[Placeholder]]

9.6. Problemas presentados y ajustes realizados

[[Placeholder]]

9.7. Validación, pruebas o evidencias

[[Placeholder]]

9.8. Aprendizajes y transferencia de conocimiento

[[Placeholder]]

9.9. Limitaciones

[[Placeholder]]

9.10. Recomendaciones específicas

[[Placeholder]]

Capítulo 10

Conclusiones generales

10.1. Síntesis de resultados técnicos

[[Placeholder]]

10.2. Logros del diseño e implementación

[[Placeholder]]

10.3. Resultados de la integración

[[Placeholder]]

10.4. Lecciones aprendidas

[[Placeholder]]

10.5. Cumplimiento de objetivos

[[Placeholder]]

Capítulo 11

Recomendaciones

11.1. Mejoras de arquitectura

[[Placeholder]]

11.2. Mejoras de hardware

[[Placeholder]]

11.3. Mejoras de software

[[Placeholder]]

11.4. Mejoras de documentación

[[Placeholder]]

11.5. Reducción de riesgos identificados

[[Placeholder]]

Capítulo 12

Referencias

[[Placeholder]]

Apéndice A

Material de apoyo: uso de agentes de IA para documentación, desarrollo y revisión técnica del proyecto

A.1. Nota de alcance y propósito

Este capítulo no describe un módulo funcional o componente obligatorio del sistema ANE. Se presenta exclusivamente como **material de apoyo metodológico** que documenta las técnicas de asistencia técnica utilizadas por el equipo de trabajo. El objetivo es proporcionar una guía sobre cómo el uso responsable de Inteligencia Artificial (IA) ha apoyado la documentación, revisión de código y organización de la información durante el ciclo de vida del proyecto.

A.2. Contexto y diferenciación funcional

Es fundamental diferenciar entre el uso de la IA como herramienta de apoyo al desarrollo y su posible rol como componente funcional del sistema.

- **IA como herramienta de apoyo:** Metodología aplicada para mejorar la productividad, coherencia documental y calidad del código mediante revisión asistida.
- **IA como componente del sistema:** Funcionalidades de análisis inteligente de señales o gestión autónoma que se plantean como propuestas futuras y no forman parte de la arquitectura operativa actual de ANE.

A.3. Metodología de trabajo: Ecosistema de Agentes

El equipo emplea herramientas de IA aprobadas institucionalmente para asistir en tareas técnicas. Aunque el enfoque es agnóstico al modelo, se documentan a continuación los pasos de habilitación para las herramientas utilizadas como referencia durante el proyecto.

A.3.1. Habilitación del Entorno (Ejemplos CLI)

Para permitir una interacción directa desde la terminal y facilitar la automatización, se utilizaron los siguientes agentes:

- **GitHub Copilot CLI:** Utilizado para asistencia en codificación y gestión de repositorios.
 - Instalación: `npm install -g @github/copilot`
 - Autenticación: `copilot /login`
- **Gemini CLI:** Utilizado para tareas multimodales, análisis de arquitectura y gestión de habilidades.
 - Instalación: `npm install -g @google/gemini-cli`
 - Inicio: `gemini`

A.3.2. Arquitectura Multi-Agente Asistida

La arquitectura multi-agente se define aquí como una **metodología de trabajo asistido**, no como una implementación automática dentro del sistema. Se sugieren los siguientes roles para la colaboración:

- **Agente Documental:** Apoyo en redacción técnica, generación de tablas y coherencia de estilo.
- **Agente de Código:** Revisión de scripts, detección de errores lógicos y sugerencias de optimización.
- **Agente de Pruebas:** Generación de casos de prueba y protocolos de validación.
- **Agente de Arquitectura:** Análisis de dependencias y revisión de diagramas de diseño.
- **Agente de Señales/RF:** Apoyo en el análisis de logs y procesamiento teórico de espectro.

- **Agente de Revisión:** Validación cruzada de respuestas para minimizar alucinaciones.

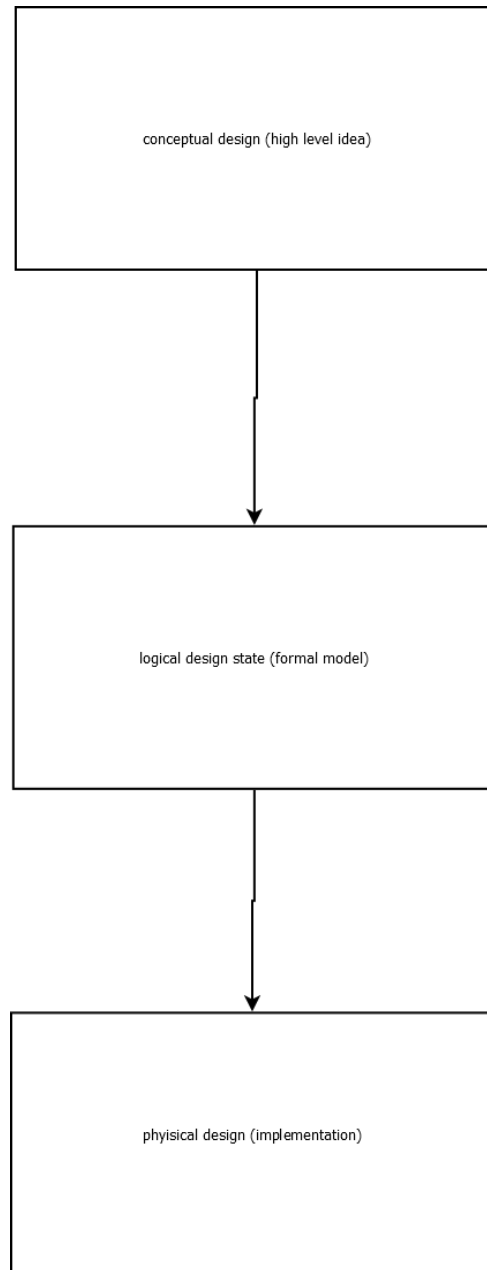


Figura A.1: Diseño conceptual del sistema multi-agente: Relación metodológica entre roles de asistencia. Fuente: Elaboración propia.

A.4. Gobernanza y Uso Responsable de IA

El uso de herramientas de IA en el proyecto ANE está sujeto a las siguientes restricciones de seguridad y ética:

- **Revisión Humana Obligatoria:** Ningún resultado generado por IA se considera definitivo sin la validación técnica de un responsable humano.
- **Confidencialidad:** Está estrictamente prohibido cargar información sensible, credenciales, llaves de API o datos institucionales restringidos en herramientas externas.
- **No Normatividad:** La IA no es una fuente normativa; las decisiones técnicas se basan en estándares de ingeniería y requerimientos del proyecto.
- **Validación de Código:** No se acepta código sugerido sin pruebas unitarias y validación en el entorno de desarrollo.

A.5. Trazabilidad Metodológica

A continuación, se presenta la trazabilidad del uso de IA frente a las actividades de apoyo del proyecto.

Tabla A.1: Trazabilidad de actividades de apoyo con IA.

Actividad de Apoyo	Aplicación en el Proyecto	Estado
Documentación Técnica	Generación de estructuras y tablas iniciales.	Aplicado
Revisión de Código	Análisis de scripts de adquisición edge.	Parcial
Generación de Pruebas	Creación de protocolos de validación HW.	Sugerido
Análisis de Errores	Interpretación de logs de comunicación cloud.	Aplicado
Organización de Info.	Clasificación de requisitos y dependencias.	Recomendado

A.6. Flujo de Trabajo Colaborativo (Paso a Paso)

Para asegurar la calidad y trazabilidad, se sigue el siguiente flujo:

1. **Definir Tarea:** Clarificar el objetivo técnico y las restricciones.
2. **Preparar Contexto:** Seleccionar archivos, logs o requisitos relevantes.

3. **Consultar IA:** Utilizar prompts estructurados (ROLE-OBJECTIVE-CONTEXT).
4. **Revisar Respuesta:** Identificar posibles errores o inconsistencias.
5. **Validar Técnicamente:** Probar el código o verificar los datos en el sistema real.
6. **Integrar y Documentar:** Subir al repositorio (GitHub) indicando la asistencia recibida y quién validó.

A.7. Trazabilidad de Prompts y Relación con GitHub

Cada interacción significativa con la IA debe ser trazable. Se recomienda asociar los prompts importantes a:

- **GitHub Issues:** Documentar el prompt usado para resolver un bug específico.
- **Commits/PRs:** Indicar en el mensaje del commit si se utilizó asistencia de IA y quién realizó la revisión final.
- **Archivo de Evidencias:** Registro del prompt, respuesta y decisión tomada.

A.8. Problemas, Limitaciones y Ajustes

La IA presenta limitaciones inherentes que deben ser gestionadas mediante supervisión constante.

Tabla A.2: Gestión de problemas y limitaciones de la IA.

Problema/Riesgo	Manifestación	Causa	Ajuste/Acción	Resultado
Alucinaciones	Librerías in-existentes o comandos falsos.	Falta de datos en el modelo.	Validación cruzada y búsqueda en docs oficiales.	Eliminación de errores técnicos.
Exposición de Datos	Riesgo de subir credenciales.	Descuido en la preparación de contexto.	Uso de <code>.gitignore</code> y limpieza de prompts.	Integridad de secretos mantenida.
Dependencia	Pérdida de criterio técnico propio.	Uso excesivo sin razonamiento.	Rotación de tareas y revisión por pares.	Mantenimiento del conocimiento.

A.9. Métricas de Utilidad y Evidencias

Para evaluar la efectividad de esta metodología, se consideran las siguientes métricas simples:

- **Tareas Apoyadas:** Número de capítulos o módulos revisados con asistencia.
- **Propuestas Aceptadas/Descartadas:** Porcentaje de sugerencias de IA integradas tras revisión.
- **Tiempo Ahorrado:** Estimación del impacto en la velocidad de documentación y depuración.

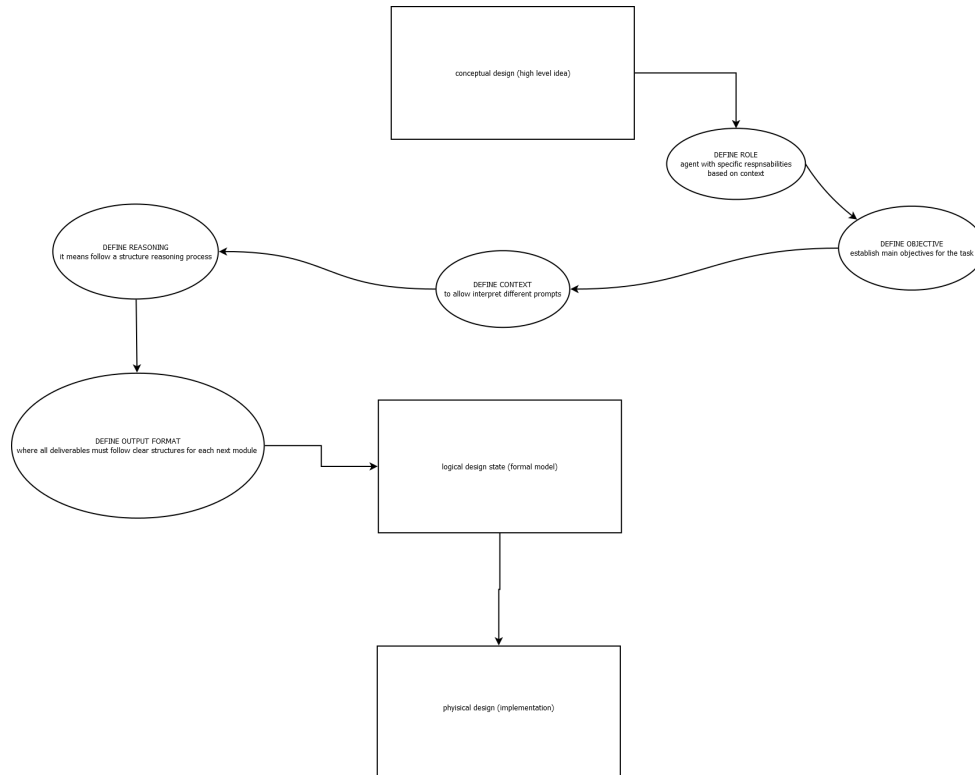


Figura A.2: Flujo de trabajo y validación humana en la integración de resultados. Fuente: Elaboración propia.

A.10. Conclusión del capítulo

La Inteligencia Artificial se propone y utiliza como un multiplicador de productividad para el equipo técnico. Sin embargo, se enfatiza que la responsabilidad final de todas las

decisiones, la seguridad del código y la veracidad de la documentación recae exclusivamente en los ingenieros responsables del proyecto. La validación humana es el pilar central que garantiza la integridad del sistema ANE.

Apéndice B

Diagramas adicionales

B.1. Diagramas de arquitectura extendida

[[Placeholder]]

B.2. Diagramas de despliegue y conectividad

[[Placeholder]]

B.3. Secuencias operativas

[[Placeholder]]

B.4. Esquemas detallados de componentes

[[Placeholder]]

Apéndice C

Manuales o guías de uso

C.1. Requisitos previos de instalación

[[Placeholder]]

C.2. Instrucciones de operación cotidiana

[[Placeholder]]

C.3. Solución de problemas frecuentes

[[Placeholder]]

C.4. Recomendaciones para mantenimiento

[[Placeholder]]

Apéndice D

Resultados de pruebas

D.1. Tablas de métricas y mediciones

[[Placeholder]]

D.2. Evidencias visuales de ejecución

[[Placeholder]]

D.3. Comparación entre escenarios de prueba

[[Placeholder]]

D.4. Observaciones y anomalías detectadas

[[Placeholder]]